



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage



Institut Supérieur des Technologies de
l'Information et de la Communication

RAPPORT DE PROJET DE FIN D'ANNÉE

Présenté en vue de l'obtention de la
LICENCE EN SCIENCES DE L'INFORMATIQUE
Parcours : Génie Logiciel et Systèmes d'Information

Subject:

Design and Development of an Administrative and Online Survey Collection Web Platform

Prepared by: Aziz KARNIT

Conducted at: National Institute of Statistics (INS)



Deposit Authorization of the Project Report:

Professional Supervisor:

Date:

Signature:

To my family.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this project.

I extend my sincere thanks to my professional supervisors at the National Institute of Statistics (INS), **Mrs. Imen Mannai** and **Mr. Hatem Sedghiani**, for their guidance, availability, and continuous support throughout this project. Mrs. Mannai provided invaluable practical advice that shaped the platform's design and implementation, while Mr. Sedghiani's clear explanations of the statistical survey processes were essential in helping me fully understand the subject domain.

I am grateful to the technical staff at the INS for welcoming me and providing access to the operational context necessary for this work.

Above all, I owe an immeasurable debt of gratitude to my **family**, whose love and presence have been my greatest source of strength throughout this journey.

To my **father** — thank you is far too small a phrase for everything you have done. You have been there for me every single step of the way, through every doubt, every late night, and every challenge. Your sacrifices, your encouragement, and your unconditional faith in me are the true foundation of this work. This achievement is yours as much as it is mine.

To my **mother** — you are always in my heart. May your soul rest in eternal peace. (*Inna lillahi wa inna ilayhi raji'un.*)

Finally, I thank all my colleagues and friends for their encouragement and continuous support throughout this academic year.

Abstract

This report presents the design and development of a web platform for administrative tracking and online survey data collection at the National Institute of Statistics (INS) in Tunisia. The existing survey workflow relied on manual spreadsheet processing, physical mail distribution, and synchronous data entry, causing delays and limiting scalability. The proposed platform addresses these issues through a separated architecture: an Angular single-page application frontend, a FastAPI REST API backend, a MySQL relational database, and a Redis/Celery asynchronous worker infrastructure. Key features include secure authentication with two-factor OTP verification, role-based access control for three administrative profiles, asynchronous bulk company import via background task queues, passwordless UUID-based access for external company respondents, real-time WebSocket notifications via Redis Pub/Sub, and comprehensive audit logging with asynchronous Excel export. The project was developed using the Agile Scrum methodology across three development sprints.

Keywords: Web Platform, Survey Collection, FastAPI, Angular, Celery, Asynchronous Processing, UUID Token, WebSocket, Audit Logging.

Résumé

Ce rapport présente la conception et le développement d'une plateforme web pour le suivi administratif et la collecte de données d'enquêtes statistiques en ligne à l'Institut National de la Statistique (INS) en Tunisie. Le système existant reposait sur un traitement manuel des fichiers Excel, une distribution par courrier physique et une saisie synchrone des données, entraînant des retards et limitant la capacité de traitement. La plateforme proposée résout ces problèmes grâce à une architecture découplée : une application monopage Angular, une API REST FastAPI, une base de données relationnelle MySQL et une infrastructure de workers asynchrones Redis/Celery. Les fonctionnalités principales incluent une authentification sécurisée avec vérification OTP à deux facteurs, un contrôle d'accès basé sur les rôles pour trois profils administratifs, une importation en masse asynchrone via des files de tâches, un accès sans mot de passe par jeton UUID pour les contacts externes, des notifications en temps réel par WebSocket via Redis Pub/Sub, et une journalisation complète des audits avec export Excel asynchrone. Le projet a été développé selon la méthodologie Agile Scrum en trois sprints.

Mots-clés : Plateforme Web, Collecte d'Enquêtes, FastAPI, Angular, Celery, Traitement Asynchrone, Jeton UUID, WebSocket, Journalisation d'Audit.

Contents

Acknowledgements	2
Abstract	3
Résumé	4
Table des matières	8
Table des figures	10
Liste des tableaux	11
Liste des abréviations	12
General Introduction	13
1 General Context	15
1.1 Introduction	15
1.2 Host Organization: The INS	15
1.2.1 History and Status	15
1.2.2 Core Mandate and Responsibilities	15
1.2.3 Organizational Structure	16
1.3 Project Overview	16
1.3.1 Analysis of the Existing System	16
1.3.2 Problem Statement	16
1.3.3 Proposed Solution	17
1.3.4 Comparison with Existing Survey Platforms	17
1.4 Preliminary Study and Architectural Choices	18
1.5 Conclusion	20
2 Requirements Specification	21
2.1 Introduction	21
2.2 System Context	21

2.3	Identification of Functional Requirements	21
2.4	Identification of Non-Functional Requirements	22
2.5	Identification of Actors	23
2.6	Global Use Case Diagram	24
2.7	Product Backlog	25
2.8	Work Environment	27
2.8.1	Design Methodology	27
2.8.2	Software Environment	27
2.8.2.1	Development Tools	27
2.8.2.2	Database Tools	28
2.8.2.3	Background Processing & Libraries	29
2.8.2.4	Testing and Validation Tools	30
2.8.2.5	Conception and Modeling Tools	30
2.8.2.6	Collaboration Tools	31
2.9	Conclusion	31
3	Sprint 0 – Authentication & Administrative Core	32
3.1	Introduction	32
3.2	Sprint 0 Backlog Identification	32
3.3	Sprint 0 Refinement	33
3.3.1	Refinement of the Use Case “Login”	34
3.3.2	Refinement of the Use Case “Manage admins”	36
3.3.3	Refinement of the Use Case “Manage contacts”	38
3.3.4	Refinement of the Use Case “Manage companies”	40
3.3.5	Refinement of the Use Case “Manage surveys”	42
3.4	Sprint 0 Conception	44
3.4.1	Class Diagram – Sprint 0 Core Domain	44
3.4.2	Activity Diagram – Use Case “Login”	46
3.4.3	Sequence Diagram – Use Case “Manage admins”	47
3.4.4	Sequence Diagram – Use Case “Manage contacts”	48
3.4.5	Sequence Diagram – Use Case “Manage companies”	49
3.4.6	Activity and State Diagrams – Use Case “Manage surveys”	50
3.5	Sprint 0 Implementation	52
3.5.1	Login Interface	52
3.5.2	Security Architecture	54
3.5.3	Manage admins Interface	56
3.5.4	Manage contacts Interface	57
3.5.5	Manage companies Interface	58
3.5.6	Manage surveys Interface	62

3.6	Sprint 0 Conclusion	66
4	Sprint 1 – Survey Portals & Account Management	67
4.1	Introduction	67
4.2	Sprint 1 Backlog	67
4.3	Sprint 1 Refinement	68
4.3.1	Refinement of the Use Case “Consult assigned surveys”	68
4.3.2	Refinement of the Use Case “Consult assigned results”	70
4.3.3	Refinement of the Use Case “Manage system settings”	71
4.3.4	Refinement of the Use Case “Manage locked accounts”	72
4.3.5	Refinement of the Use Case “Manage profile”	73
4.4	Sprint 1 Conception	74
4.4.1	Class Diagram – Sprint 1 Core Domain	74
4.4.2	Sequence Diagram – Use Case “Consult assigned surveys”	75
4.4.3	Sequence Diagram – Use Case “Consult assigned results”	76
4.4.4	Sequence Diagram – Use Case “Manage system settings”	77
4.4.5	Sequence Diagram – Use Case “Manage locked accounts”	78
4.4.6	State Diagram – Use Case “Manage locked accounts”	79
4.4.7	Sequence Diagram – Use Case “Manage profile”	80
4.5	Sprint 1 Implementation	81
4.5.1	Company Contact Portal & Surveys	81
4.5.2	Consult Assigned Results Interface	82
4.5.3	Manage System Settings Interface	83
4.5.4	Manage Locked Accounts & Security Monitoring	83
4.5.5	Manage Profile Interface	85
4.6	UUID Token Architecture & Security Analysis	86
4.7	Sprint 1 Conclusion	88
5	Sprint 2 – Notification Engine & Systemic Auditing	89
5.1	Introduction	89
5.2	Sprint 2 Backlog	89
5.3	Sprint 2 Refinement	89
5.3.1	Refinement of the Use Case “Consult notifications”	90
5.3.2	Refinement of the Use Case “Consult audit logs”	91
5.4	Sprint 2 Conception	92
5.4.1	Class Diagram – Sprint 2 Core Domain	92
5.4.2	Activity Diagram – Use Case “Consult notifications”	93
5.4.3	State Diagram – Notification Life Cycle	94
5.4.4	Sequence Diagram – Use Case “Consult audit logs”	95
5.5	Sprint 2 Implementation	96

5.5.1	Consult Notifications Interface	96
5.5.2	Consult Audit Logs Interface	97
5.6	Testing and System Validation	97
5.6.1	Test Strategy	97
5.6.2	Test Cases and Results	98
5.6.3	Performance and Stress Test Verification	99
5.7	Sprint 2 Conclusion	99
General Conclusion		100
Bibliography		102

List of Figures

1	Actors Diagram	23
2	Global Use Case Diagram	25
3	Refinement of the use case “Login”	34
4	Refinement of the use case “Manage admins”	36
5	Refinement of the use case “Manage contacts”	38
6	Refinement of the use case “Manage companies”	40
7	Refinement of the use case “Manage surveys”	42
8	Sprint 0 Class Diagram	44
9	Activity Diagram – Login	46
10	Sequence Diagram – Manage admins	47
11	Sequence Diagram – Manage contacts	48
12	Sequence Diagram – Manage companies	49
13	State Diagram – Manage surveys	50
14	Activity Diagram – Manage surveys	51
15	Login Interface	52
16	OTP Verification Interface	53
17	Reset Password Interface	54
18	Admin Directory Interface	56
19	Add Admin Form	56
20	Edit and Disable Admin Actions	57
21	Contact Directory Interface	57
22	Add Contact Form	58
23	Company Directory Interface	58
24	Add Company Form	59
25	Batch Import - Upload File	59
26	Batch Import – Data Validation, showing results of a controlled synthetic stress test with a deliberately injected majority of error rows to validate the pipeline’s per-row error reporting under adversarial conditions.	60
27	Batch Import - Processing	61
28	Survey Directory Interface	62

29	Survey Overview Dashboard	62
30	Edit Survey Configuration	63
31	Add Survey Passage Form	63
32	Survey Passage Details	64
33	Company Sample Assignment Interface	64
34	Questionnaire Template Management	65
35	Survey Passage Monitoring Dashboard	65
36	Survey Results Interface	66
37	Use Case Diagram – Consult assigned surveys	68
38	Use Case Diagram – Consult assigned results	70
39	Use Case Diagram – Manage system settings	71
40	Use Case Diagram – Manage locked accounts	72
41	Use Case Diagram – Manage profile	73
42	Sprint 1 Class Diagram	74
43	Sequence Diagram – Consult assigned surveys	75
44	Sequence Diagram – Consult assigned results	76
45	Sequence Diagram – Manage system settings	77
46	Sequence Diagram – Manage locked accounts	78
47	State Diagram – Manage locked accounts	79
48	Sequence Diagram – Manage profile	80
49	My Surveys Interface	81
50	Company Contact Overview Dashboard	81
51	Survey Assignment Email Notification	82
52	Consult Assigned Results & Reports Interface	82
53	System Settings Configuration Interface	83
54	Locked Accounts Management Interface	83
55	Incorrect OTP Validation Error	84
56	Account Lockout Security Warning	84
57	Admin Profile Management Interface	85
58	Contact Profile Management Interface	86
59	Use Case Diagram – Consult notifications	90
60	Use Case Diagram – Consult audit logs	91
61	Sprint 2 Class Diagram	92
62	Activity Diagram – Consult notifications	93
63	State Diagram – Notification Life Cycle	94
64	Sequence Diagram – Consult audit logs	95
65	Consult Notifications Interface	96
66	Consult Audit Logs Interface	97

List of Tables

1	Comparison with Existing Survey Platforms	18
2	Technology Choices and Justifications	19
3	Functional Requirements	22
4	Actor Roles and Associated Tasks	24
5	Product Backlog User Stories	26
6	Development Tools	28
7	Database Tools	28
8	Task Queues and Libraries	29
9	Testing Tools	30
10	Conception and Modeling Tools	31
11	Collaboration Tools	31
12	Sprint 0 Backlog – Authentication & Administrative Core	33
13	Textual description – Login	35
14	Textual description – Manage admins	37
15	Textual description – Manage contacts	39
16	Textual description – Manage companies	41
17	Textual description – Manage surveys	43
18	Sprint 1 Backlog	67
19	Textual description – Consult assigned surveys	69
20	Textual description – Consult assigned results	70
21	Textual description – Manage system settings	71
22	Textual description – Manage locked accounts	72
23	Textual description – Manage profile	73
24	Sprint 2 Backlog	89
25	Textual description – Consult notifications	90
26	Textual description – Consult audit logs	91
27	System Test Cases and Execution Results	98

Liste des abréviations

INS	Institut National de la Statistique
PFA	Projet de Fin d'Année
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
UUID	Universally Unique Identifier
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
SPA	Single Page Application
UI	User Interface
UX	User Experience

General Introduction

Modernizing data collection is a significant challenge for national statistical institutes. Traditional survey workflows that rely on manual data entry, offline spreadsheets, and physical mail distribution introduce processing delays and increase the risk of data-entry errors. Transitioning to web-based platforms enables these organizations to automate data collection, strengthen security, and maintain centralized progress tracking.

Official statistics are essential for public decision-making, economic monitoring, and demographic analysis. Without reliable and timely data, public authorities cannot plan infrastructure, monitor inflation, or measure unemployment rates accurately. The quality of national statistics depends directly on the reliability and efficiency of the data collection tools that produce them.

This project was carried out at the National Institute of Statistics (INS) [1] in Tunisia. The objective was to design and develop a web platform for administrative tracking and online survey data collection. The platform replaces manual workflows with a secure relational database, role-based administrative dashboards, asynchronous background processing for heavy operations, and an integration layer with an external questionnaire system.

The scope of this project covers the internal administrative portal for INS staff, the contact-facing survey consultation portal, and the server-to-server integration with the external questionnaire system. It does not cover the design of the external questionnaire application itself, nor statistical analysis of collected data. The platform was developed as a solo project by the author under the guidance of a professional supervisor at the INS.

This report is structured as follows:

- **Chapter 1** presents the organizational context at the INS, analyzes the limitations of the existing survey system, and introduces the proposed technical solution.
- **Chapter 2** defines the functional and non-functional requirements, identifies system actors, and presents the development methodology and tools.
- **Chapter 3** covers Sprint 0: the authentication system and core administrative management features.
- **Chapter 4** covers Sprint 1: the survey consultation portal for company contacts and secondary administrative features.

- **Chapter 5** covers Sprint 2: the real-time notification engine, systemic audit logging, and platform testing.

Chapter 1

General Context

1.1 Introduction

This chapter presents the organizational and technical framework of the project. It describes the host organization (INS), its role, and its internal structure. It then evaluates the existing survey workflows, outlines the problems that motivated this project, and describes the proposed architecture designed to solve them.

1.2 Host Organization: The INS

1.2.1 History and Status

The National Institute of Statistics (INS) is the main organization for Tunisia's national statistics. Created on December 31, 1969, its administrative structure was later defined on October 17, 2000. The INS operates as a public establishment. It has its own legal status and financial independence, and is placed under the supervision of the Ministry of Economy and Planning.

1.2.2 Core Mandate and Responsibilities

The INS is responsible for collecting, processing, analyzing, and publishing official national statistics. Its operations cover three main areas:

- **Censuses and Surveys:** Conducting the General Population and Housing Census every 10 years, alongside regular employment and social surveys.
- **Economic Indicators:** Preparing national accounts, measuring the Consumer Price Index (CPI), and compiling foreign trade statistics.
- **Coordination and Documentation:** Maintaining standard classification lists (economic codes) and managing the secretariat of the National Statistics Council.

1.2.3 Organizational Structure

The INS is structured into central and regional offices:

- **Central Level:** Composed of seven directorates in Tunis that handle database design, national accounts, statistical coordination, and IT systems.
- **Regional Level:** Divided into six regional offices that manage local field staff, run census operations, and perform primary data entry.

1.3 Project Overview

1.3.1 Analysis of the Existing System

The existing survey workflow at the INS follows a multi-step manual process. The legacy workflow operates as follows:

1. **Sample Preparation:** Survey managers prepare company sample lists in Excel spreadsheets, manually selecting target enterprises from internal registry databases.
2. **Questionnaire Distribution:** Paper questionnaires or PDF forms are sent to company representatives via physical mail or email attachments.
3. **Response Collection:** Company representatives complete the forms offline and return them by mail, fax, or email.
4. **Data Entry:** Administrative staff manually enter received responses into legacy intranet database tools.
5. **Progress Tracking:** Survey managers track response rates using separate spreadsheets, cross-referencing mailed questionnaires against received responses.

This workflow presents several operational issues: the distribution-to-collection cycle spans weeks due to postal delays, manual data entry introduces transcription errors, and the absence of a centralized tracking system makes it difficult to identify non-responding companies in real time. Furthermore, the legacy intranet tools process file uploads synchronously, meaning that uploading a large company sample file blocks the server for all concurrent users.

1.3.2 Problem Statement

The combination of manual operations and legacy database utilities presents several key limitations:

1. **Synchronous Processing Bottlenecks:** When administrators upload company sample files using the legacy intranet tools, the file-parsing script executes synchronously. Processing large spreadsheets blocks the single-threaded web server, leading to API timeouts and freezing the system for other staff.
2. **Lack of Real-Time Tracking:** Because questionnaires are filled offline, administrators cannot monitor live survey progress, making it difficult to identify active or inactive company participants.
3. **Access Friction:** Forcing external company representatives to go through account creation and password management procedures for simple survey participation results in high support overhead and low submission rates.

1.3.3 Proposed Solution

To address these limitations, we developed the **INS Statistical Survey Collection Platform**. The platform uses a separated architecture with three key components:

- **Administrative Portal (Angular [2]):** A dashboard that allows INS managers to configure surveys, assign passages, upload company samples, and track response rates in real-time.
- **Asynchronous Task Queue (FastAPI [3] + Celery [5] + Redis [6]):** Offloads file parsing, database staging, and export generation to background tasks to prevent backend blocking.
- **Passwordless Access (UUID Token System):** Generates a unique, stable UUID token for each (contact, passage) pair during invitation dispatch. External companies use this token link to access the questionnaire directly without logging in, enabling a secure save-and-resume workflow.

1.3.4 Comparison with Existing Survey Platforms

Before developing a custom solution, existing open-source survey platforms were evaluated. Table 1 summarizes the comparison:

Table 1: Comparison with Existing Survey Platforms

Criterion	ODK [20]	KoboToolbox [21]	LimeSurvey [22]	INS Platform
Role-based admin management	No	No	Partial	Yes (3 roles)
Internal company registry	No	No	No	Yes
Passwordless token access	No	No	No	Yes (UUID)
Async background processing	No	No	No	Yes (Celery)
Bulk Excel import	No	Partial	No	Yes
On-premise data control	Yes	Partial	Yes	Yes
Statistical secrecy compliance	Not designed	Not designed	Not designed	Built-in

Platforms such as ODK [20], KoboToolbox [21], and LimeSurvey [22] offer mature questionnaire distribution features. However, they are designed for generic survey workflows and do not support the INS-specific requirements: role-segmented internal management, passwordless token-based access for external company respondents, or tight coupling with an internal company/contact registry. Adopting a generic tool would have required maintaining a parallel system for company and passage management, duplicating data and increasing the compliance surface for statistical secrecy obligations under Tunisian law [19]. A custom platform, hosted entirely within INS infrastructure, was chosen to maintain direct institutional control over survey response data throughout its lifecycle.

1.4 Preliminary Study and Architectural Choices

The platform is built using a modern separated architecture. The main technology choices and their justifications are summarized in Table 2:

Table 2: Technology Choices and Justifications

Layer	Technology	Alternative Considered	Justification
Frontend SPA	Angular [2]	React	Angular provides a complete framework with built-in routing, dependency injection, HTTP client, and i18n support, reducing the need for third-party libraries.
Backend API	FastAPI [3] (Python [4])	Django REST Framework	FastAPI offers native async support, automatic OpenAPI documentation, and high performance via Uvicorn ASGI server.
Database	MySQL [15]	PostgreSQL	MySQL was selected for compatibility with existing INS infrastructure and local XAMPP development environments.
Task Queue	Celery [5] + Redis [6]	Python threading	Celery provides a production-grade distributed task queue with retry policies, result backends, and multi-threaded worker pools.
Message Broker	Redis [6]	RabbitMQ	Redis serves a dual role as both message broker and in-memory security cache (OTP, lockout, JWT blacklist), reducing infrastructure complexity.

The platform architecture consists of five interconnected components:

1. **Angular SPA Frontend:** Communicates with the backend via HTTP REST calls. JWT tokens stored in `localStorage` are injected into all outgoing requests through a custom HTTP interceptor.
2. **FastAPI REST Backend:** Handles all business logic, authentication, role-based access control, and database transactions via SQLAlchemy ORM [17].
3. **MySQL Database:** Stores persistent data including users, companies, contacts, surveys, passages, samples, audit logs, and notifications.

4. **Redis In-Memory Store:** Operates in a dual capacity: (1) as the Celery message broker for task queuing, and (2) as a volatile security cache storing OTP codes, rate-limiting counters, account lockout flags, and JWT blacklist entries with dynamic TTL expirations.
5. **Celery Worker Pool:** Runs as a separate process with multi-threaded concurrency (`-P threads -c 4`), executing background tasks such as Excel file parsing, batch email dispatch, and audit log export without blocking the main API.

1.5 Conclusion

This chapter defined the project framework by presenting the INS and outlining the limitations of the existing survey collection system. The proposed platform addresses these issues through a separated architecture with asynchronous background processing, role-based access control, and passwordless external access. The next chapter will define the functional and non-functional requirements of the system.

Chapter 2

Requirements Specification

2.1 Introduction

This chapter defines the functional and non-functional requirements of the INS Statistical Survey Collection Platform. We first outline the general system context and identify the functional and non-functional specifications. We then identify the system actors, introduce a global use case diagram, and lay out the product backlog. Finally, we describe the development work environment, detailing both the design methodology and the software tools utilized.

2.2 System Context

The INS Statistical Survey Collection Platform is the central hub for administrative configuration and survey tracking. The system context involves four human roles (System Administrator, Account Manager, Survey Manager, and Company Contacts) and one external software interface (the Questionnaire System). INS managers configure surveys and passages, uploading company lists. The platform generates secure UUID tokens and handles API updates. Contacts access assigned surveys and are redirected to the external Questionnaire System, which reports completion status back to the platform.

2.3 Identification of Functional Requirements

A functional requirement specifies an action that the system must be able to perform from the user's perspective. The functional requirements of the platform are listed in Table 3:

Table 3: Functional Requirements

ID	Requirement	Description
FR-01	Login	Authenticate users via credentials and 2FA OTP verification.
FR-02	Manage admins	Create, update, and archive administrative accounts.
FR-03	Manage locked accounts	View and unlock accounts suspended by security policies.
FR-04	Manage contacts	Register, assign, and maintain company contact profiles.
FR-05	Manage companies	Maintain the company registry via manual entry or bulk Excel import.
FR-06	Manage surveys	Configure survey campaigns, passages, samples, and templates.
FR-07	Manage profile	Update personal information and change password.
FR-08	Manage system settings	Configure security, language, and email parameters.
FR-09	Consult assigned surveys	View active survey assignments and access questionnaires.
FR-10	Consult assigned results	View historical survey submissions and download reports.
FR-11	Consult notifications	View, filter, and manage platform activity alerts.
FR-12	Consult audit logs	Monitor system activities and export security logs.

2.4 Identification of Non-Functional Requirements

Non-functional requirements specify quality and operational attributes:

- **Security:** Secure password storage (bcrypt) and cryptographically secure passwordless tokens (UUID).
- **Performance:** Asynchronous background parsing (Celery [5] + Redis [6]) to prevent API blocking.

- **Usability:** A responsive layout localized in Arabic, French, and English.
- **Reliability:** Relational consistency (ACID database transactions) and robust error handling.
- **Confidentiality and Legal Compliance:** All individual survey response data is subject to statistical secrecy under *Loi n° 99-32 du 13 avril 1999, relative au système national de la statistique* (Art. 5) [19]; role-based access control and full audit logging implement this legal obligation at the application layer.
- **Scalability:** The asynchronous Celery/Redis worker pool decouples heavy operations (bulk company import, audit-log export) from the synchronous API, allowing the system to absorb bursts of concurrent activity without blocking other users.
- **Configurable Security Posture:** Authentication parameters are centrally configurable rather than hardcoded: OTP expiration (default 5 minutes), maximum sign-in attempts before lockout (default 3), account lockout duration (default 24 hours), and a temporary password-only bypass window after manual unlock (default 1 hour).

2.5 Identification of Actors

An actor, in software development, is an entity (either a user or a system) that interacts with the application to accomplish a specific task. Identifying the actors is crucial to understand the users and their interactions with the system, which helps define the functional and non-functional requirements. Thus, we ensure that the application meets the real needs of the users. Our application includes four key actors who interact directly with the system:

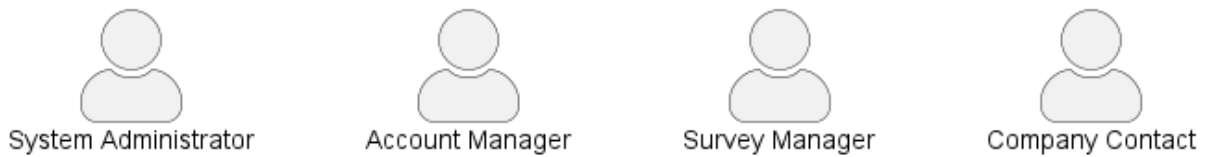


Figure 1: Actors Diagram

The roles, descriptions, and associated tasks of each actor are summarized in the table below:

Table 4: Actor Roles and Associated Tasks

Actor	Role Description	Associated Tasks
System Administrator	Configures global parameters, manages administrative accounts, company profiles, and reviews audit logs.	• Login • Manage admins • Manage locked accounts • Manage companies • Manage profile • Manage system settings • Receive notifications • Consult system audit logs
Account Manager	Manages directory data, and representative contacts.	• Login • Manage contacts • Manage locked accounts • Manage profile • Receive notifications
Survey Manager	Manages surveys and passages, uploads PDF questionnaires, and tracks response progress.	• Login • Manage surveys • Manage profile • Receive notifications
Company Contact	Represents external entities and accesses assignments.	• Login • Manage profile • Consult assigned surveys • Consult assigned results • Receive notifications

2.6 Global Use Case Diagram

The global use case diagram illustrates actor interactions across platform functional boundaries:

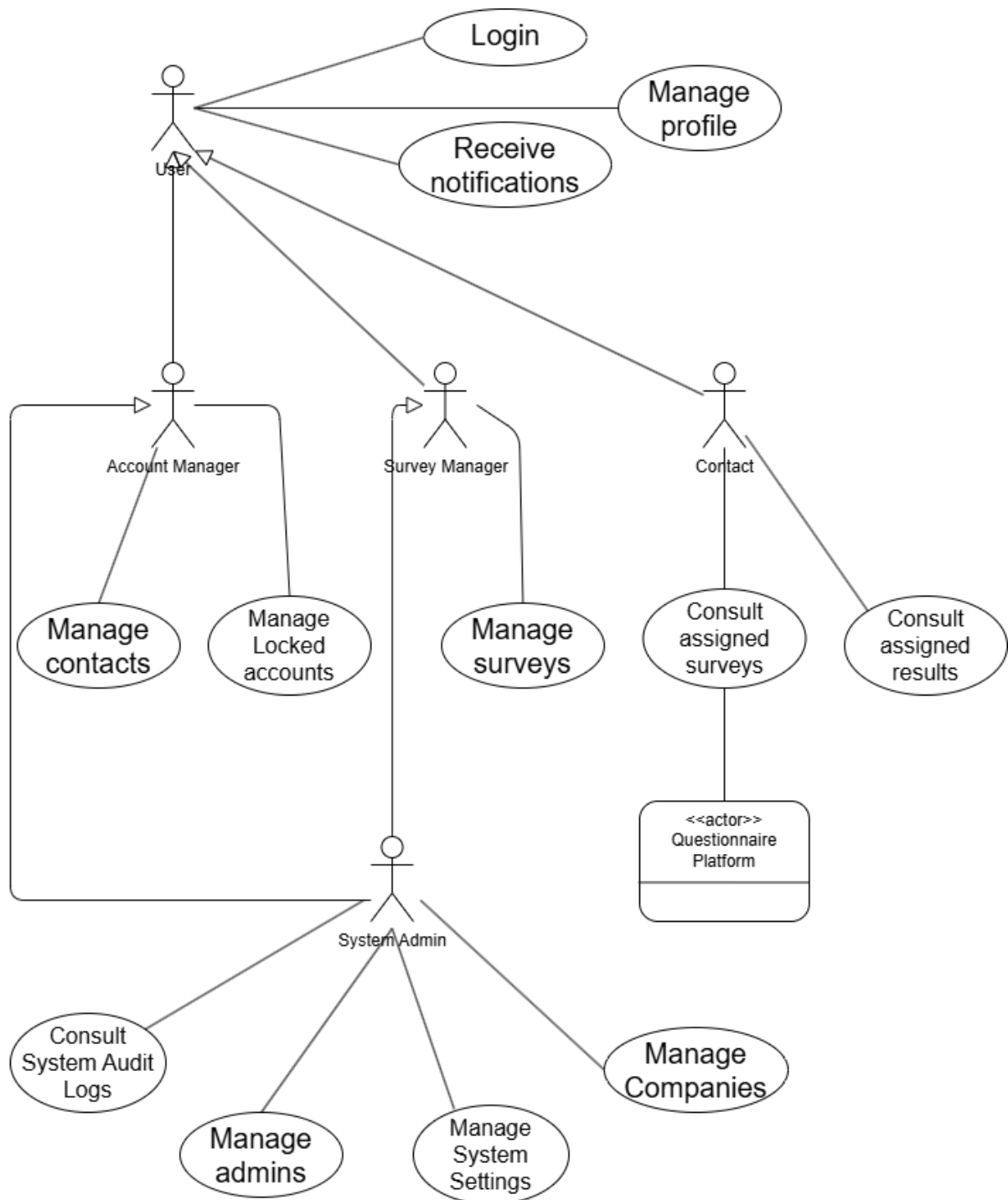


Figure 2: Global Use Case Diagram

2.7 Product Backlog

The Product Backlog lists user stories mapped to priorities:

Table 5: Product Backlog User Stories

ID	User Story	Priority	Sprint
US01	As a user, I want to Login to access my secure workspace.	1	0
US02	As a System Administrator, I want to Manage admins to set up administrative user accounts.	1	0
US03	As a System Administrator or Account Manager, I want to Manage contacts to assign representatives to surveyed companies.	1	0
US04	As a System Administrator, I want to Manage companies to maintain business registry files.	1	0
US05	As a System Administrator or Survey Manager, I want to Manage surveys to configure survey passages and upload reference forms.	1	0
US06	As a Company Contact, I want to Consult assigned surveys to view my active questionnaires.	2	1
US07	As a Company Contact, I want to Consult assigned results to check my past survey responses.	2	1
US08	As a System Administrator, I want to Manage system settings to configure global parameters.	2	1
US09	As a System Administrator or Account Manager, I want to Manage locked accounts to restore or restrict system access.	2	1
US10	As a user, I want to Manage profile to modify my personal contact information.	3	1
US11	As a user, I want to Receive notifications when relevant platform updates or survey submissions occur.	2	2
US12	As a System Administrator, I want to Consult system audit logs to audit system activities.	2	2

2.8 Work Environment

2.8.1 Design Methodology

The project was designed using the **Agile Scrum Methodology** [8]. This choice is motivated by the need for iterative feedback and incremental releases, enabling rapid integration tests between the backend API and the external systems. Development was organized into Sprints of two weeks each:

- **Sprint 0:** Backend database setup, authentication, and core administration for Login, Manage admins, Manage contacts, Manage companies, and Manage surveys.
- **Sprint 1:** Frontend portal interface integration, and configuration of Consult assigned surveys, Consult assigned results, Manage system settings, Manage locked accounts, and Manage profile.
- **Sprint 2:** System integration webhooks, Receive notifications engine, Consult system audit logs, and testing.

2.8.2 Software Environment

Various software tools, frameworks, and packages were utilized during the design and development of the platform. These tools are grouped into the following categories:

2.8.2.1 Development Tools

The table below lists the primary development tools used to build the application:

Table 6: Development Tools

	Visual Studio Code: Code editor used for writing Python backend services and TypeScript Angular components. [9]
	Angular: Single-Page Application (SPA) framework used to develop the responsive administration portal and contact dashboards. [2]
	FastAPI: Modern, high-performance web framework for Python used to create the REST API endpoints. [3]
	Python: Main programming language used to implement the backend business logic and database queries. [4]

2.8.2.2 Database Tools

The database backend consists of the following data management tools:

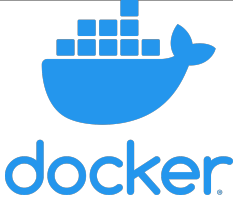
Table 7: Database Tools

	MySQL: Relational database system utilized through the local XAMPP environment to store secure user records, company profiles, and survey assignments. [15]
	SQLite: Lightweight database engine utilized in development for rapid testing and zero-configuration local database initialization. [16]

2.8.2.3 Background Processing & Libraries

The background task workers and helper libraries used to optimize platform workflows are listed below:

Table 8: Task Queues and Libraries

	Celery: Distributed task queue used to run asynchronous background operations. [5]
	Redis: High-performance in-memory data store utilized as the message broker for Celery queues. [6]
	Docker: Containerization platform used to deploy Redis as an isolated, reproducible container during development. [7]
	SQLAlchemy: Object-Relational Mapper (ORM) used to map Python models to SQL database tables securely. [17]
	NGX-Translate: Internationalization (i18n) library for Angular used to enable dynamic multi-language UI translation (Arabic, French, English) via asynchronous JSON translation loaders. [18]

To prevent synchronous blocking of the main web server and ensure full multi-language accessibility, the platform architecture incorporates dedicated asynchronous background processing and client-side internationalization layers:

1. **Dual Role of Redis:** Redis [6] operates in a dual capacity within the platform. First, it serves as the high-throughput **Message Broker** that holds queued tasks waiting for execution by background workers. Second, it acts as a volatile **Security & Session In-Memory Cache**, storing short-lived security tokens such as 2FA

OTP codes with dynamic Time-To-Live (TTL) expirations, login rate-limiting counters, and account lockout tracking flags.

2. **Celery Task Worker Pipeline:** Celery [5] is integrated alongside FastAPI to decouple I/O-heavy and computationally intensive operations from the synchronous HTTP request-response cycle. By delegating long-running tasks—such as parsing large company sample Excel files, dispatching batch email notifications, and generating audit log exports—to a multi-threaded worker pool, the primary API maintains near-zero latency and remains completely responsive to concurrent user requests.
3. **NGX-Translate Internationalization Engine:** NGX-Translate [18] is integrated into the Angular Single-Page Application to deliver dynamic, client-side multi-language translation (Arabic, French, and English). Utilizing `TranslateHttpLoader`, the engine asynchronously fetches localized JSON dictionary assets (`fr.json`, `ar.json`, `en.json`) upon application initialization or language switching. The framework injects translations seamlessly into UI templates via RxJS streams and Angular pipes, guaranteeing full internationalization (i18n) support across all administrative and contact dashboards without requiring page reloads or backend rendering overhead. Furthermore, full Right-to-Left (RTL) layout mirroring is dynamically applied to component containers and CSS input properties via Angular `[dir]="currentLang() === 'ar' ? 'rtl' : 'ltr'"` bindings whenever Arabic is selected, ensuring fluent bidirectionality across LTR and RTL scripts.

2.8.2.4 Testing and Validation Tools

The API testing and interface verification was performed using the following tool:

Table 9: Testing Tools

	Postman: API client platform used to construct, document, and test backend REST endpoints and webhook payloads. [12]
---	--

2.8.2.5 Conception and Modeling Tools

The conceptual structure and diagrams of the system were designed using the following tool:

Table 10: Conception and Modeling Tools

	PlantUML: Open-source modeling tool used to define and generate UML use case diagrams, sequence diagrams, and class diagrams from plain text descriptions. [13]
---	---

2.8.2.6 Collaboration Tools

To organize version branches and documents during development, the following tools were used:

Table 11: Collaboration Tools

	Git: Distributed version control system used to track source code edits. [10]
	GitHub: Repository hosting service used to push code versions, manage pull requests, and review commits. [11]
	Overleaf: Online LaTeX editor used for collaborative writing and document compilation of this report. [14]

2.9 Conclusion

This chapter defined the Requirements Specification of the INS Statistical Survey Collection Platform. We detailed the system context, identified functional and non-functional requirements, mapped user roles, presented the product backlog, and outlined the Scrum methodology and software environment. The next chapter will present the first development sprint covering authentication and core administrative features.

Chapter 3

Sprint 0 – Authentication & Administrative Core

3.1 Introduction

In this chapter, we present the first sprint of the project, which covers the foundational authentication and administrative core features. The study of this sprint encompasses backlog identification, use case refinement, system conception, and user interface implementation.

Sprint 0 focuses on establishing secure user authentication, role-based access control, and administrative management of users, contacts, company registries, and survey campaign configurations. These features provide the core operational foundation upon which subsequent survey management sprints are constructed.

3.2 Sprint 0 Backlog Identification

The sprint backlog contains all User Stories selected from the Product Backlog for Sprint 0. Each story follows the "INVEST" criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable). Table 12 outlines the user stories prioritized for Sprint 0:

Table 12: Sprint 0 Backlog – Authentication & Administrative Core

ID	User Story	Estimation
US01	As a user, I want to Login to access my secure workspace.	High
US02	As a System Administrator, I want to Manage admins to set up administrative user accounts.	Medium
US03	As a System Administrator or Account Manager, I want to Manage contacts to assign representatives to surveyed companies.	Medium
US04	As a System Administrator, I want to Manage companies to maintain business registry files.	High
US05	As a System Administrator or Survey Manager, I want to Manage surveys to configure survey passages and upload reference forms.	High

A User Story in Sprint 0 is considered "completed" when:

- Frontend UI components and backend REST API endpoints are developed and integrated.
- Input validation, password hashing, and token handling are verified.
- Core functionality and REST endpoints are manually verified via Postman collections and technical documentation is updated.

3.3 Sprint 0 Refinement

In this section, we refine the functional details and textual specifications for the following four use cases:

- Login;
- Manage admins;
- Manage contacts;
- Manage companies;
- Manage surveys.

3.3.1 Refinement of the Use Case “Login”

The diagram below illustrates the refinement of the “Login” use case and its interactions with system actors:

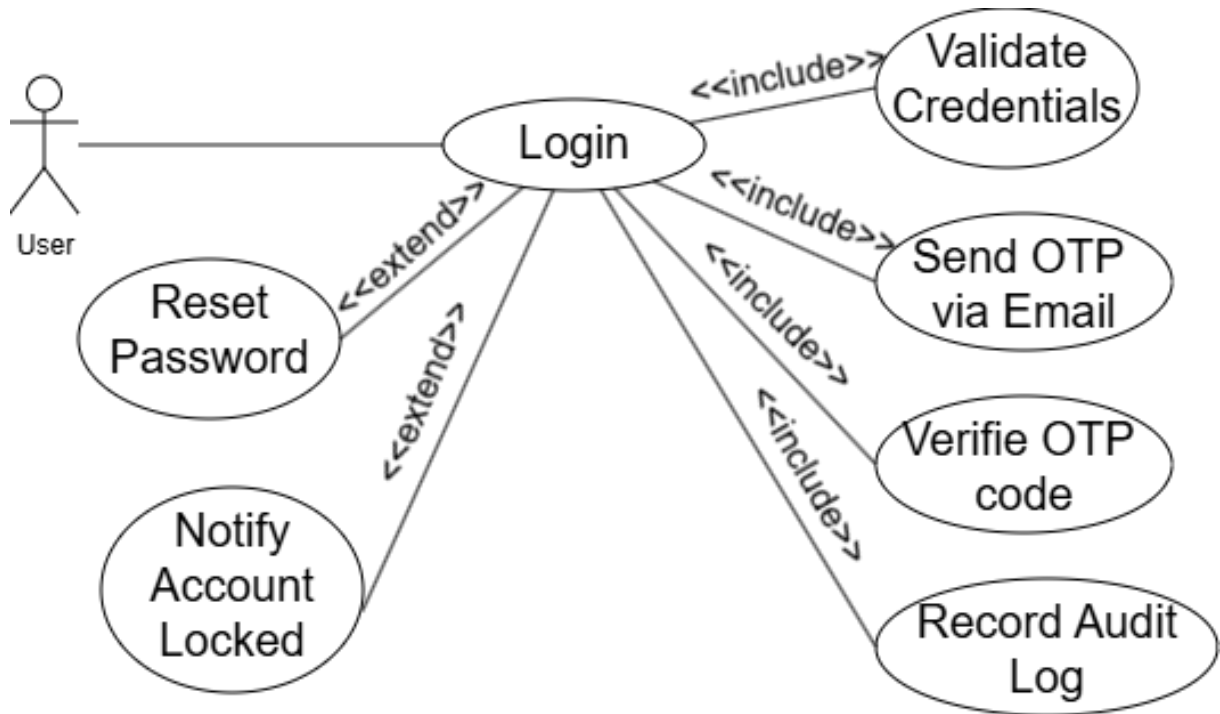


Figure 3: Refinement of the use case “Login”

Table 13 presents the textual description of the “Login” use case:

Table 13: Textual description – Login

Use Case	Login
Actor	System Administrator, Account Manager, Survey Manager, Company Contact
Pre-condition	The user has registered credentials or an active account in the system database.
Post-condition	The user is authenticated, passes OTP verification, completes initial password setup if first-time user, is redirected to their role dashboard, and the login event is recorded in the system audit logs.
Main Scenario	1. The user navigates to the login interface. 2. The system presents the authentication form (email and password). 3. The user submits their credentials. 4. The system validates credentials against database records (bcrypt password verification). 5. The system generates a One-Time Password (OTP) code, stores it in an in-memory Redis data store [6] with a dynamic expiration TTL configured via global system settings, and dispatches it to the user's registered email address. 6. The system presents the OTP verification interface. 7. The user enters the received OTP code and submits the form. 8. The system verifies the submitted OTP code against the Redis store [6]. 9. If it is the user's first time logging in, the system redirects them to the password reset page; otherwise, the system issues an authentication payload (JWT token) and redirects them to their role-specific dashboard. 10. The system creates an audit log entry recording the user authentication event (user ID, timestamp, IP address, and status).
Alternative Scenarios	• Invalid credentials: The system records a failed login attempt in the audit logs and displays an error message indicating invalid email or password. • Invalid or expired OTP: The system displays an error message and prompts the user to request a new OTP code. • First-time login password change incomplete: The system restricts workspace access until a new password is set. • Locked account: The system notifies the user that their account is locked and directs them to an administrator.
Exception	Database or mail server connection failure; the system displays a server error notification.

3.3.2 Refinement of the Use Case “Manage admins”

The diagram below illustrates the refinement of the “Manage admins” use case and its interactions with system actors:

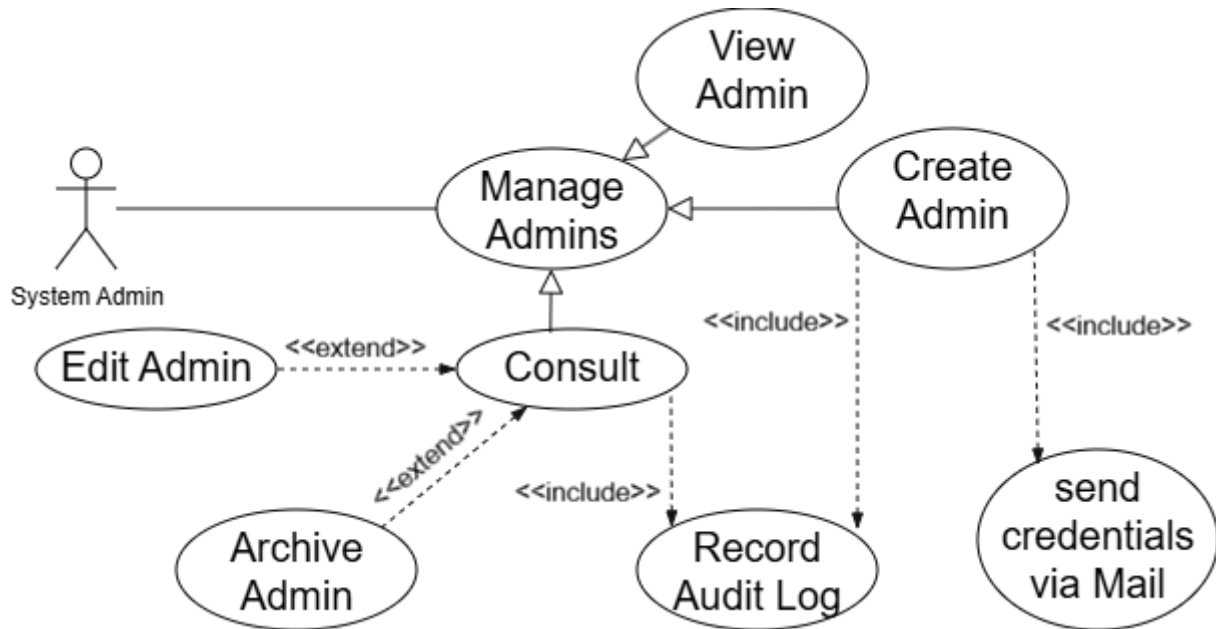


Figure 4: Refinement of the use case “Manage admins”

Table 14 presents the textual description of the “Manage admins” use case:

Table 14: Textual description – Manage admins

Use Case	Manage admins
Actor	System Administrator
Pre-condition	The System Administrator is authenticated with active super-admin privileges.
Post-condition	Administrative accounts are created, updated, or archived in the database, initial credentials are sent to the new administrator via email, and the administrative action is recorded in the system audit logs.
Main Scenario	1. The administrator navigates to the admin management dashboard. 2. The system retrieves and displays the table of existing administrative accounts. 3. The administrator selects an action (create new admin, update role/email, or archive account). 4. The administrator fills in the required fields and submits the request. 5. The system validates the input data (format and email uniqueness). 6. The system executes the requested transaction in the database (creating, updating, or archiving the admin account). 7. Upon account creation, the system dispatches an automated email to the new administrator containing their temporary credentials. 8. The system creates an entry in the system audit logs recording the administrative management action (action type, target admin ID, performing admin ID, timestamp, and IP address). 9. The system displays a confirmation notification and updates the account list.
Alternative Scenarios	• Duplicate email: The system blocks submission and alerts the administrator. • Invalid data format: The system highlights invalid fields in the form. • Mail server error: The system creates the account, logs a notification warning in audit logs, and permits resending credentials.
Exception	Database connection failure; the system displays a generic server error.

3.3.3 Refinement of the Use Case “Manage contacts”

The diagram below illustrates the refinement of the “Manage contacts” use case and its interactions with system actors:

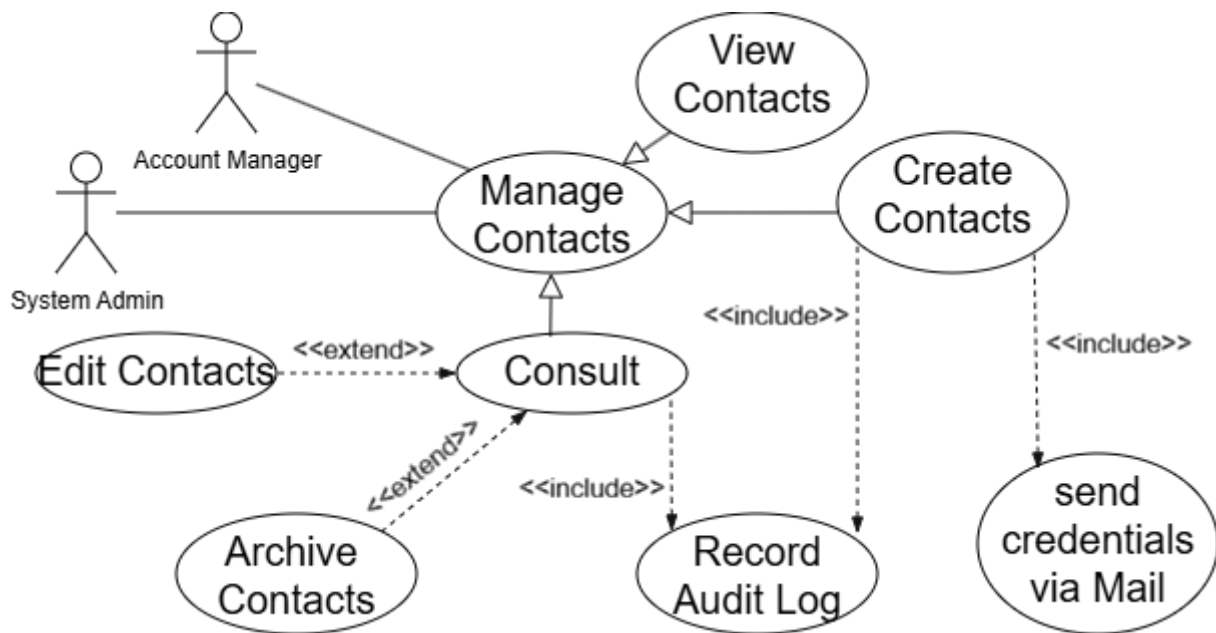


Figure 5: Refinement of the use case “Manage contacts”

Table 15 presents the textual description of the “Manage contacts” use case:

Table 15: Textual description – Manage contacts

Use Case	Manage contacts
Actor	System Administrator, Account Manager
Pre-condition	The user is logged in with System Administrator or Account Manager role privileges.
Post-condition	Company contact records are created, linked to targeted companies, updated, or enabled/disabled in the database, initial access credentials are sent to the contact via email, and the action is recorded in the system audit logs.
Main Scenario	1. The manager accesses the contact directory interface. 2. The system displays company contacts filtered by search or company criteria. 3. The manager selects an action (add contact, assign to company, edit contact details, or disable/enable contact). 4. The manager submits the contact profile form. 5. The system validates contact phone, email, and company association. 6. The system updates database records (creating, updating, linking, or disabling/enabling the contact profile). 7. Upon contact creation or assignment, the system dispatches an automated email to the contact containing their initial login credentials. 8. The system creates an entry in the system audit logs recording the contact management action (action type, target contact ID, performing user ID, timestamp, and IP address). 9. The system confirms successful operation and updates the contact directory view.
Alternative Scenarios	• Unassigned company: The system prompts the manager to select a valid registered company. • Form validation errors: The system prevents saving until all mandatory contact fields are valid. • Mail server error: The system saves the contact profile, logs a notification warning in audit logs, and permits re-inviting the contact.
Exception	Server connection drop; the system displays a network retry prompt.

3.3.4 Refinement of the Use Case “Manage companies”

The diagram below illustrates the refinement of the “Manage companies” use case and its interactions with system actors:

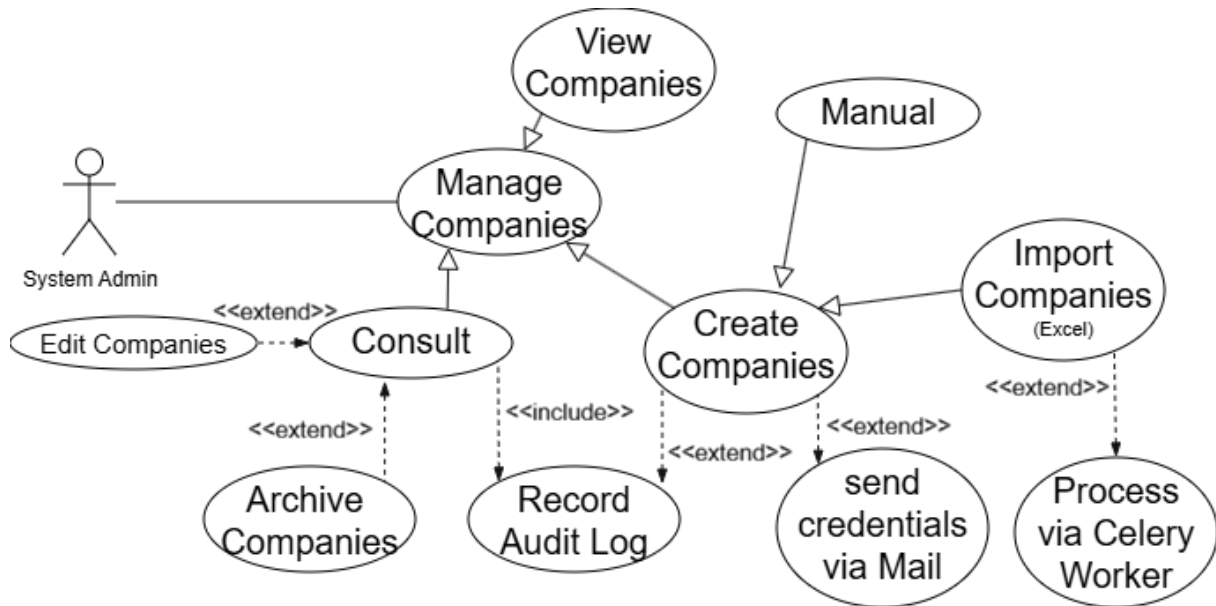


Figure 6: Refinement of the use case “Manage companies”

Table 16 presents the textual description of the “Manage companies” use case:

Table 16: Textual description – Manage companies

Use Case	Manage companies
Actor	System Administrator
Pre-condition	The System Administrator is authenticated on the platform with administrative privileges.
Post-condition	Company records are created, updated, imported, or archived in the database, bulk Excel imports are parsed asynchronously via Celery background workers [5], and the action is recorded in the system audit logs.
Main Scenario	1. The administrator opens the company registry management interface. 2. The system presents the company directory with search and filter capabilities. 3. The administrator selects an action (manually add a company, edit company details, archive a company, or batch import companies via Excel file). 4. Manual Entry: The administrator enters or updates company fields (tax ID / NIS code, legal name, sector, address); the system validates and commits database changes. 5. Excel Batch Import: The administrator selects and uploads an Excel file (.xlsx / .xls) containing company rows. 6. The system validates the file structure and offloads parsing to an asynchronous Celery worker [5] via Redis broker [6] to prevent API blocking. 7. The Celery worker parses the file in the background, validates each row, creates or updates company records, and records parsing status. 8. The system dispatches an in-app notification summarizing import results (inserted rows, updated rows, and error count). 9. The system creates an entry in the system audit logs recording the company management or batch import action (performing admin ID, timestamp, processed row count, and status). 10. The system displays a success message and refreshes the company directory.
Alternative Scenarios	• Duplicate tax ID / NIS code: The system rejects manual creation or flags the duplicate row during batch import. • Invalid Excel file structure: The system blocks upload and alerts the administrator to check mandatory column headers. • Partial batch import errors: Celery processes valid rows, logs bad rows, and attaches an error report to the summary notification.
Exception	Database or background task queue connection failure; the system logs the failure in audit logs and displays an error message.

3.3.5 Refinement of the Use Case “Manage surveys”

The diagram below illustrates the refinement of the “Manage surveys” use case and its interactions with system actors:

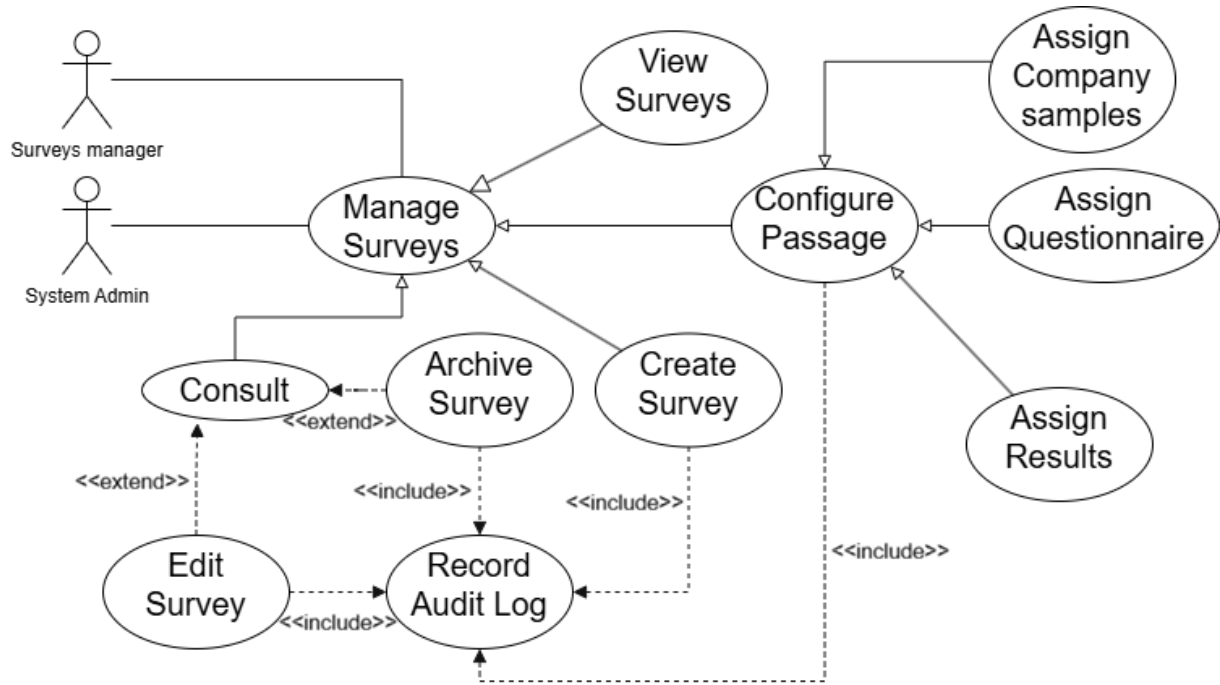


Figure 7: Refinement of the use case “Manage surveys”

Table 17 presents the textual description of the “Manage surveys” use case:

Table 17: Textual description – Manage surveys

Use Case	Manage surveys
Actor	System Administrator, Survey Manager
Pre-condition	The user is authenticated on the platform with System Administrator or Survey Manager role privileges.
Post-condition	Survey definitions, periodic passages, targeted company samples, and questionnaire reference forms are created, configured, or archived in the database, and recorded in system audit logs.
Main Scenario	1. The survey manager navigates to the survey management dashboard. 2. The system displays the directory of statistical survey campaigns. 3. The manager selects a survey campaign and configures its periodic passages (e.g., annual or quarterly survey editions). 4. For each passage, the manager sets passage parameters (passage title, start/end collection dates, statistical domain) and attaches questionnaire reference templates (PDF/Excel). 5. The manager defines and assigns the company sample (targeted list of registered enterprise representatives) for the specific survey passage. 6. The system validates passage dates, company sample selections, and template file formats. 7. The system commits the survey passage and its associated company sample to the database. 8. The system creates an entry in the system audit logs recording the survey passage and sample configuration action (performing user ID, survey ID, passage ID, assigned sample count, timestamp, IP address). 9. The system displays a confirmation notification and updates the survey passage directory.
Alternative Scenarios	• Invalid date range: The system alerts the manager if the passage end date precedes the start date. • Empty company sample: The system warns the manager if no companies are assigned to the passage sample. • Invalid form file format: The system blocks upload and specifies valid template extensions.
Exception	Database transaction failure; the system rolls back changes and displays an error prompt.

3.4 Sprint 0 Conception

This section presents the structural and dynamic design models for the use cases implemented in Sprint 0.

3.4.1 Class Diagram – Sprint 0 Core Domain

The class diagram models the entity structures, attributes, and relationships supporting authentication and administrative management:

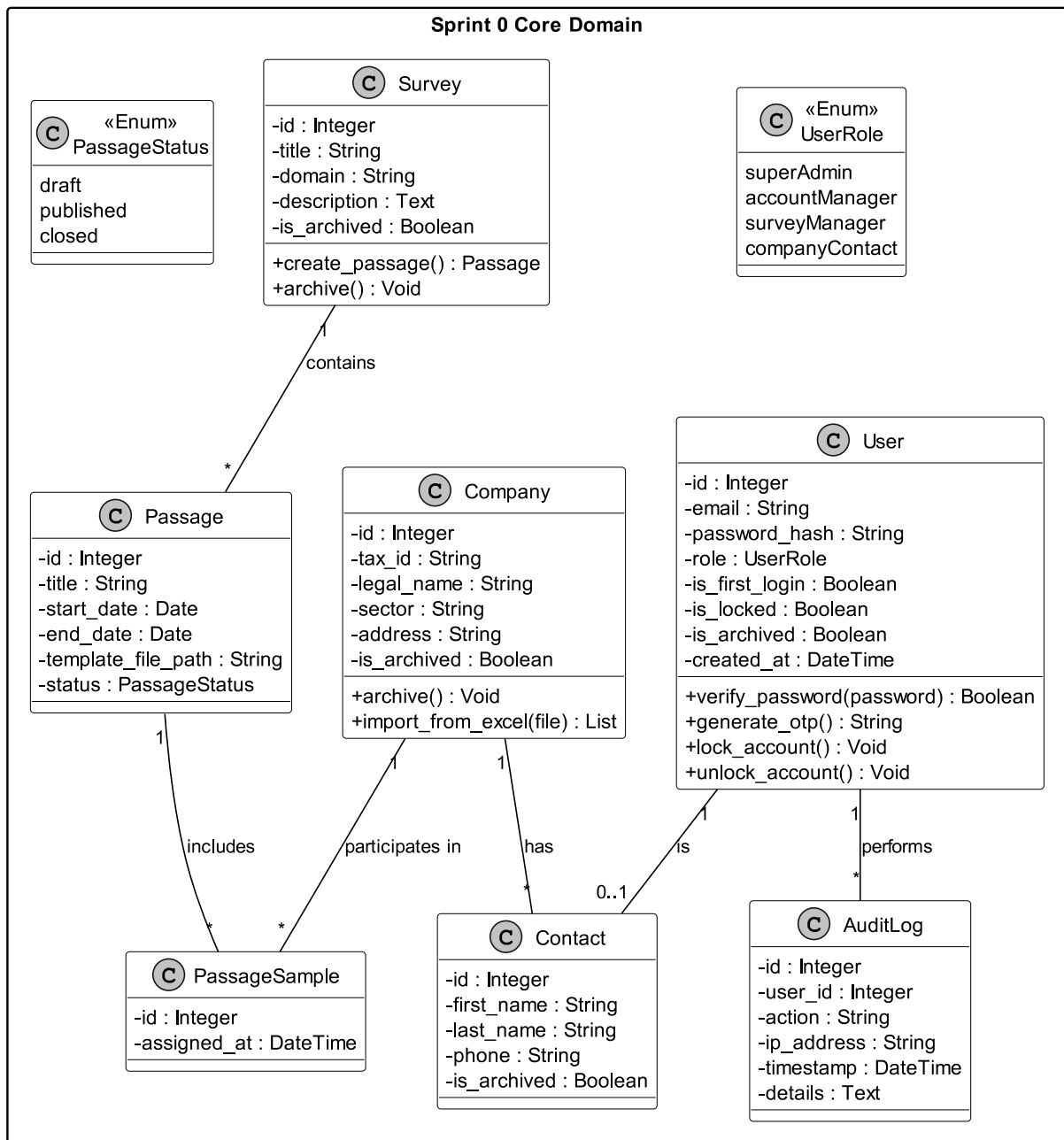


Figure 8: Sprint 0 Class Diagram

The Sprint 0 class diagram (Figure 8) establishes the structural core of the platform data model. The **User** entity serves as the primary authentication actor, possessing a foreign key relationship to **Role** (System Administrator, Account Manager, Survey Manager, Company Contact). The administrative domain connects **Company** to economic classification entities (**Sector** and **Activity**) via foreign keys, establishing normalized company master records. Company contact representatives are modeled through the **Contact** entity, which establishes a one-to-many relationship with **Company**. Statistical survey campaigns are modeled by **Survey**, linked to periodic **SurveyPassage** instances. Each passage maintains relationships to **SampleCompany** (the assigned company sample) and **SurveyQuestionnaire** (reference documents).

3.4.2 Activity Diagram – Use Case “Login”

The activity diagram below illustrates the decision flows, validations, and branching logic during the login process, including OTP verification and first-time login redirection:

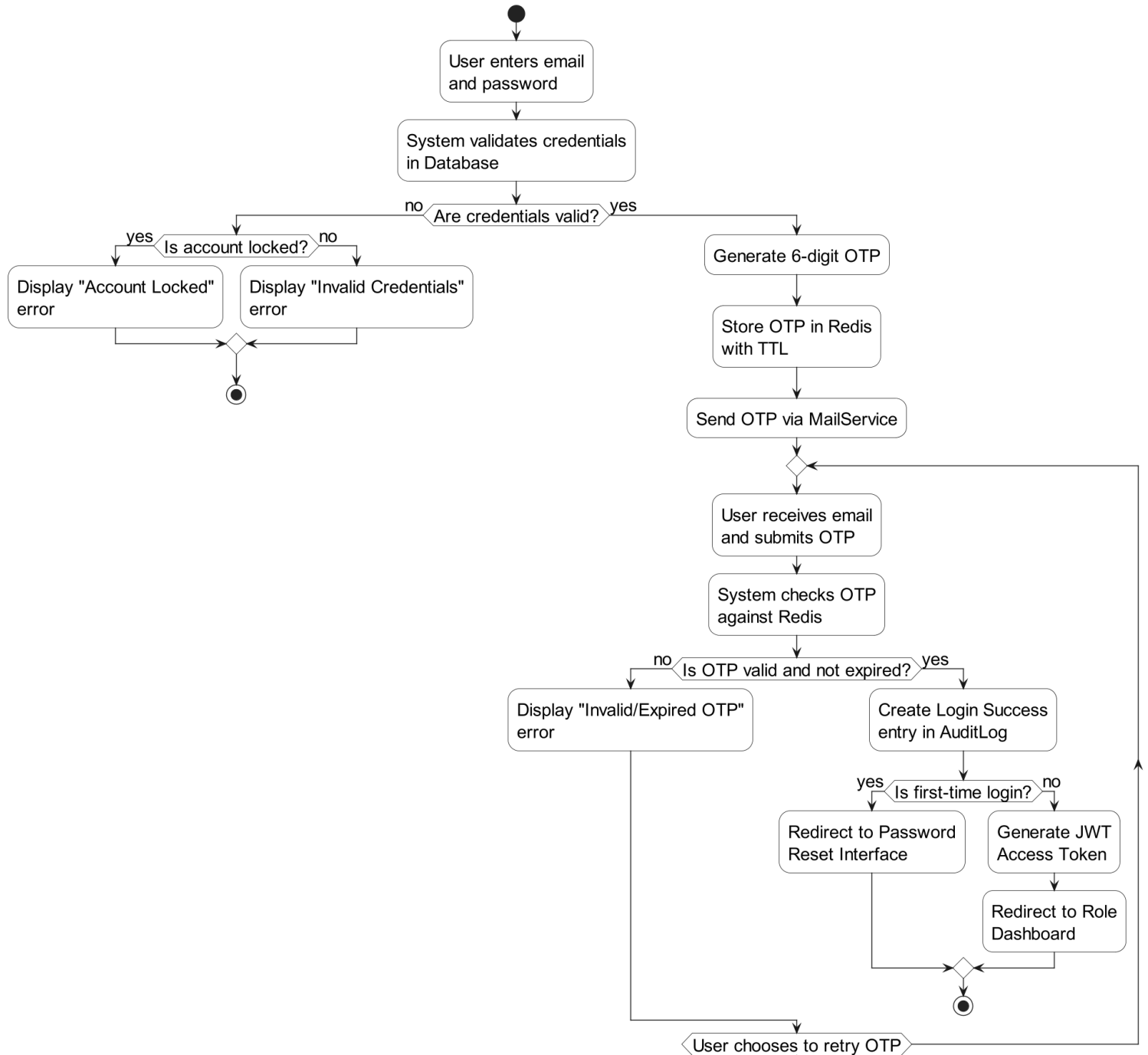


Figure 9: Activity Diagram – Login

3.4.3 Sequence Diagram – Use Case “Manage admins”

The sequence diagram below details administrative account creation and management interactions:

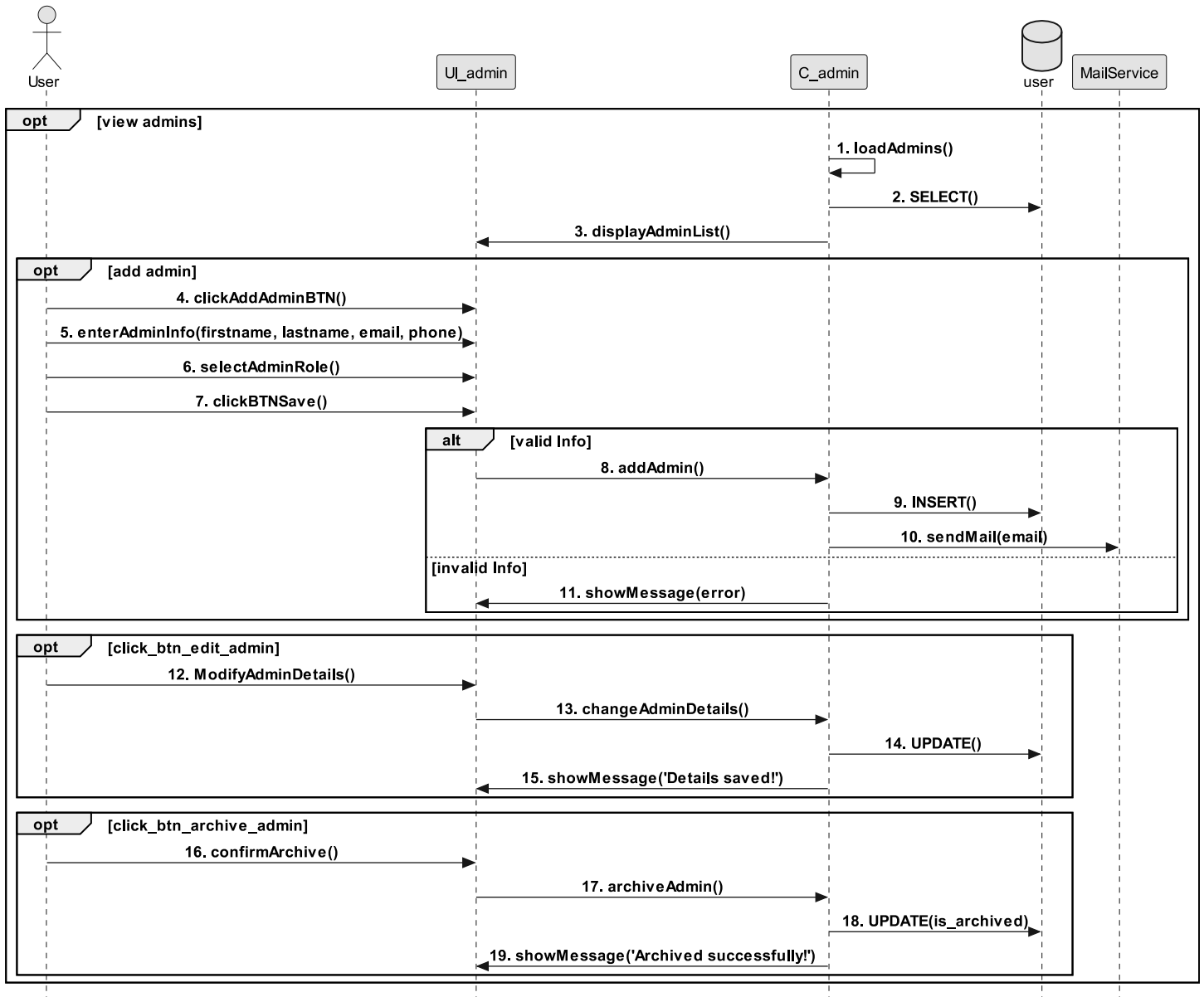


Figure 10: Sequence Diagram – Manage admins

3.4.4 Sequence Diagram – Use Case “Manage contacts”

The sequence diagram below shows contact registration and company assignment interactions:

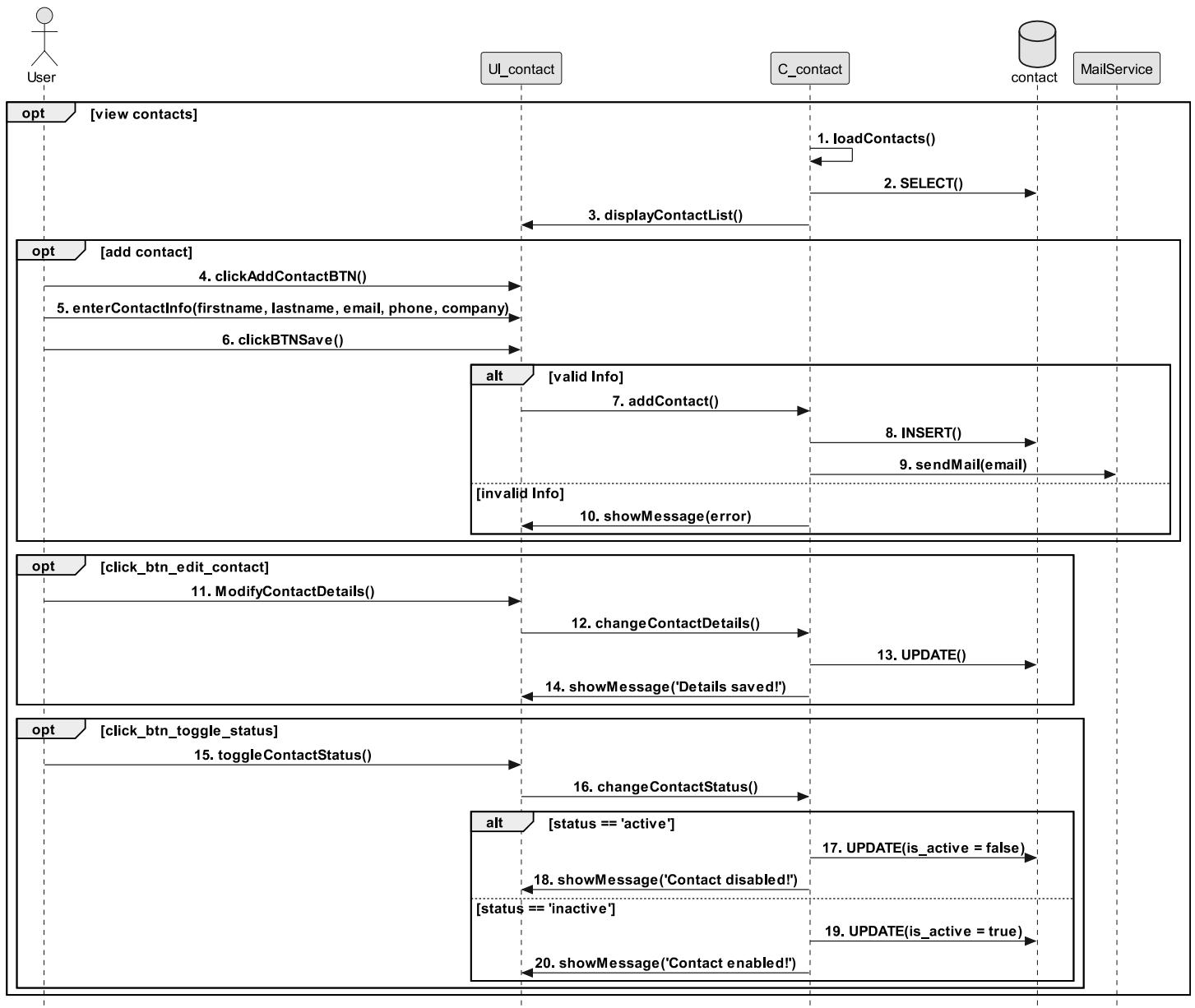


Figure 11: Sequence Diagram – Manage contacts

3.4.5 Sequence Diagram – Use Case “Manage companies”

The sequence diagram below models company profile creation, editing, and directory lookup interactions:

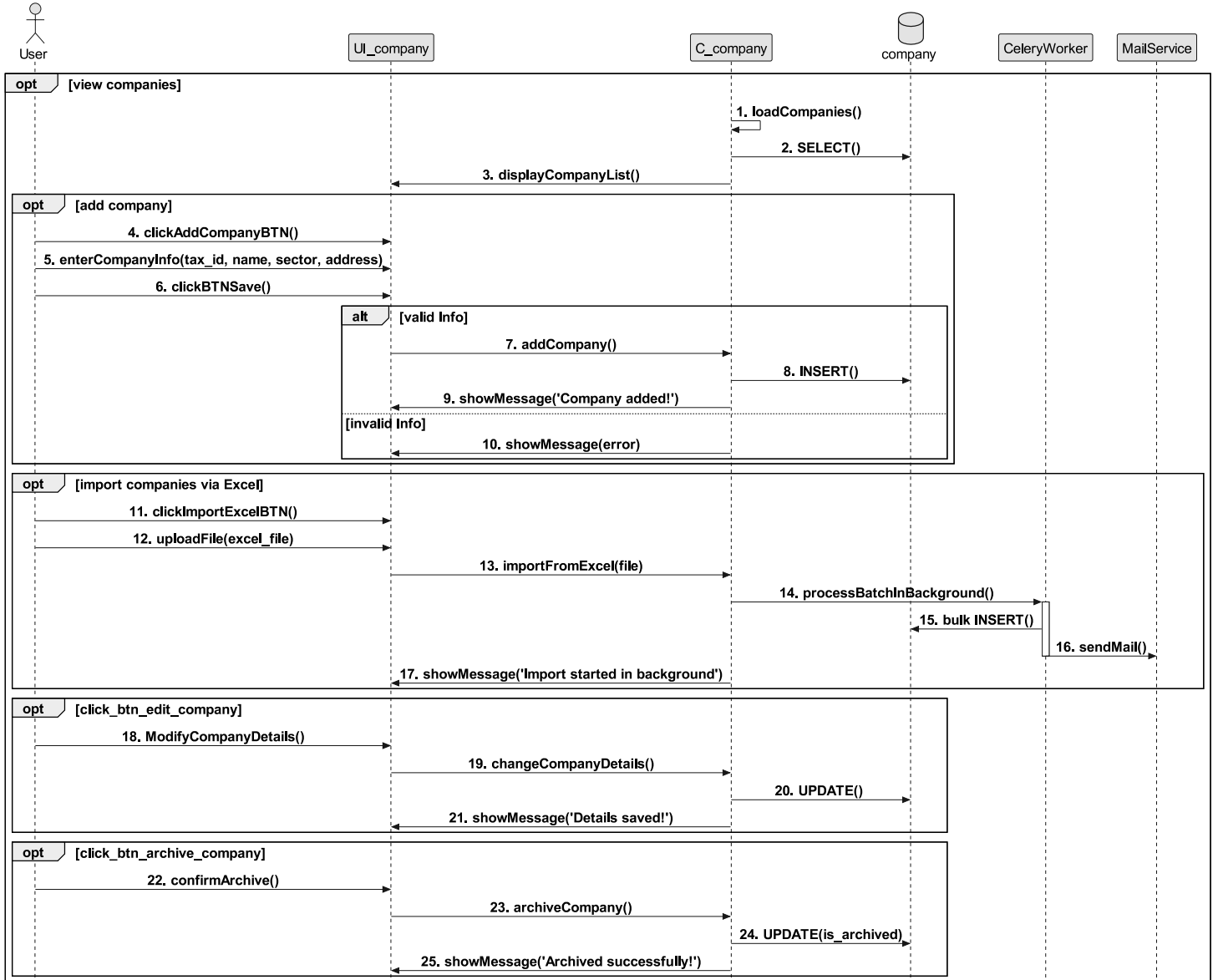


Figure 12: Sequence Diagram – Manage companies

3.4.6 Activity and State Diagrams – Use Case “Manage surveys”

Due to the complexity of the survey management process, the dynamic behavior is modeled using both an Activity Diagram and a State Machine Diagram. The activity diagram illustrates the workflow of defining a survey, configuring passages, uploading templates, and assigning targeted company samples. The state diagram models the lifecycle of a survey passage from draft to archiving.

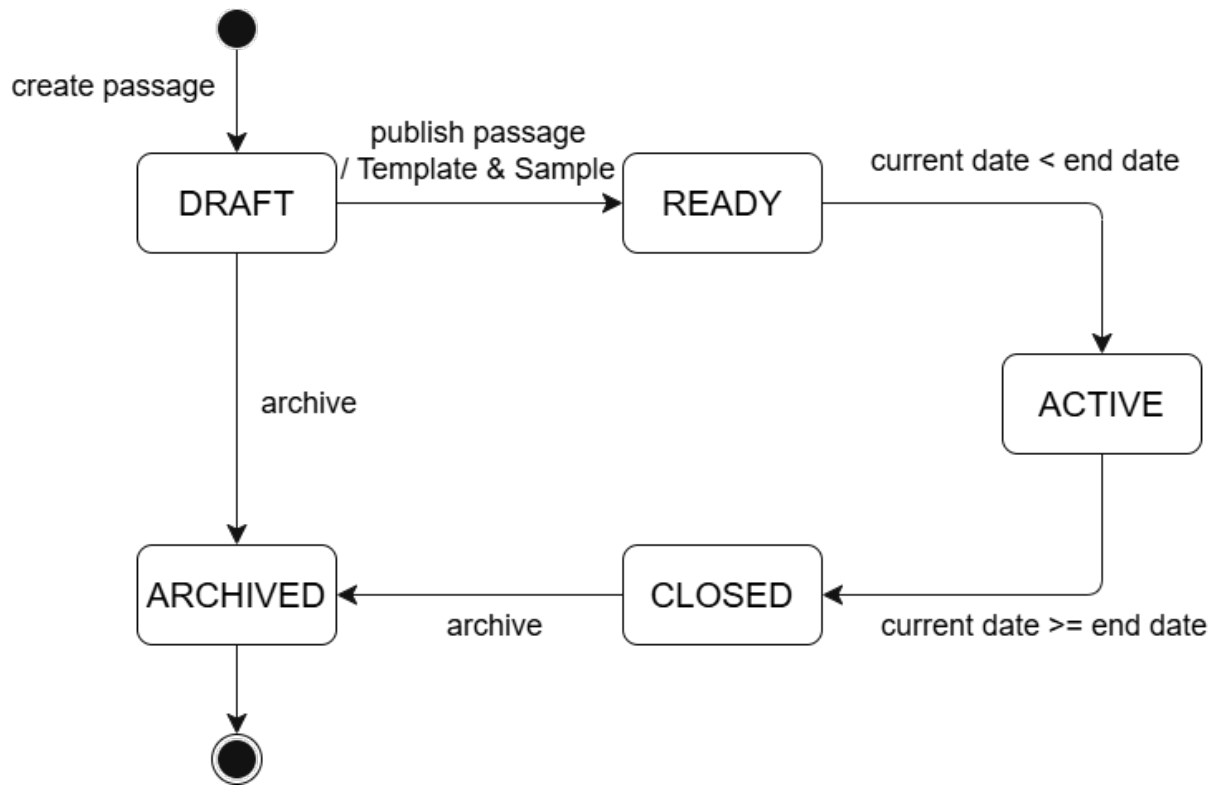


Figure 13: State Diagram – Manage surveys

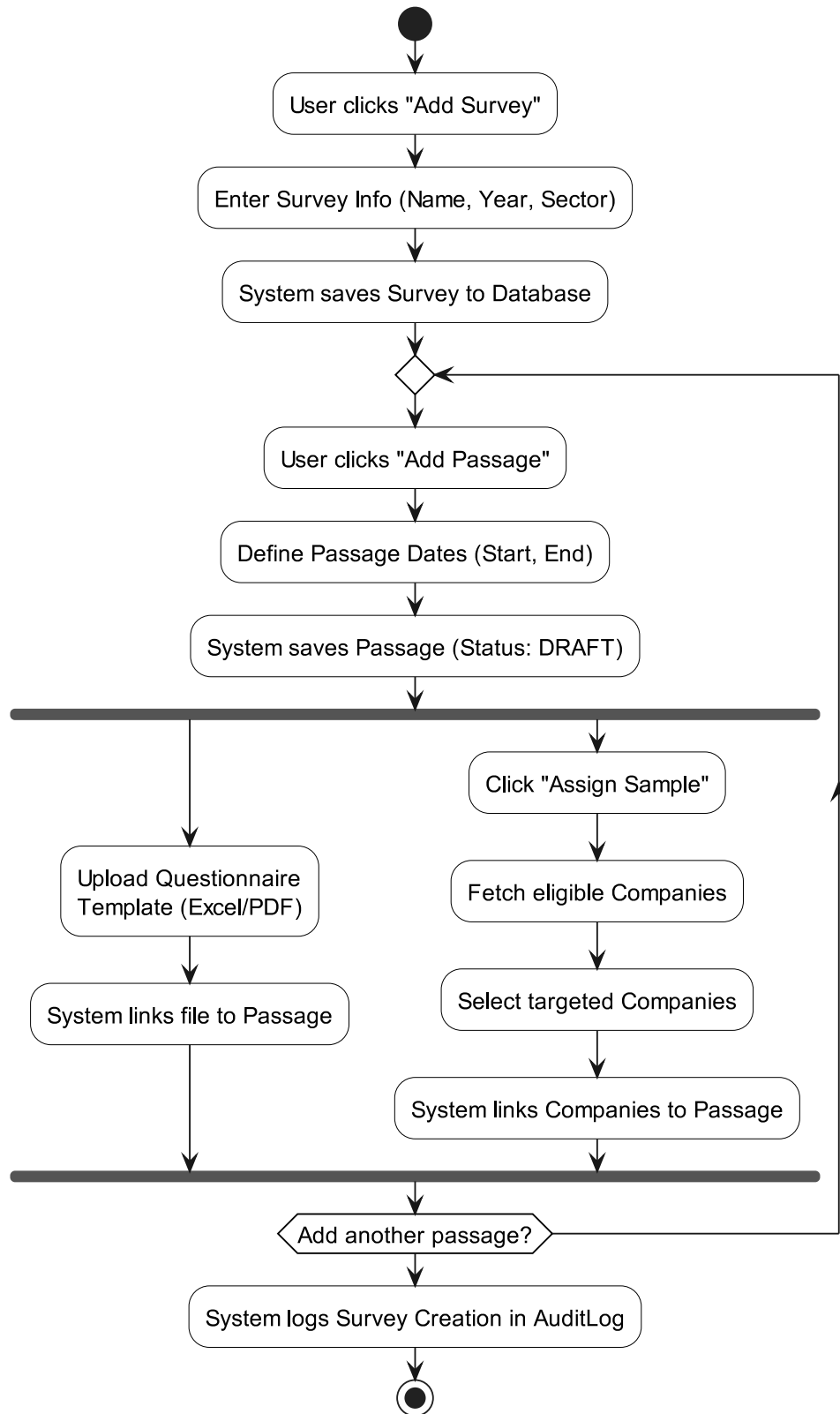


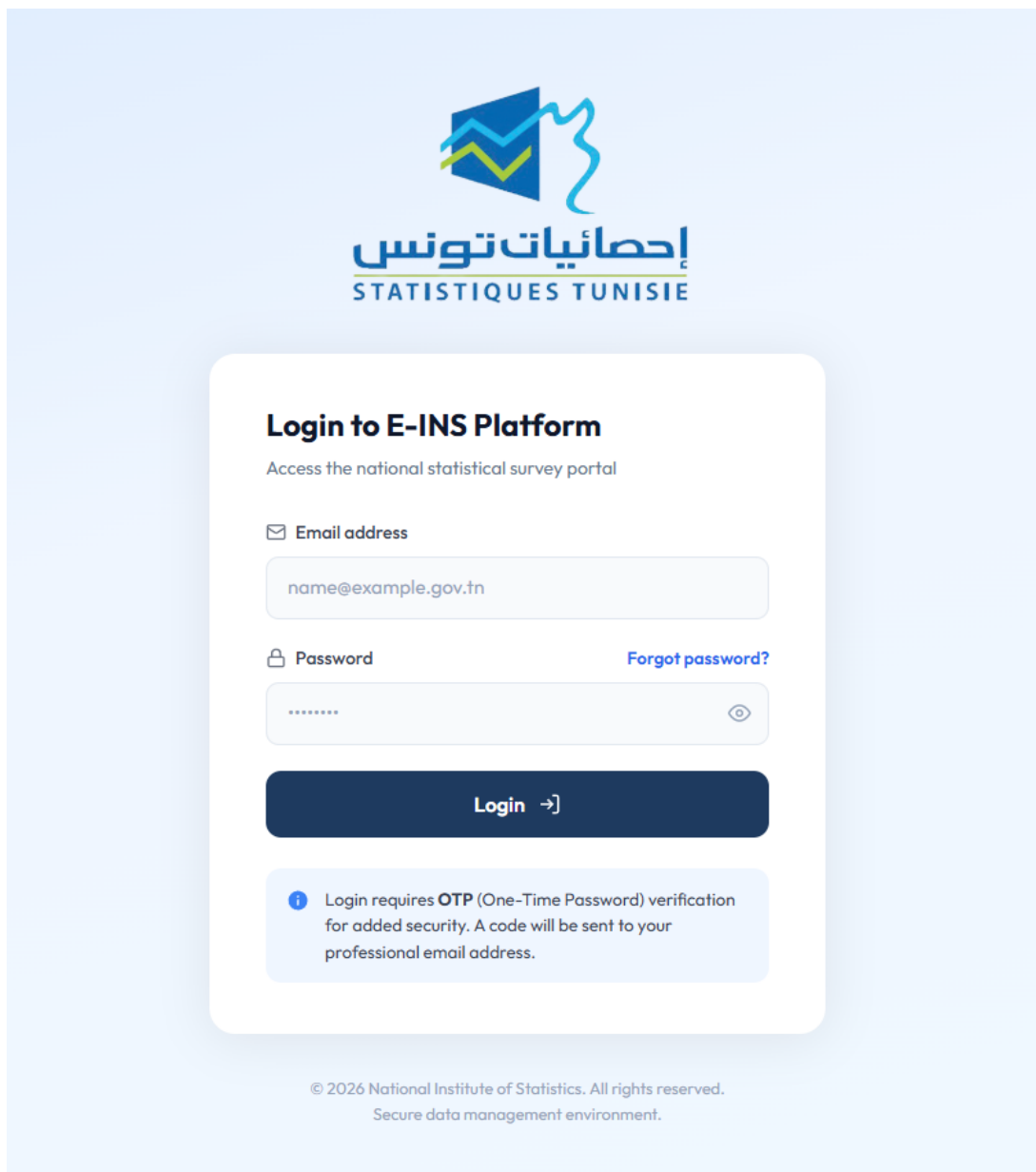
Figure 14: Activity Diagram – Manage surveys

3.5 Sprint 0 Implementation

This section presents the user interface implementations developed during Sprint 0, demonstrating the core authentication and administrative flows.

3.5.1 Login Interface

The authentication module provides secure access to the platform. It handles credential validation, two-factor authentication via OTP, and first-time password resets.



The login interface is displayed on a light blue background. At the top center is the logo for 'إحصائيات تونس' (STATISTIQUES TUNISIE), which features a stylized blue and yellow line graph. Below the logo is a white rounded rectangle containing the login form. The form has a title 'Login to E-INS Platform' and a subtitle 'Access the national statistical survey portal'. It includes two input fields: 'Email address' with a placeholder 'name@example.gov.tn' and 'Password' with a masked password '*****' and a 'Forgot password?' link. A dark blue 'Login →' button is positioned below the password field. A light blue information box at the bottom of the form states: 'Login requires **OTP** (One-Time Password) verification for added security. A code will be sent to your professional email address.' At the very bottom of the page, a footer reads: '© 2026 National Institute of Statistics. All rights reserved. Secure data management environment.'

Figure 15: Login Interface



Two-Factor Auth (OTP)

A 6-digit code has been sent to your email address

admin@ins.tn

✓ Verification code sent to your email address.

Enter security code

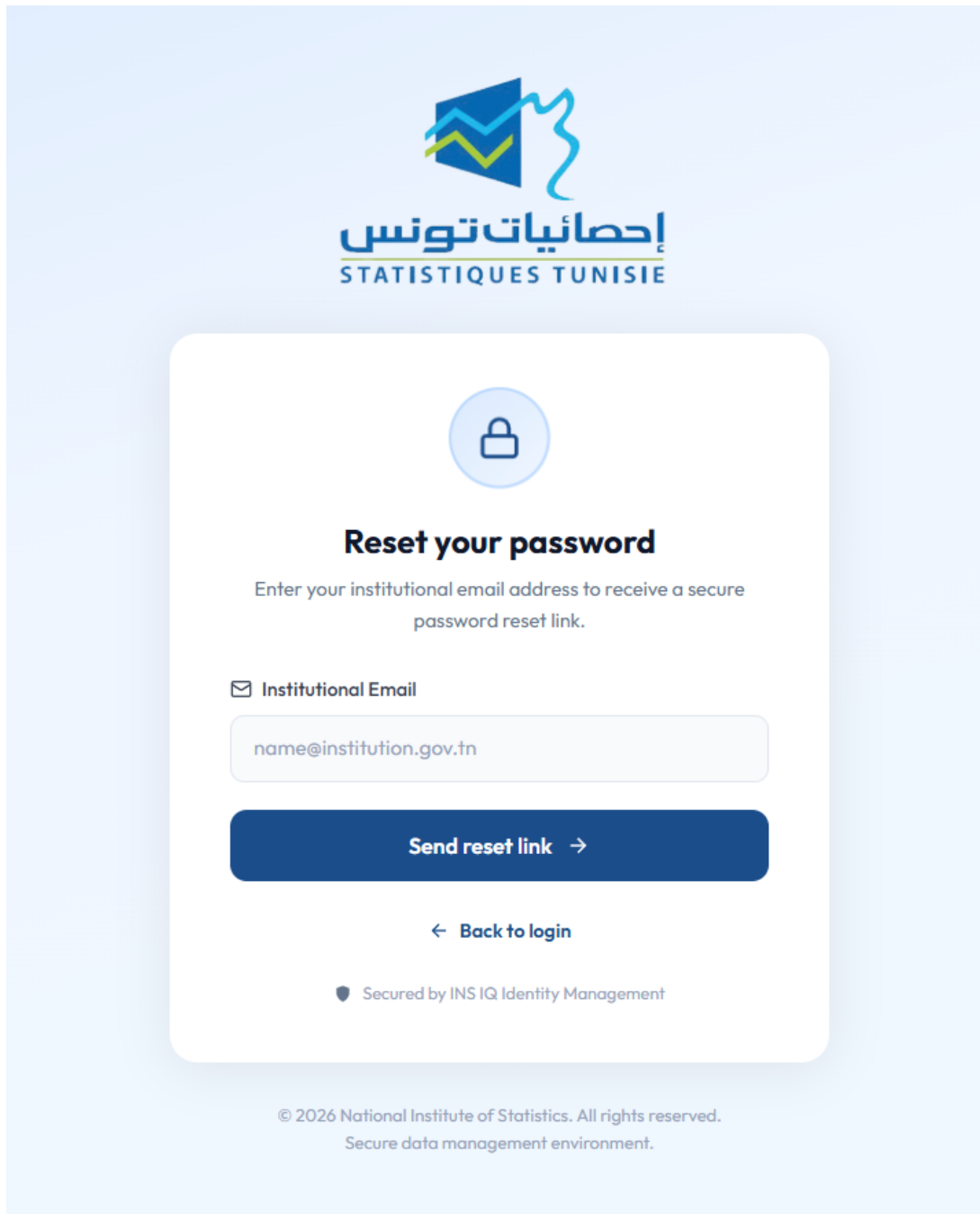
0 0 0 | 0 0 0

✓ Verify Code

Resend Code

← Back to login

Figure 16: OTP Verification Interface



The image shows a web interface for resetting a password. At the top, there is a logo for 'إحصائيات تونس' (Statistics Tunisia) with the text 'STATISTIQUES TUNISIE' below it. The logo consists of a blue square with a white line graph and a blue wavy line. Below the logo, there is a circular icon containing a padlock. The main heading is 'Reset your password'. Below this, there is a text prompt: 'Enter your institutional email address to receive a secure password reset link.' There is a label 'Institutional Email' with an envelope icon. Below the label is a text input field containing 'name@institution.gov.tn'. Below the input field is a blue button with the text 'Send reset link' and a right-pointing arrow. Below the button is a link with a left-pointing arrow and the text 'Back to login'. At the bottom of the form, there is a shield icon and the text 'Secured by INS IQ Identity Management'. Below the form, there is a footer with the text '© 2026 National Institute of Statistics. All rights reserved. Secure data management environment.'

Figure 17: Reset Password Interface

3.5.2 Security Architecture

The platform enforces configurable authentication security parameters at the system-settings level: OTP codes expire after 5 minutes by default, accounts lock after 3 failed sign-in attempts, lockouts last 24 hours, and a 1-hour password-only bypass window is granted after manual administrator unlock. Authentication tokens are issued as JSON Web Tokens (JWT) signed with the HS256 algorithm and a 24-hour expiration duration, configured

via Pydantic Settings loaded from environment variables (`.env`). To ensure immediate access revocation upon logout, each token includes a unique `jti` claim that is blacklisted in Redis until expiration. On the client side, the Angular application manages user session tokens via `localStorage` and injects them automatically into outgoing HTTP headers through a custom HTTP interceptor. API endpoints enforce access control via FastAPI's `Depends(get_current_user)` dependency guard. Listing 3.1 illustrates the backend token creation implementation incorporating the `jti` blacklist mechanism.

```
1 def create_access_token(
2     data: dict,
3     expires_delta: Optional[datetime.timedelta] = None
4 ) -> str:
5     to_encode = data.copy()
6     ttl = expires_delta or datetime.timedelta(
7         minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
8     )
9     expire = datetime.datetime.now() + ttl
10    # Unique JWT ID (jti) for logout blacklisting
11    to_encode.update({
12        "exp": expire,
13        "jti": secrets.token_hex(16),
14    })
15    return jwt.encode(
16        to_encode,
17        settings.SECRET_KEY,
18        algorithm=settings.ALGORITHM
19    )
```

Listing 3.1: JWT Token Generation with Blacklist Support (`app/core/security.py`)

3.5.3 Manage admins Interface

The admin management interface allows the System Administrator to view, filter, create, and update administrative accounts across system roles.

STATISTIQUES TUNISIE

Accueil > Administrateurs

Gestion des Administrateurs

Gérez les administrateurs système et les gestionnaires de compte.

[+ Ajouter Administrateur](#)

Rechercher: [Rechercher](#)

Rôle:

Statut:

[Effacer tous les filtres](#)

NOM	ADRESSE EMAIL	RÔLE	NUMÉRO DE TÉLÉPHONE	STATUT	DERNIÈRE CONNEXION	ACTIONS
test ahmed	ahmed@test.com	Survey Administrator	515151515	Active	Never	Edit Delete
aziz karnit	azizkarnit@gmail.com	Account Manager	93030353	Active	15/07/2026	Edit Delete
aziz aziz	aziz@test.tn	System Administrator	2555151155	Active	Never	Edit Delete

Mon Profil | Déconnexion

Figure 18: Admin Directory Interface

STATISTIQUES TUNISIE

Accueil > Administrateurs > Ajouter un administrateur

Ajouter un administrateur

Prénom:

Nom de famille:

Adresse e-mail:

Numéro de téléphone:

Rôle:

[Annuler](#) [Save](#)

Lorsque vous enregistrez, le système crée le compte, génère un mot de passe temporaire et envoie automatiquement un lien d'activation par e-mail.

Mon Profil | Déconnexion

Figure 19: Add Admin Form

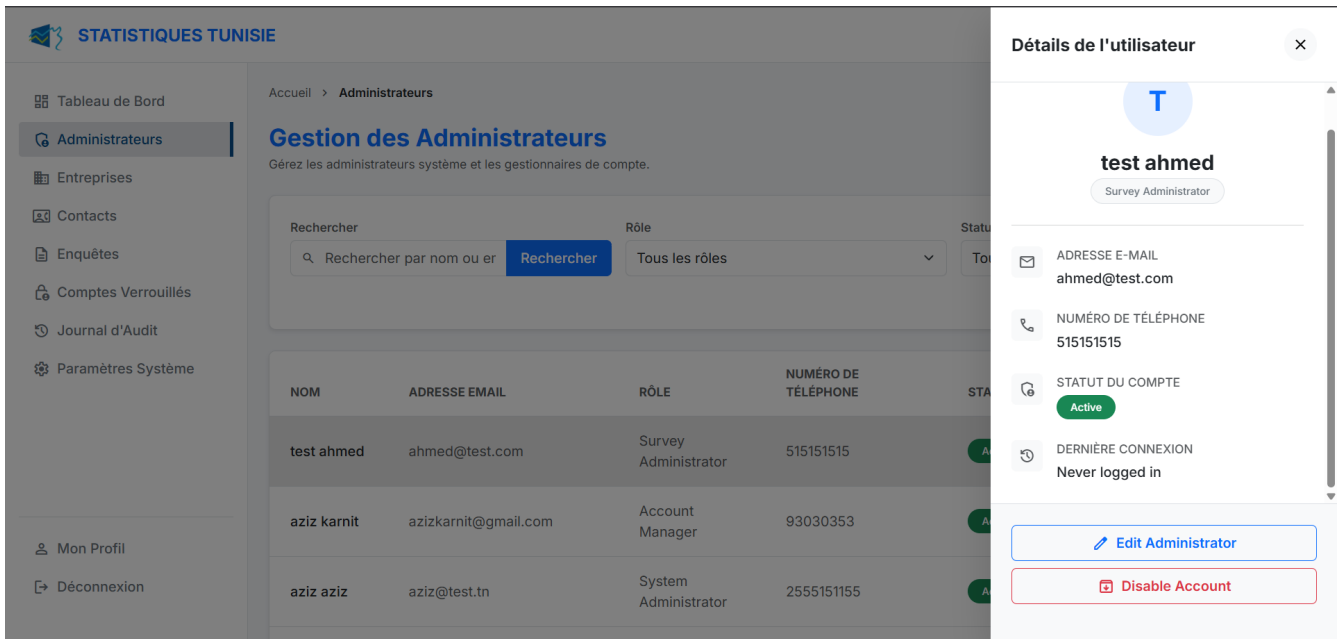


Figure 20: Edit and Disable Admin Actions

3.5.4 Manage contacts Interface

The contact directory interface enables administrators and account managers to maintain company representative profiles and link contacts to surveyed enterprises.

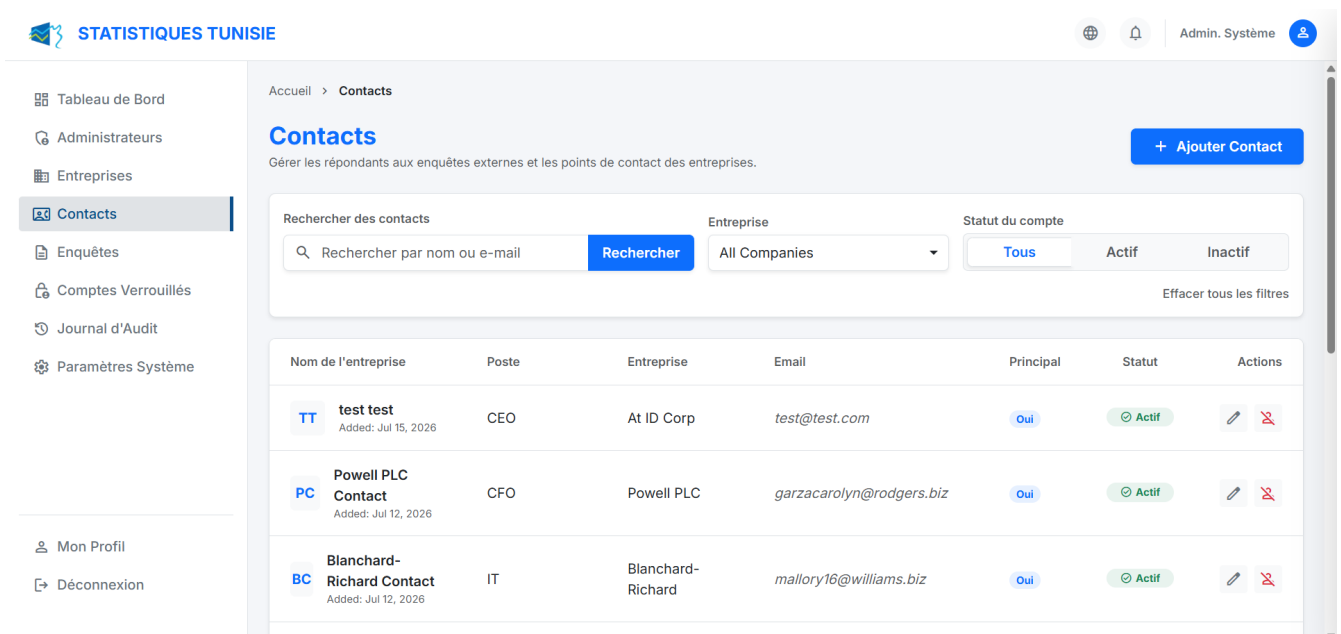


Figure 21: Contact Directory Interface

STATISTIQUES TUNISIE

Accueil > Contacts > **Ajouter un contact**

Ajouter un contact

First name *

Last name *

Email *

Numéro de téléphone

Utilisé pour les notifications

Company *

Poste

☐ **Définir comme contact principal**
Les contacts principaux sont le point de communication principal de leur entreprise.

Lorsque vous enregistrez, le système crée le contact, le lie à l'entreprise, et il recevra les détails d'accès par e-mail.

Figure 22: Add Contact Form

3.5.5 Manage companies Interface

The company registry interface provides centralized administration for company master data. It supports both manual entry and asynchronous bulk Excel imports.

STATISTIQUES TUNISIE

Accueil > **Entreprises**

Annuaire des Entreprises

Gérez toutes les entreprises inscrites.

[Importer Excel](#) [+ Ajouter Entreprise](#)

Rechercher [Rechercher](#)

Secteur

Activité

Gouvernorat

Statut

[Effacer tous les filtres](#)

Matricule	Entreprise	Secteur	Activité	Gouvernorat	Statut	Actions
ENT-0210	None Corp	Agriculture	Culture Agricole	N/A	Active	Edit Delete
ENT@0202	At ID Corp	Technologie	Développement de Logiciels	N/A	Active	Edit Delete
ENT 0201	Space ID Corp	Technologie	Développement de Logiciels	N/A	Active	Edit Delete
ENT/0200	Slash ID Corp	Technologie	Développement de Logiciels	N/A	Active	Edit Delete

Figure 23: Company Directory Interface

STATISTIQUES TUNISIE

Accueil > Entreprises > Ajouter une entreprise

Ajouter une entreprise

Identifiant * Identifiant unique	Nom de l'entreprise * Nom de l'entreprise
Matricule Fiscale Matricule fiscal	Adresse e-mail contact@entreprise.com
Numéro de téléphone +216 00 000 000	Adresse Adresse physique
Gouvernorat Sélectionner un gouvernorat...	Code postal Postal code
Secteur * Sélectionner un secteur...	Activité * Sélectionner une activité...

Mon Profil | Déconnexion

Figure 24: Add Company Form

STATISTIQUES TUNISIE

Accueil > Entreprises > Import from Excel

Importer des entreprises depuis Excel

- 1 Importer
- 2 Réviser
- 3 Confirmer

Déposez votre fichier Excel ici ou cliquez pour choisir

Formats supportés : .XLSX, .XLS (Max 500MB)

[Download Excel template](#) (Standard Schema v2.1)

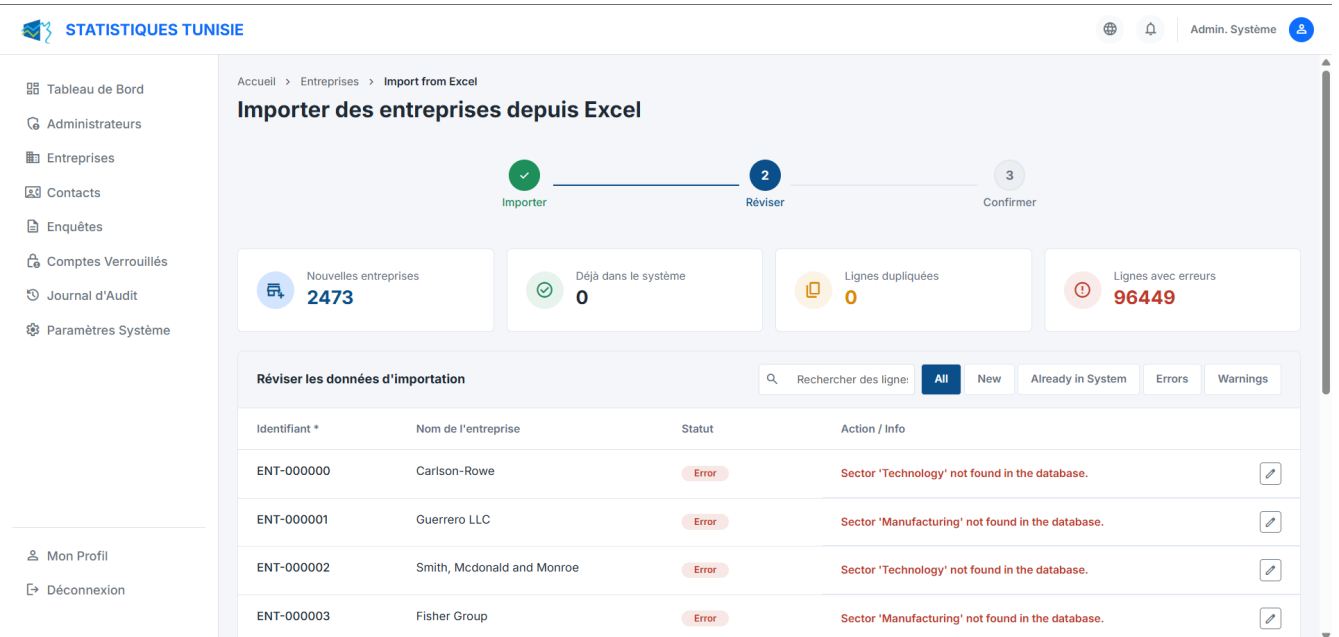
Data Requirements
 Assurez-vous que le Nom de l'entreprise et l'Identifiant sont présents dans toutes les lignes pour une validation

Batch Limit
 Vous pouvez importer jusqu'à 500 entreprises en une seule opération.

Privacy
 Toutes les données téléchargées sont chiffrées et traitées selon les normes gouvernementales.

Mon Profil | Déconnexion

Figure 25: Batch Import - Upload File



To validate the reliability of the asynchronous company import pipeline prior to production use, a synthetic dataset generator was developed using `openpyxl`’s write-only streaming mode, enabling files of realistic scale ($\approx 100\text{MB}$) to be produced without exhausting memory during generation. The generator accepts parameters controlling the exact distribution of valid, warning, error, and duplicate rows, producing reproducible, ground-truth-labeled test files for stress testing under adversarial conditions. Listing 3.2 highlights the Celery task logic for asynchronous bulk spreadsheet processing.

```

1 @celery_app.task
2 def process_excel_import(session_id: str, file_path: str):
3     db: Session = SessionLocal()
4     try:
5         session = db.query(ImportSession).filter(
6             ImportSession.id == session_id
7         ).first()
8         wb = openpyxl.load_workbook(
9             file_path, read_only=True, data_only=True
10        )
11        ws = wb.active
12        # Read and validate rows into staging table
13        for i, row_vals in enumerate(data_rows, start=2):
14            staging_records.append(CompanyImportStaging(...))
15            if len(staging_records) >= 2000:
16                db.bulk_save_objects(staging_records)

```

```

17         db.commit()
18         staging_records = []
19     finally:
20         db.close()

```

Listing 3.2: Asynchronous Company Import Task Excerpt
(app/tasks/import_tasks.py)

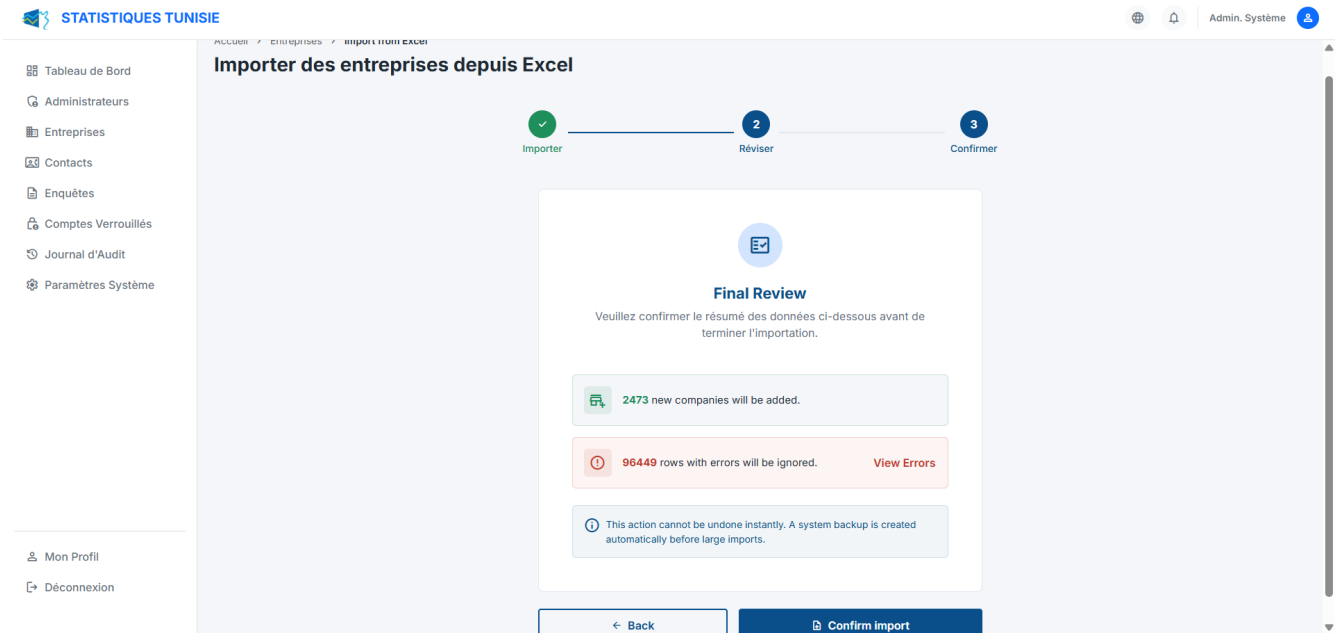


Figure 27: Batch Import - Processing

3.5.6 Manage surveys Interface

The survey management interface enables administrators and survey managers to comprehensively configure statistical campaigns, define passage periods, assign company samples, and monitor results.

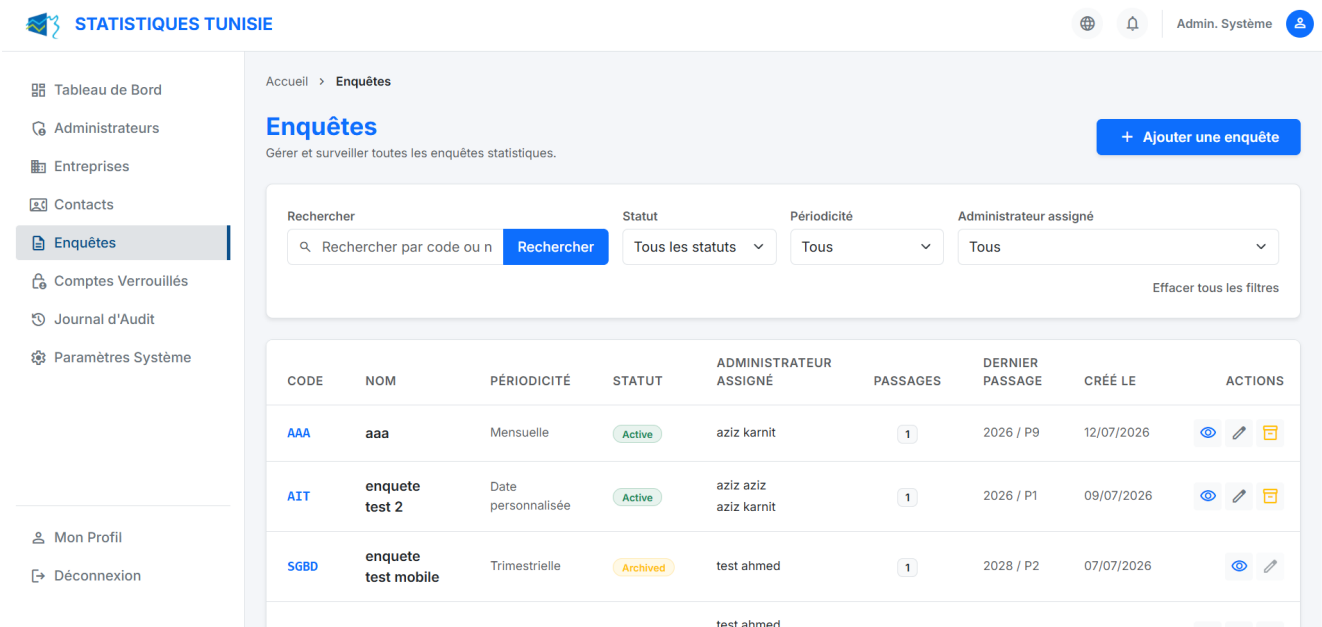


Figure 28: Survey Directory Interface

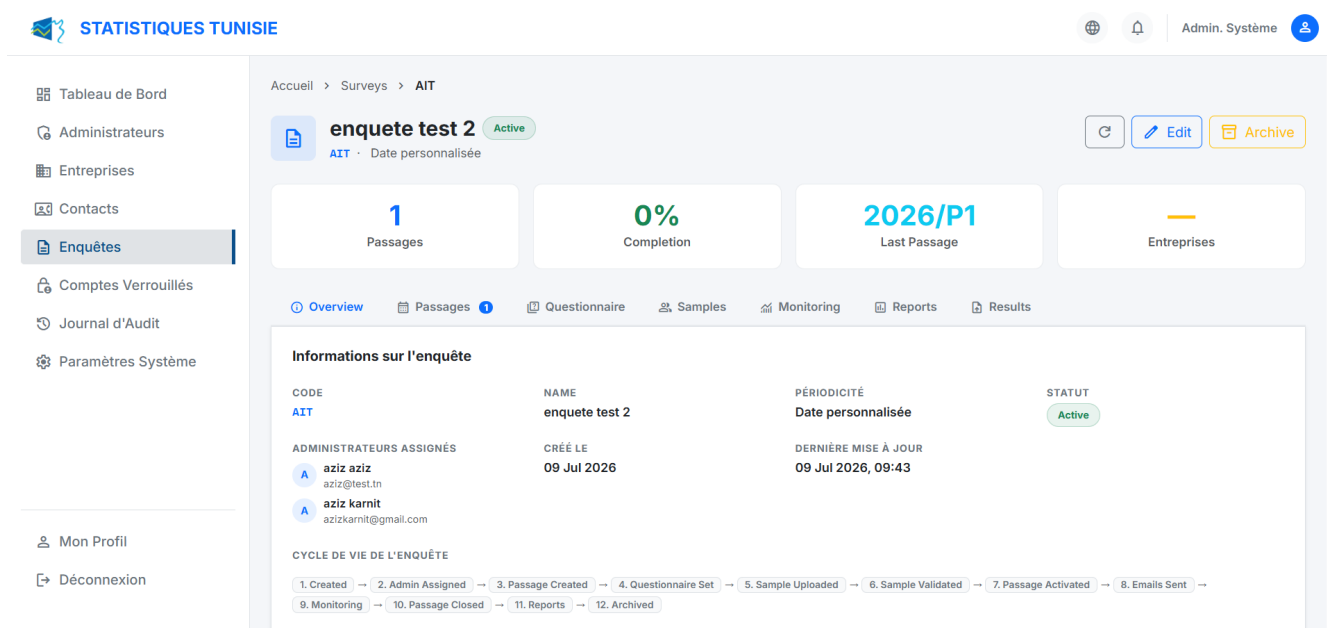


Figure 29: Survey Overview Dashboard

Modifier l'enquête
Code: AAA (cannot be changed)

Survey Name *
aaa

Description

Periodicity *
Mensuelle

Statut
Actif

Administrateurs assignés

- ☐ Aaron Douglas
pattonjessica@example.org
- ☐ Anthony Torres
stevensonbrenda@example.org
- ☐ aziz aziz
aziz@test.tn

Figure 30: Edit Survey Configuration

New Passage

Year *
2026

Passage # *
3

Opening Date *
07/01/2026 12:00 AM

Auto-calculated based on periodicity.

Closing Date *
09/30/2026 11:59 PM

Cancel Create Passage

Figure 31: Add Survey Passage Form

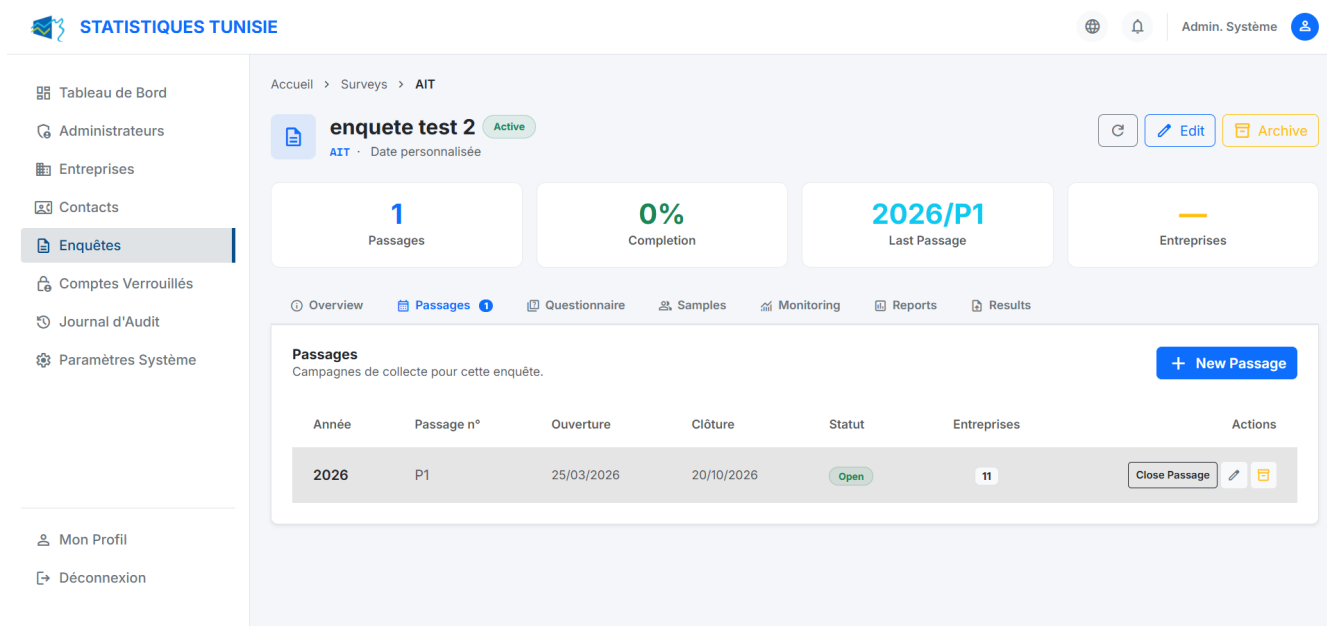


Figure 32: Survey Passage Details

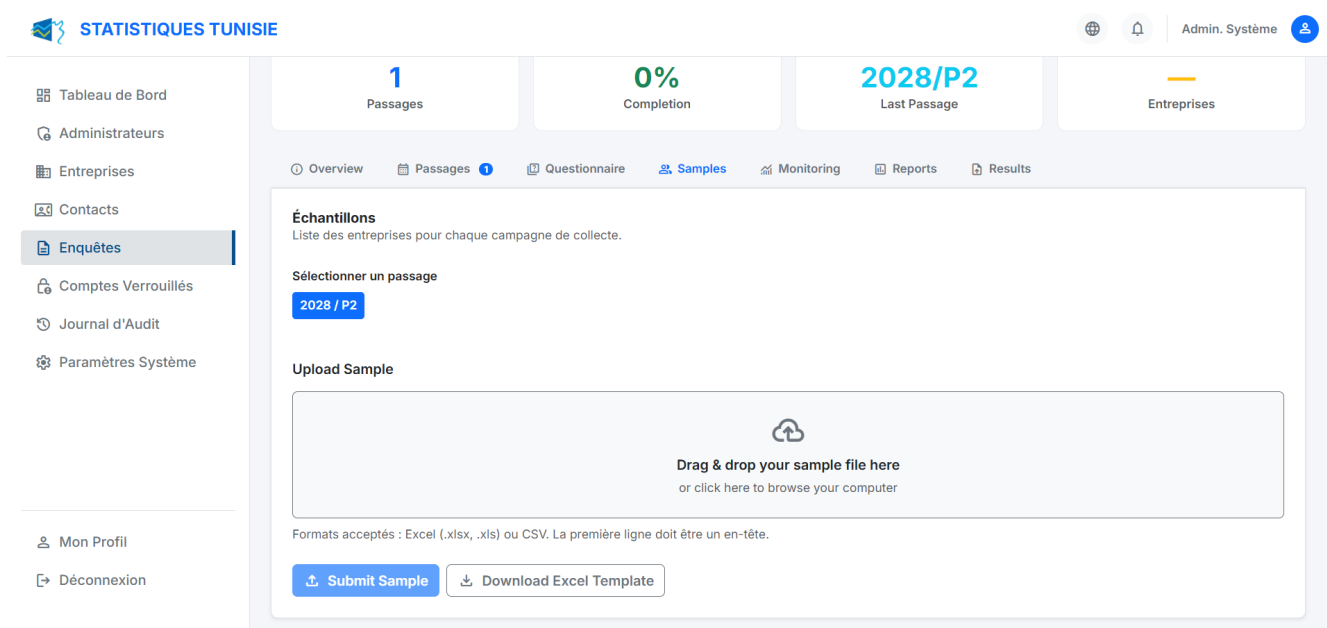


Figure 33: Company Sample Assignment Interface

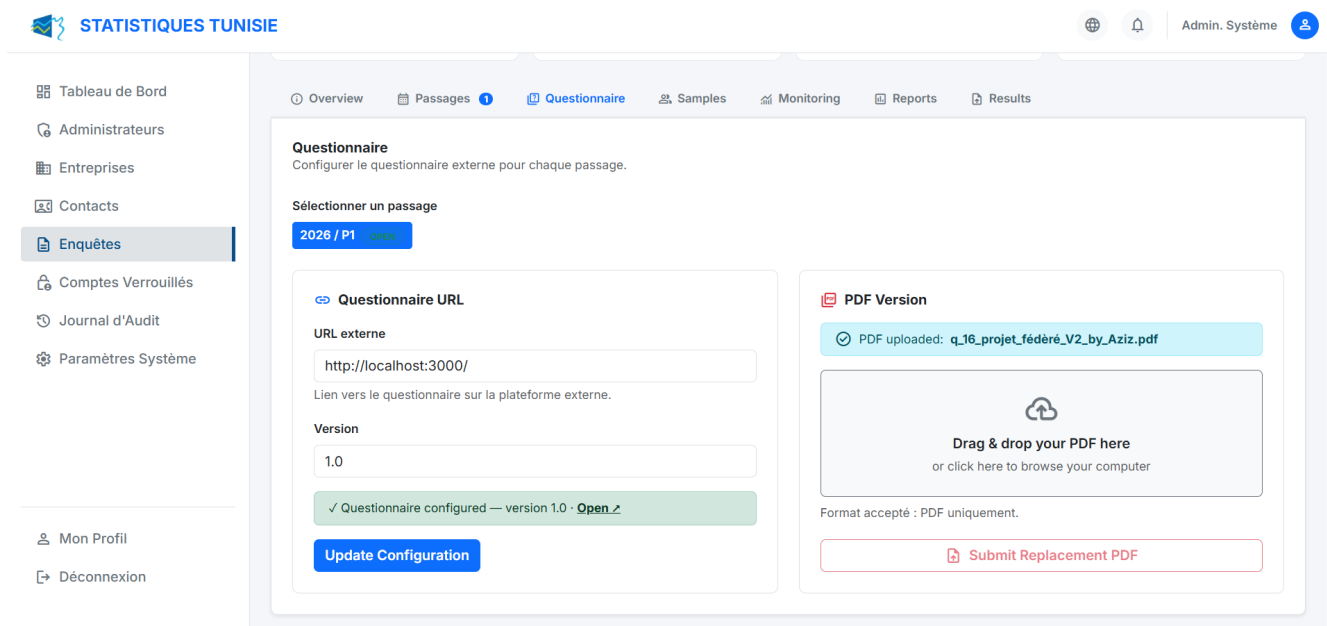


Figure 34: Questionnaire Template Management

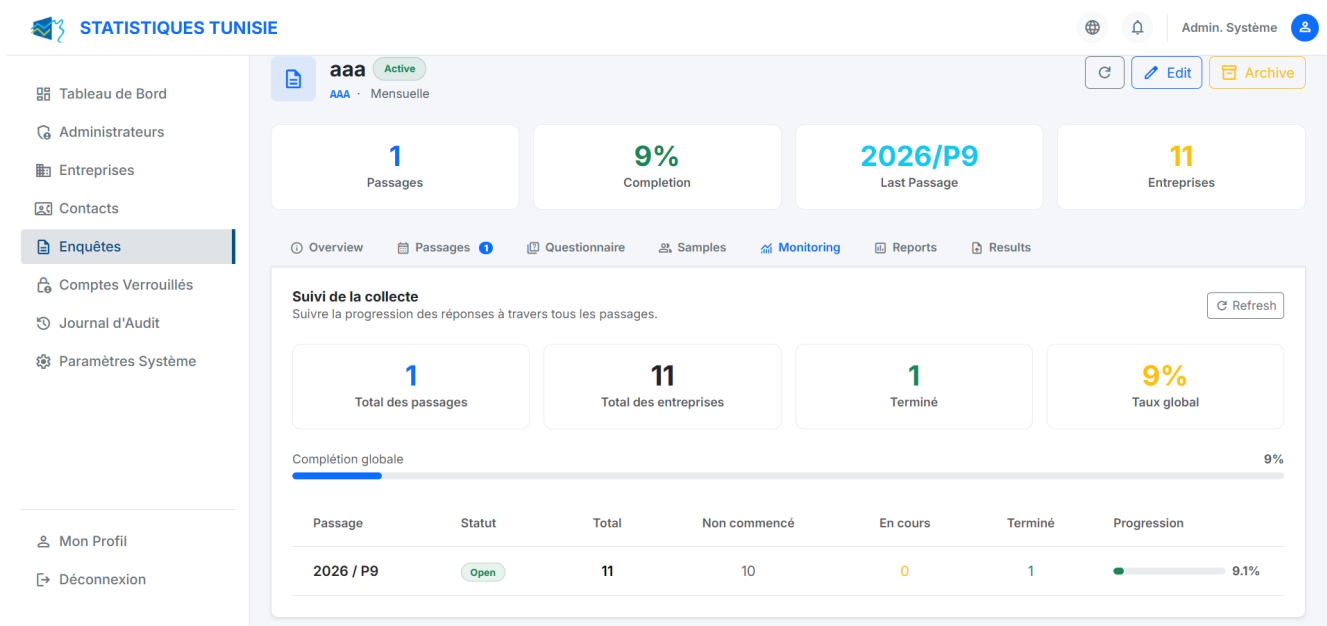


Figure 35: Survey Passage Monitoring Dashboard

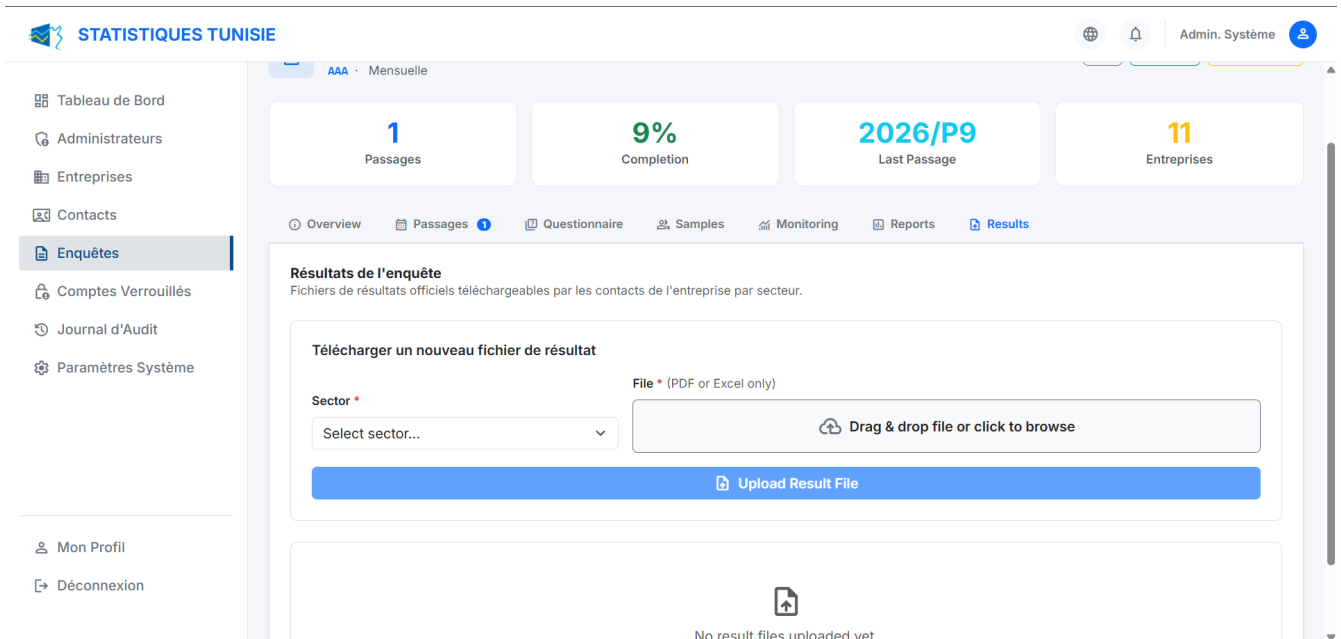


Figure 36: Survey Results Interface

3.6 Sprint 0 Conclusion

In this chapter, we detailed the development of Sprint 0, which established the authentication system and core administrative management features of the platform. We presented the sprint backlog, refined each use case with detailed textual specifications, designed structural and sequence models, and demonstrated the implemented user interfaces. As noted in sprint retrospectives, fine-tuning the 2FA OTP expiration and Redis account lockout synchronization required additional validation iterations beyond initial estimates.

All planned user stories for Sprint 0 (“Login”, “Manage admins”, “Manage contacts”, “Manage companies”, and “Manage surveys”) were successfully developed and integrated, establishing a solid foundation for the platform. In the next chapter, we will address Sprint 1, focusing on survey administration and contact survey submission portals.

Chapter 4

Sprint 1 – Survey Portals & Account Management

4.1 Introduction

This chapter covers the development of Sprint 1, which focuses on the core survey submission portal for company contacts and additional secondary administrative features. It details the sprint backlog, the refinement of each user story, dynamic flow models, and the implementation of the user interfaces.

4.2 Sprint 1 Backlog

The Sprint 1 backlog encompasses the functional requirements assigned to the first development cycle following the setup of the core administration portal.

Table 18: Sprint 1 Backlog

ID	User Story	Priority
US06	As a Company Contact, I want to Consult assigned surveys to view my active questionnaires.	High
US07	As a Company Contact, I want to Consult assigned results to check my past survey responses.	High
US08	As a System Administrator, I want to Manage system settings to configure global parameters.	Medium
US09	As a System Administrator or Account Manager, I want to Manage locked accounts to restore or restrict system access.	Medium
US10	As a user, I want to Manage profile to modify my personal contact information.	Low

4.3 Sprint 1 Refinement

In this section, we refine the functional details and textual specifications for the use cases of Sprint 1.

4.3.1 Refinement of the Use Case “Consult assigned surveys”

The diagram below illustrates the refinement of the “Consult assigned surveys” use case and its interactions with system actors:

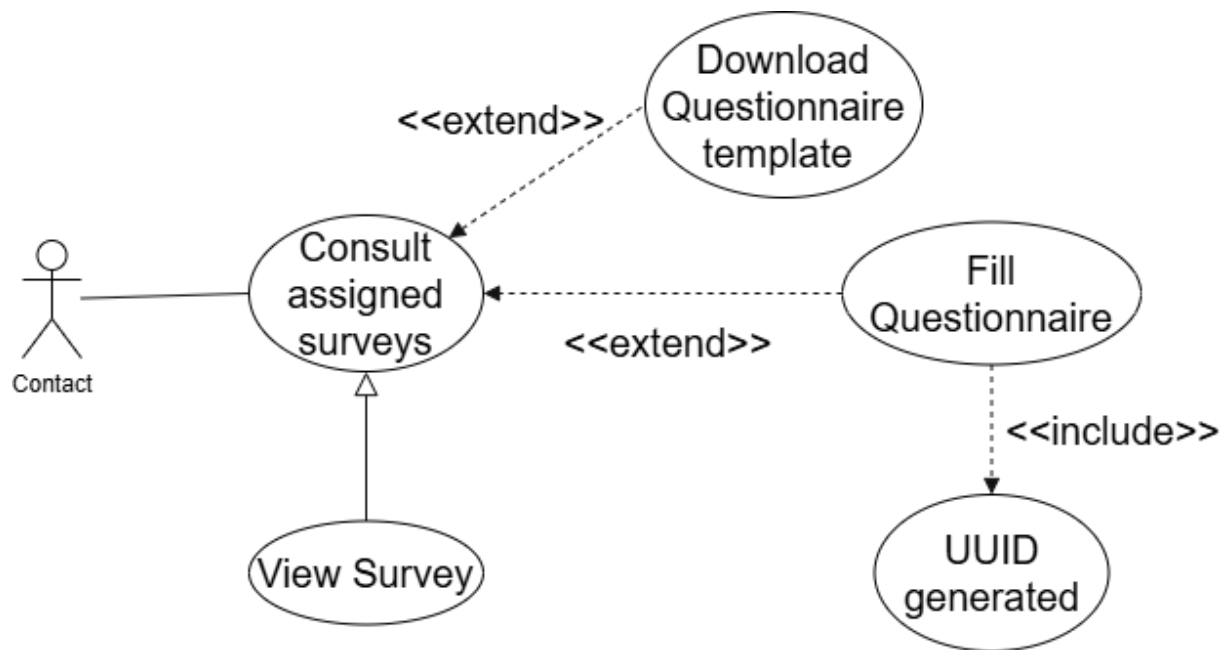


Figure 37: Use Case Diagram – Consult assigned surveys

Table 19 presents the textual description of the “Consult assigned surveys” use case:

Table 19: Textual description – Consult assigned surveys

Use Case	Consult assigned surveys
Actor	Company Contact
Pre-condition	The Company Contact is authenticated and active on the platform.
Post-condition	The contact can view active surveys, download the required reference templates, and upload their completed questionnaire.
Main Scenario	1. The contact receives an email notification upon being assigned to a survey passage. 2. The contact navigates to the “My Surveys” portal. 3. The system retrieves and lists all active survey passages assigned to the contact’s company. 4. The contact selects a specific survey passage to view details. 5. The system displays passage deadlines and provides two options: “Download Template” and “Fill Questionnaire”. 6. (Optional) The contact downloads the template to prepare answers offline. 7. The contact clicks “Fill Questionnaire”. 8. The system generates an anonymous UUID based on the contact and passage to ensure data confidentiality. 9. The system redirects the contact to the external questionnaire system, passing the UUID. 10. The contact fills the questionnaire in the external system (with the ability to save progress and return later).
Alternative Scenarios	• No active surveys: The system displays a message indicating no pending surveys are assigned. • External system unavailable: The platform displays a maintenance warning and asks the user to try again later.
Exception	Server connection drop during file upload; the system displays a network retry prompt.

4.3.2 Refinement of the Use Case “Consult assigned results”

The diagram below illustrates the refinement of the “Consult assigned results” use case and its interactions with system actors:

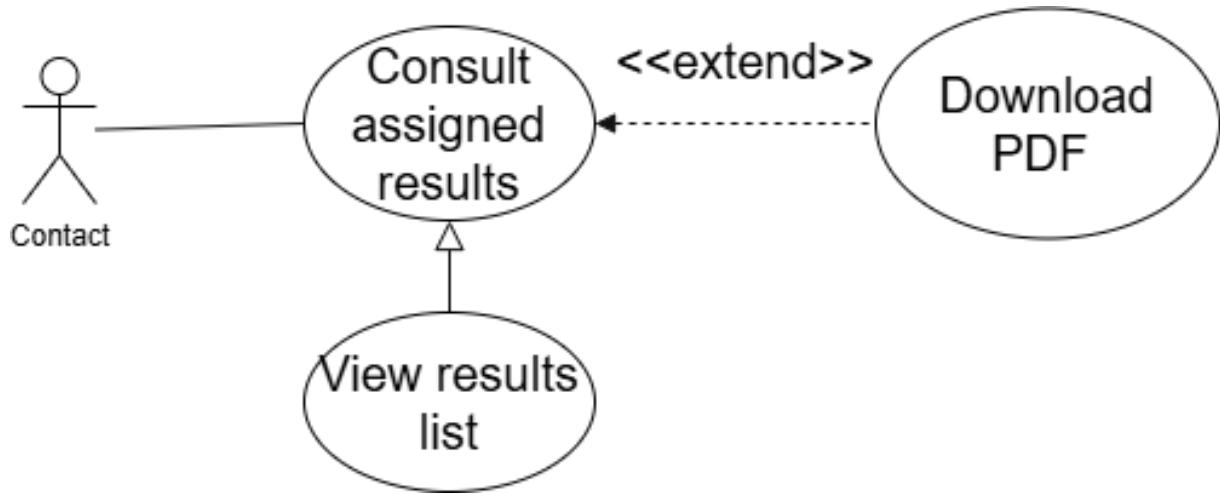


Figure 38: Use Case Diagram – Consult assigned results

Table 20 presents the textual description of the “Consult assigned results” use case:

Table 20: Textual description – Consult assigned results

Use Case	Consult assigned results
Actor	Company Contact
Pre-condition	The Company Contact is authenticated and has previously submitted surveys.
Post-condition	The contact successfully views historical submission data and validation feedback.
Main Scenario	1. The contact accesses the “Survey Results” section. 2. The system retrieves past survey submissions associated with the contact’s company. 3. The system displays a list of past survey results containing: Survey Name, File Name, Format, Published Date, and an Action column. 4. The contact clicks the “Download” action for a specific result. 5. The system initiates the download of the submitted file.
Alternative Scenarios	• No past submissions: The system displays a “No historical data” placeholder.
Exception	Database timeout; the system prompts the user to refresh the page.

4.3.3 Refinement of the Use Case “Manage system settings”

The diagram below illustrates the refinement of the “Manage system settings” use case and its interactions with system actors:

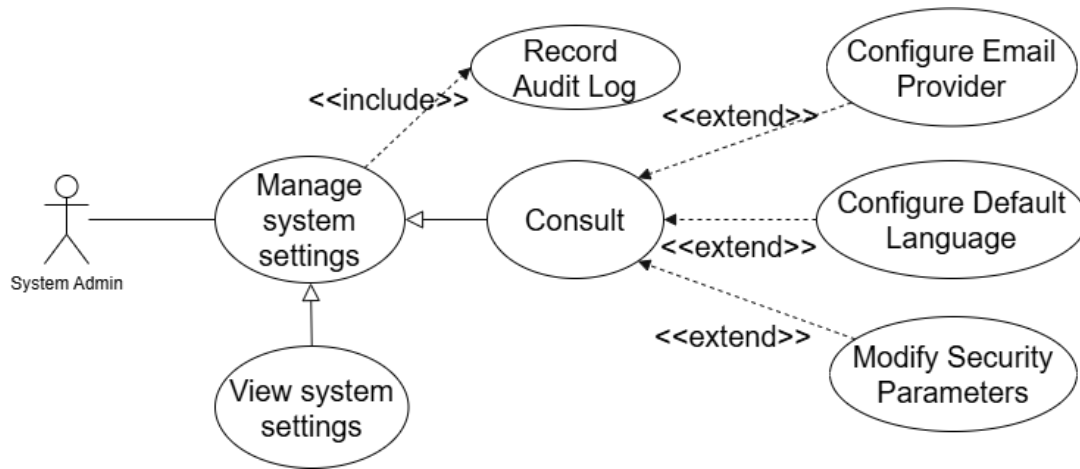


Figure 39: Use Case Diagram – Manage system settings

Table 21 presents the textual description of the “Manage system settings” use case:

Table 21: Textual description – Manage system settings

Use Case	Manage system settings
Actor	System Administrator
Pre-condition	The user is logged in as a System Administrator.
Post-condition	Global system parameters (Security protocols, Default Language, Email Provider) are securely updated.
Main Scenario	1. The administrator navigates to the “System Settings” dashboard. 2. The system loads the current configuration parameters categorized by Security, Language, and Email. 3. The administrator modifies a specific parameter value (e.g., 2FA code expiration, max login attempts, lockout duration, or default language). 4. The administrator saves the new configuration. 5. The system validates the numerical inputs and format constraints. 6. The system updates the parameters in the database and refreshes the global application state. 7. The system logs the configuration change in the audit trail.
Alternative Scenarios	• Invalid input format: The system rejects the change (e.g., providing a negative number for lockout duration) and displays a validation error.
Exception	Cache server unreachable; the system saves to DB but warns about delayed application.

4.3.4 Refinement of the Use Case “Manage locked accounts”

The diagram below illustrates the refinement of the “Manage locked accounts” use case and its interactions with system actors:

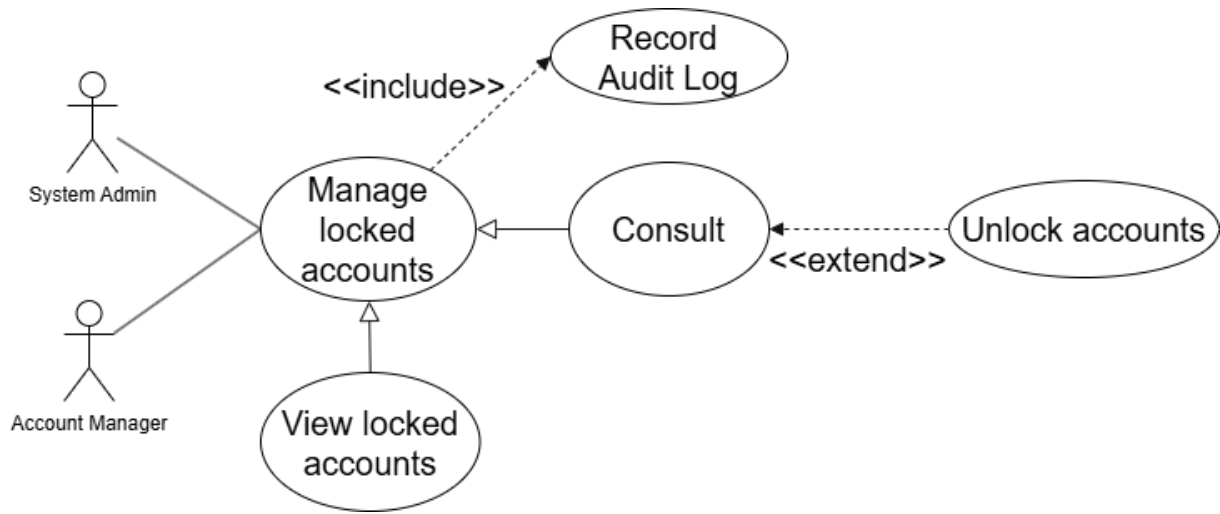


Figure 40: Use Case Diagram – Manage locked accounts

Table 22 presents the textual description of the “Manage locked accounts” use case:

Table 22: Textual description – Manage locked accounts

Use Case	Manage locked accounts
Actor	System Administrator, Account Manager
Pre-condition	The user has appropriate administrative privileges.
Post-condition	A user’s account is unlocked, allowing them to authenticate, and the action is logged.
Main Scenario	1. The manager navigates to the “Locked Accounts” security monitoring interface. 2. The system displays a list of accounts suspended due to multiple failed login attempts. 3. The manager clicks “Unlock” on a specific user’s row. 4. The manager confirms the unlock action. 5. The system resets the failed attempt counter and sets the account status to active. 6. The system grants a temporary password-only login bypass window. 7. The system records the unlock action in the security audit logs.
Alternative Scenarios	• Admin decides to permanently disable: The manager selects “Disable permanently” instead of unlocking.
Exception	Database timeout; the action fails and the manager is prompted to retry.

4.3.5 Refinement of the Use Case “Manage profile”

The diagram below illustrates the refinement of the “Manage profile” use case and its interactions with system actors:

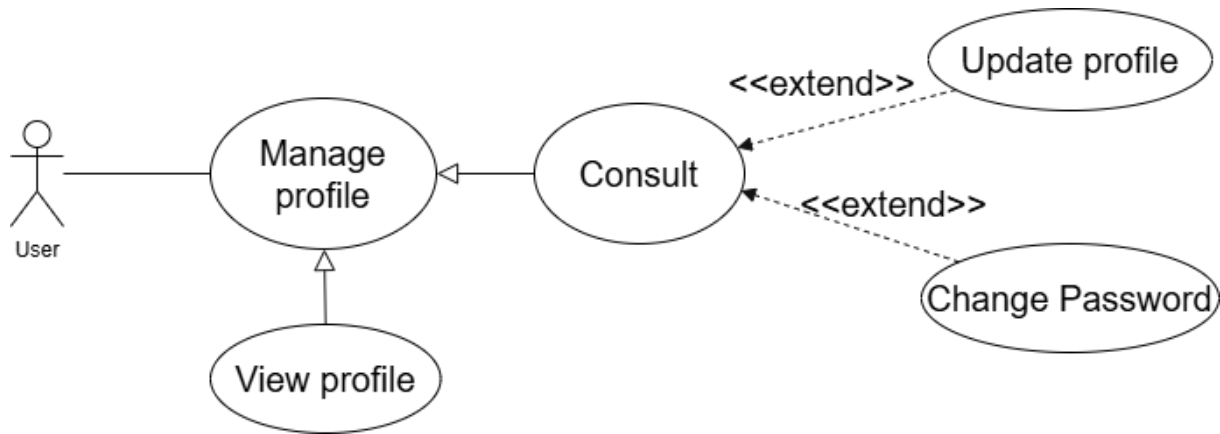


Figure 41: Use Case Diagram – Manage profile

Table 23 presents the textual description of the “Manage profile” use case:

Table 23: Textual description – Manage profile

Use Case	Manage profile
Actor	Any authenticated user
Pre-condition	The user is securely authenticated on the platform.
Post-condition	The user’s personal details or password are updated in the database.
Main Scenario	1. The user clicks on their avatar and accesses the “My Profile” interface. 2. The system loads the user’s current personal data. 3. The user modifies their phone number, interface language, or requests a password change. 4. The user clicks “Save Changes”. 5. The system validates the input (e.g., verifying old password if changing to a new one). 6. The system commits the changes to the database. 7. The system displays a success confirmation message.
Alternative Scenarios	• Incorrect old password: The system rejects the password change request.
Exception	Session expiration during edit; user is redirected to login.

4.4 Sprint 1 Conception

This section presents the structural and dynamic design models for the use cases implemented in Sprint 1.

4.4.1 Class Diagram – Sprint 1 Core Domain

The class diagram below models the entity structures, attributes, and relationships supporting the survey portal, responses, and user profile management:

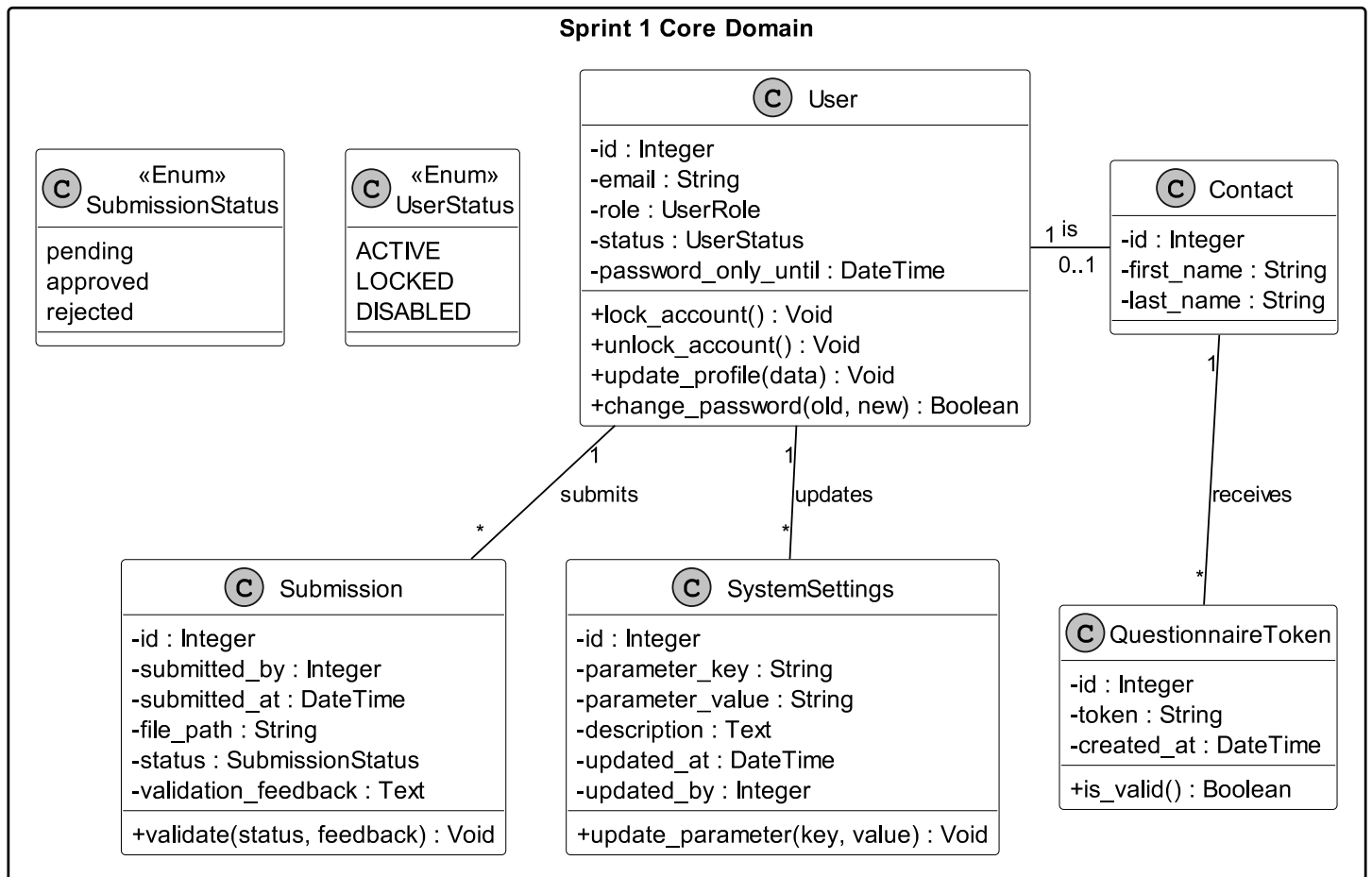


Figure 42: Sprint 1 Class Diagram

The **Contact** class connects to **User** via the **is** association, establishing the link between a company contact profile and their corresponding login account in the system. The Sprint 1 class diagram (Figure 42) incorporates this association to support direct contact authentication, profile management, and dedicated survey dashboard access.

4.4.2 Sequence Diagram – Use Case “Consult assigned surveys”

The diagram below illustrates the dynamic interactions for the “Consult assigned surveys” use case:

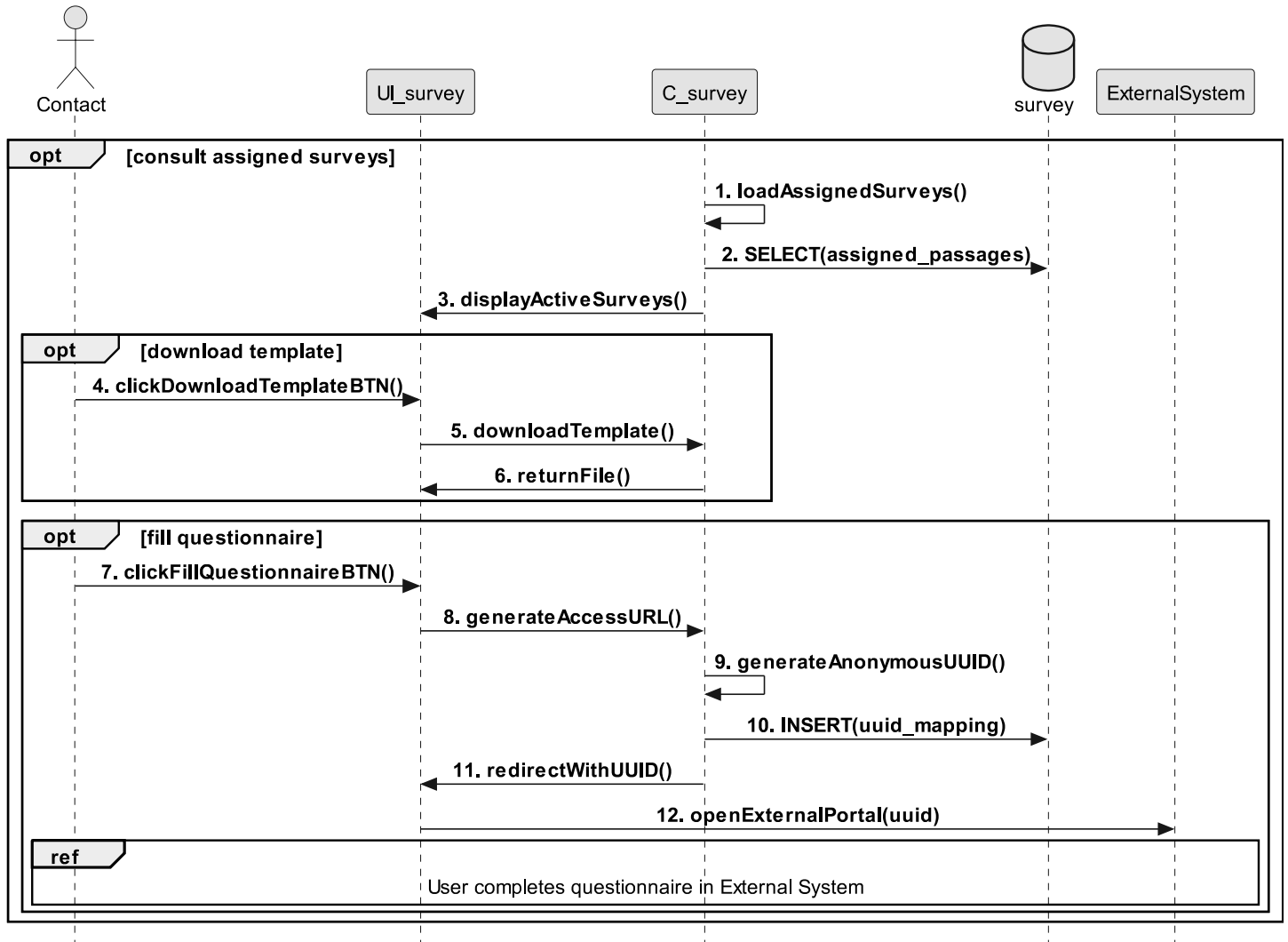


Figure 43: Sequence Diagram – Consult assigned surveys

4.4.3 Sequence Diagram – Use Case “Consult assigned results”

The diagram below illustrates the dynamic interactions for the “Consult assigned results” use case:

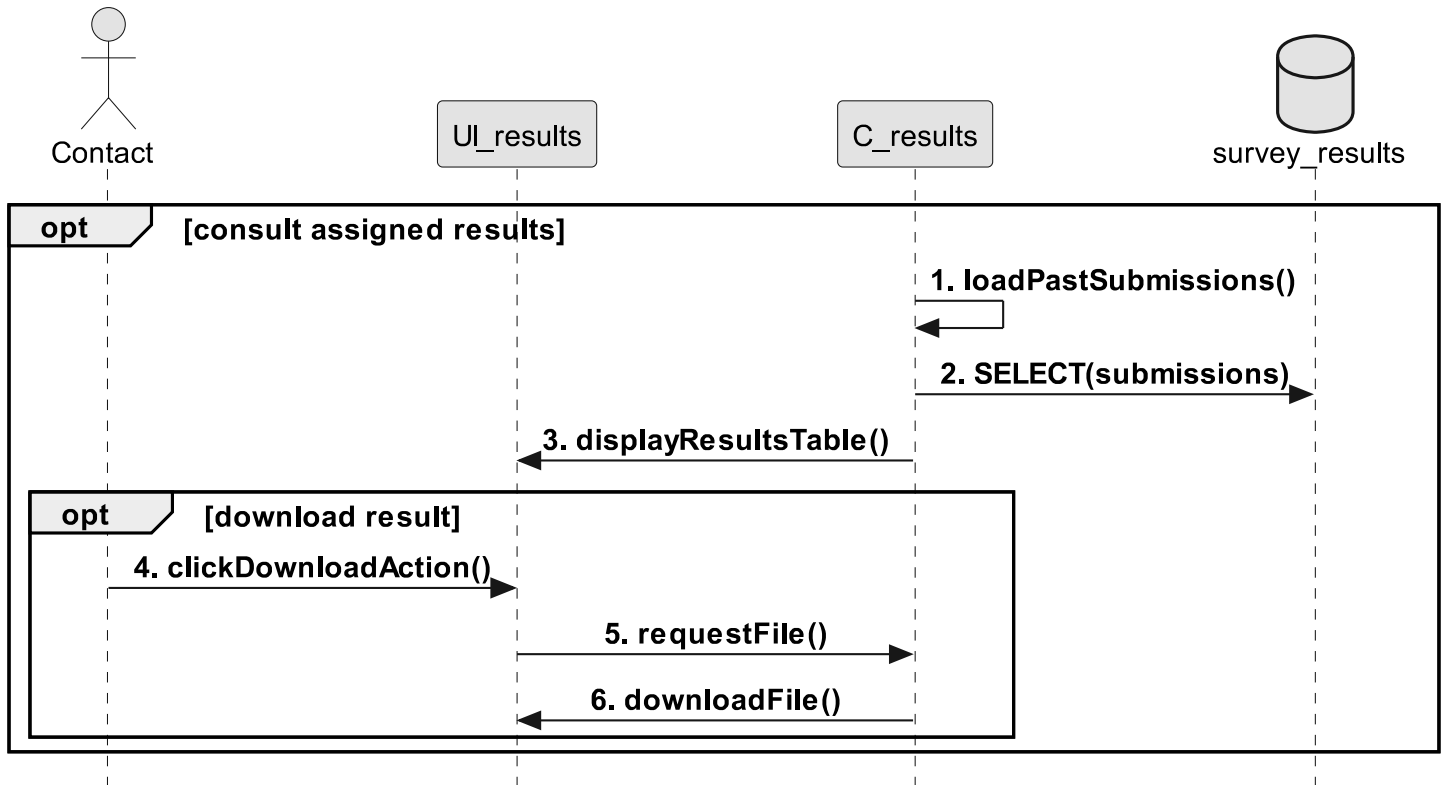


Figure 44: Sequence Diagram – Consult assigned results

4.4.4 Sequence Diagram – Use Case “Manage system settings”

The diagram below illustrates the dynamic interactions for the “Manage system settings” use case:

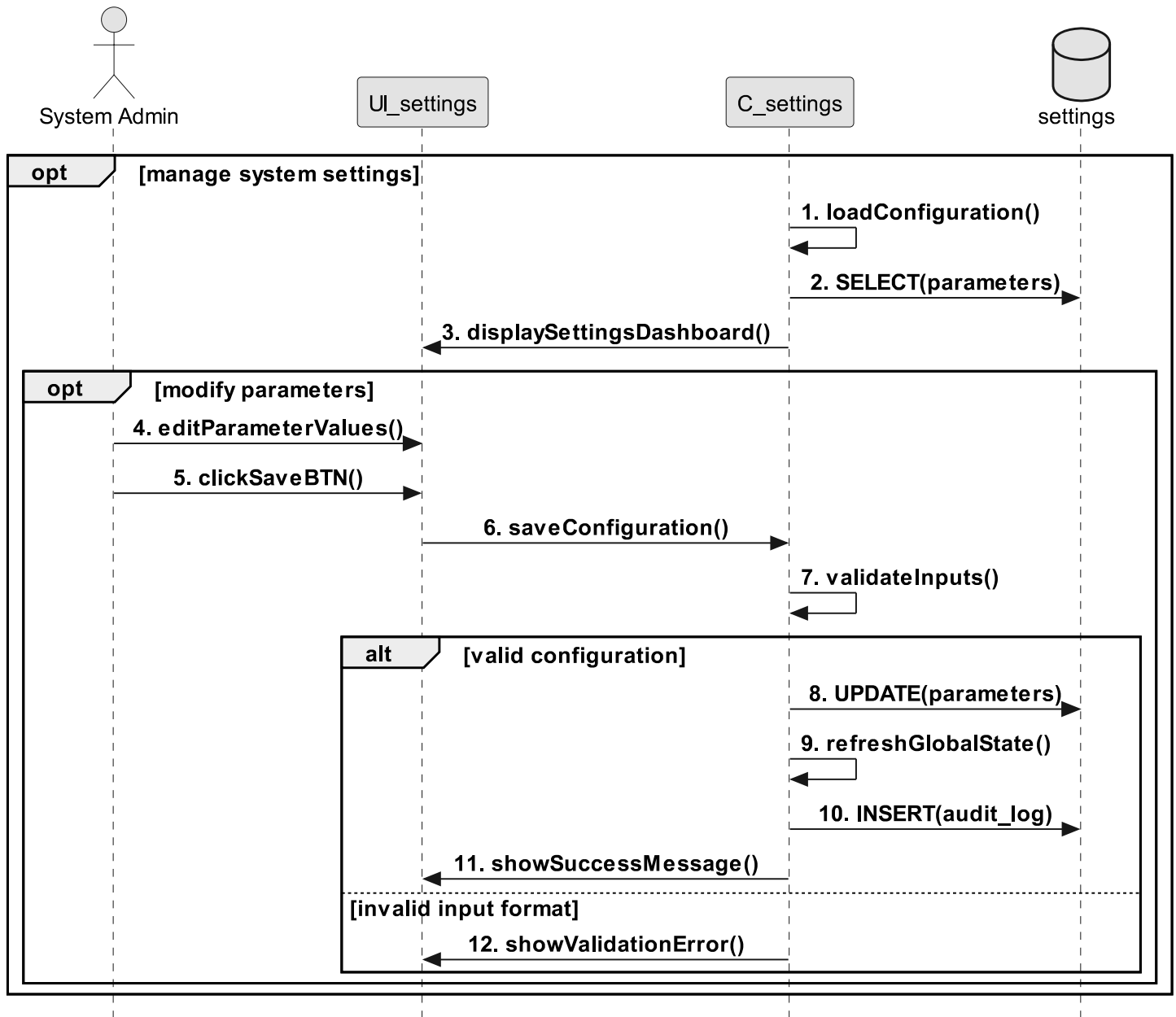


Figure 45: Sequence Diagram – Manage system settings

4.4.5 Sequence Diagram – Use Case “Manage locked accounts”

The diagram below illustrates the dynamic interactions for the “Manage locked accounts” use case:

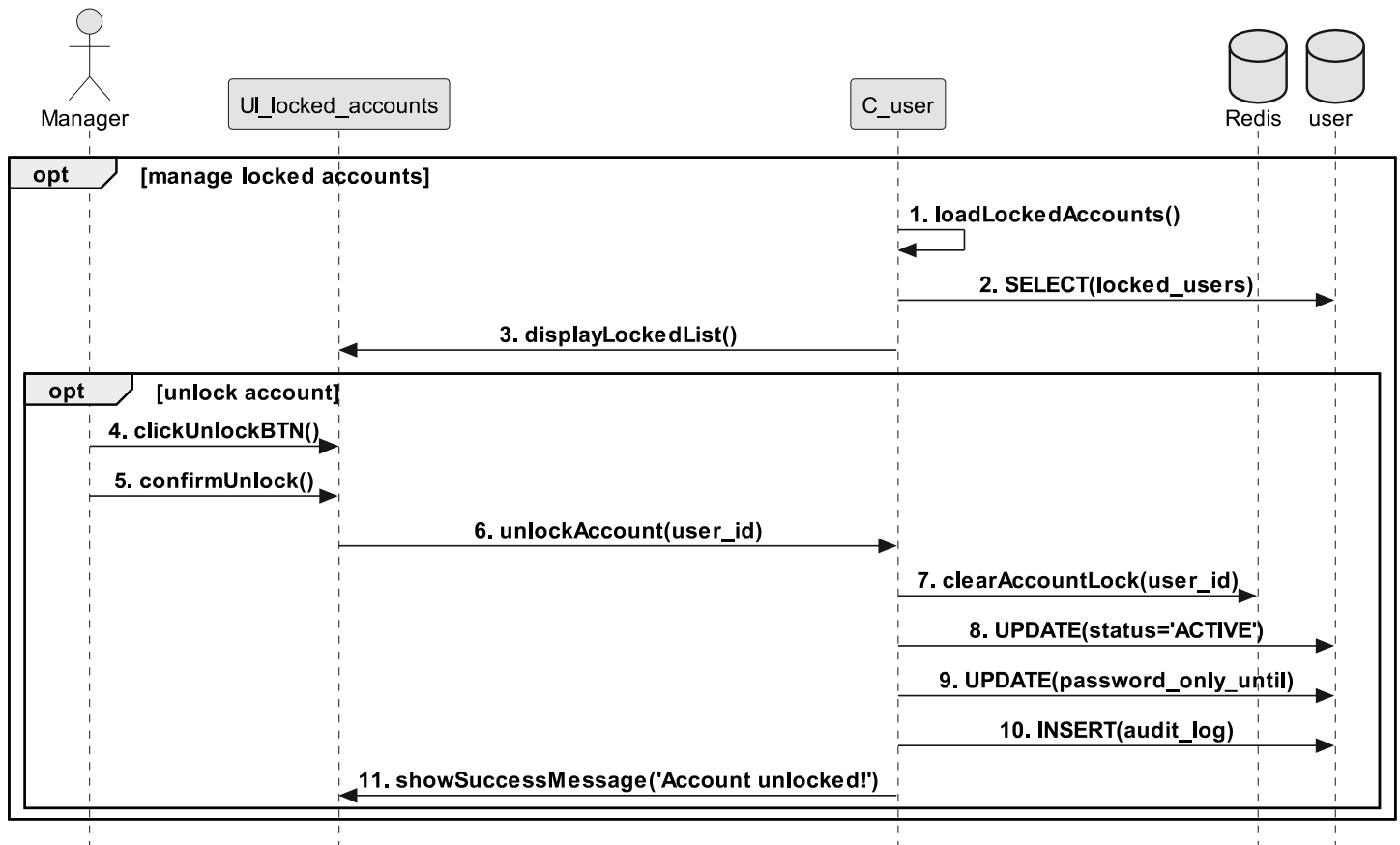


Figure 46: Sequence Diagram – Manage locked accounts

4.4.6 State Diagram – Use Case “Manage locked accounts”

The state diagram below illustrates the lifecycle and state transitions of an account during lockout and unlocking:

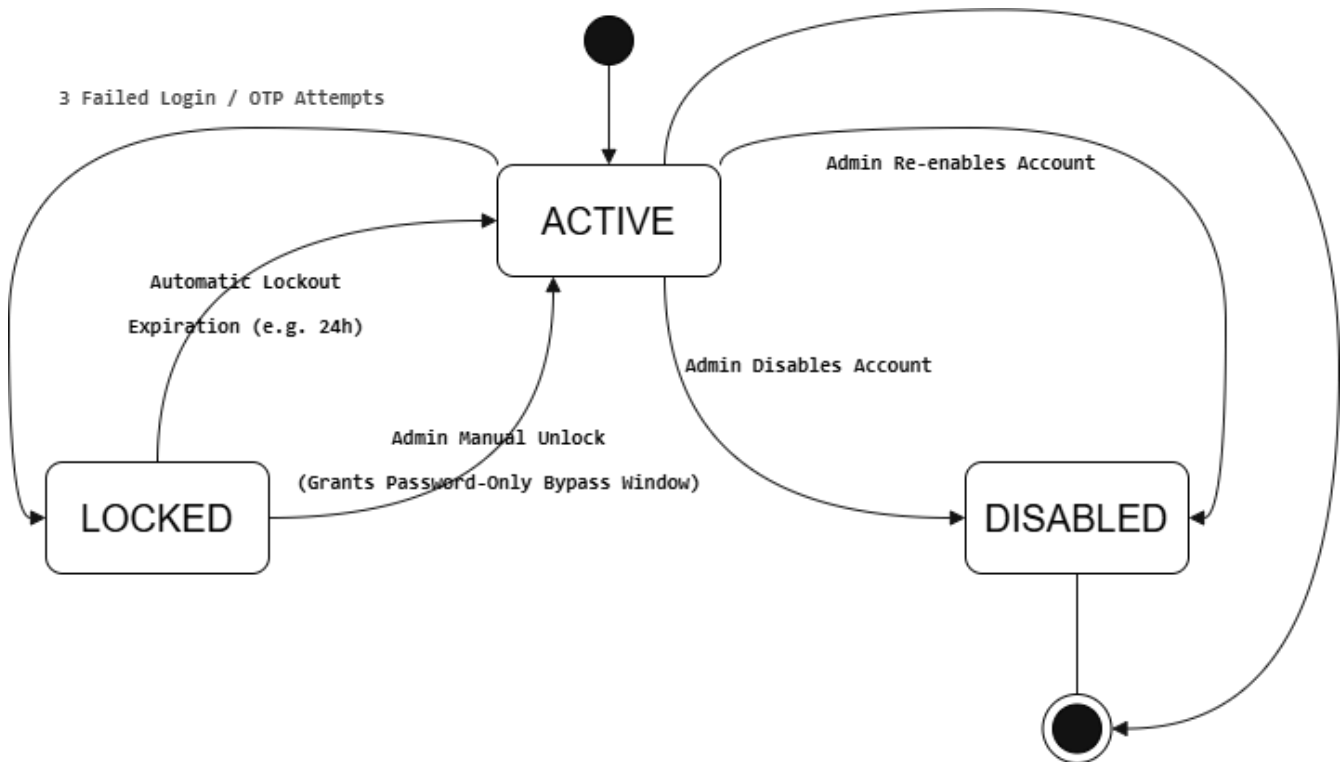


Figure 47: State Diagram – Manage locked accounts

4.4.7 Sequence Diagram – Use Case “Manage profile”

The diagram below illustrates the dynamic interactions for the “Manage profile” use case:

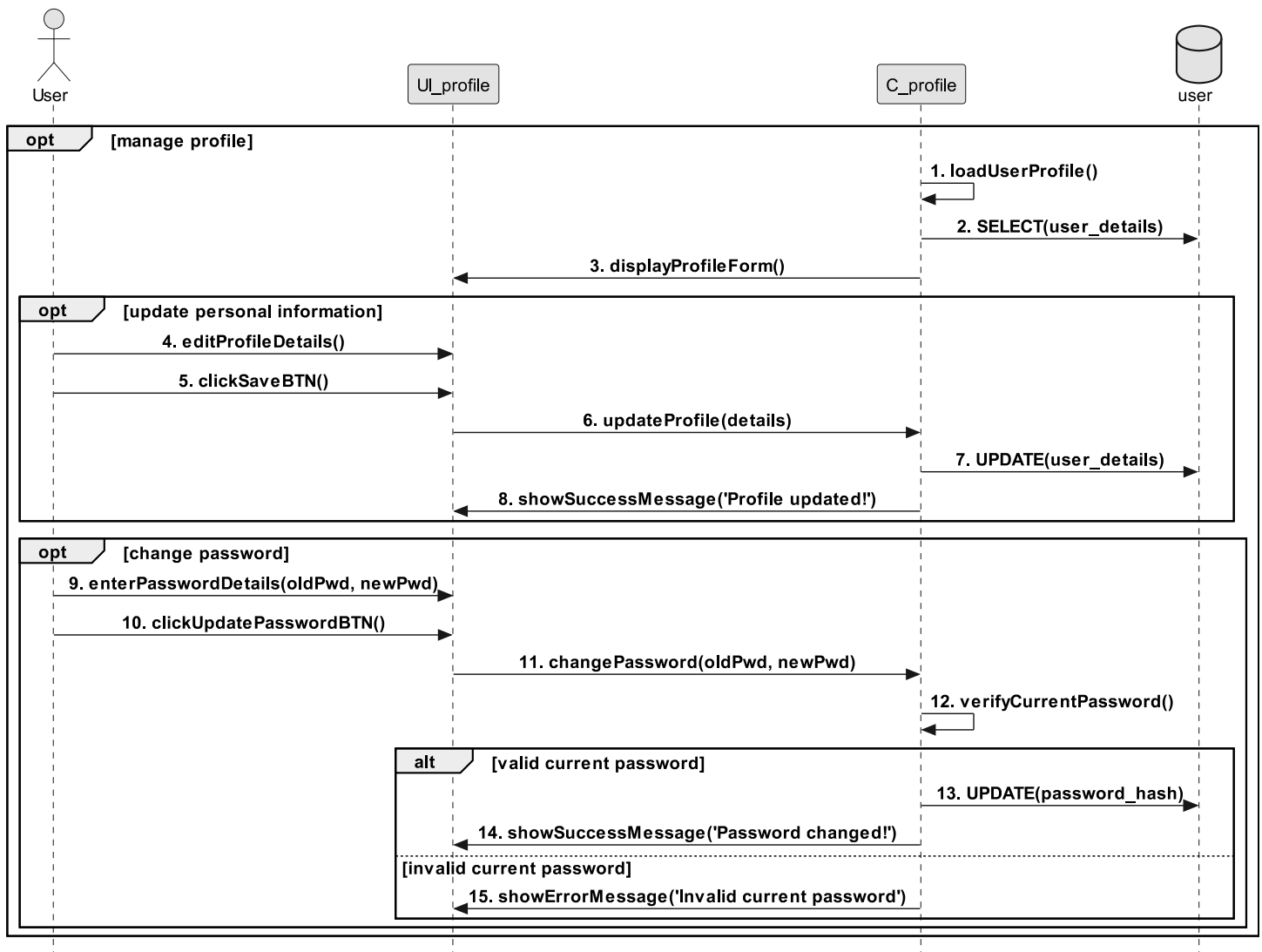


Figure 48: Sequence Diagram – Manage profile

4.5 Sprint 1 Implementation

This section presents the user interface implementations developed during Sprint 1.

4.5.1 Company Contact Portal & Surveys

Figure 49 displays the “My Surveys” interface where company contacts can consult active survey assignments, download template reference documents, and navigate to the online questionnaire.

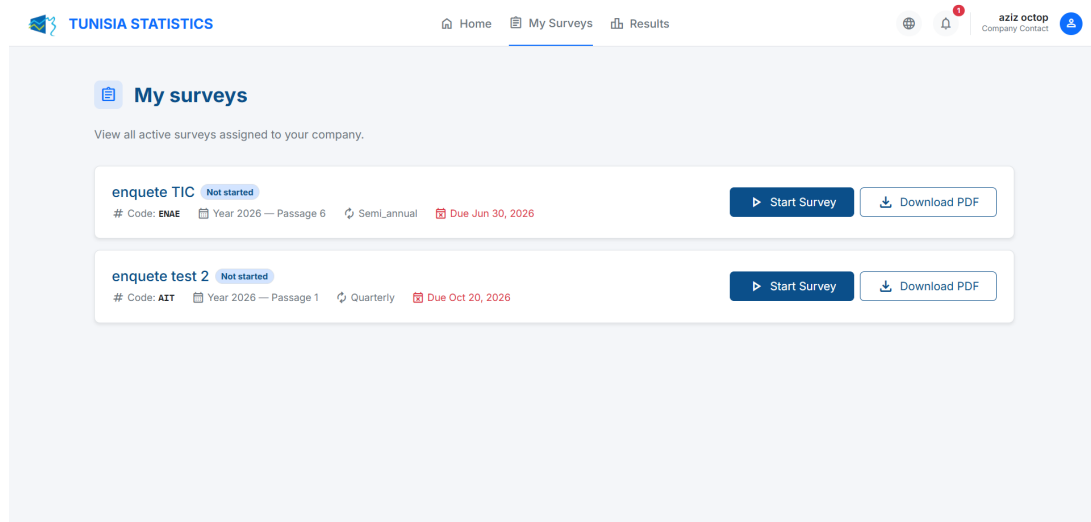


Figure 49: My Surveys Interface

Figure 50 shows the company contact portal overview dashboard presenting survey progress summary cards and quick shortcuts.

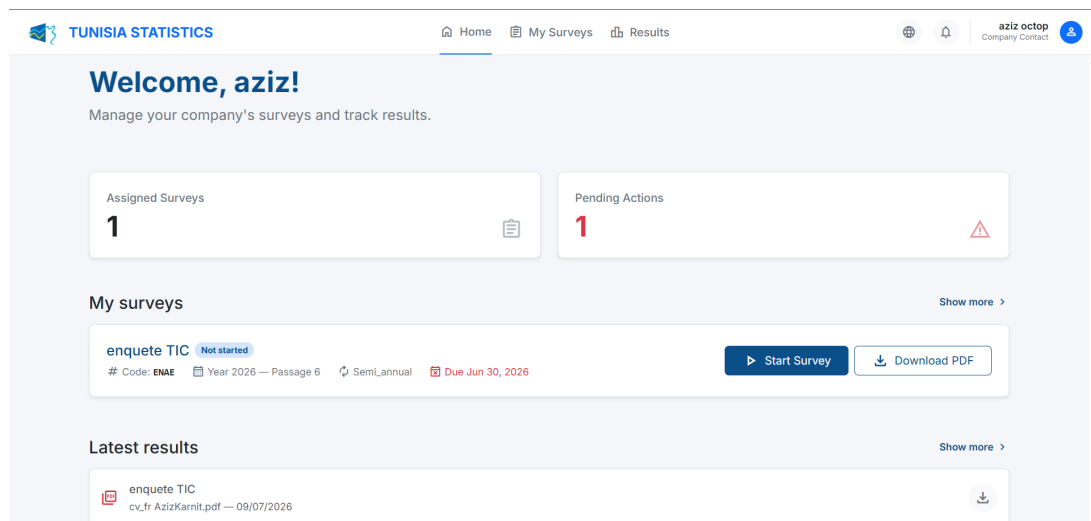


Figure 50: Company Contact Overview Dashboard

Figure 51 shows the automated email notification received by company contacts upon being assigned to a survey passage.

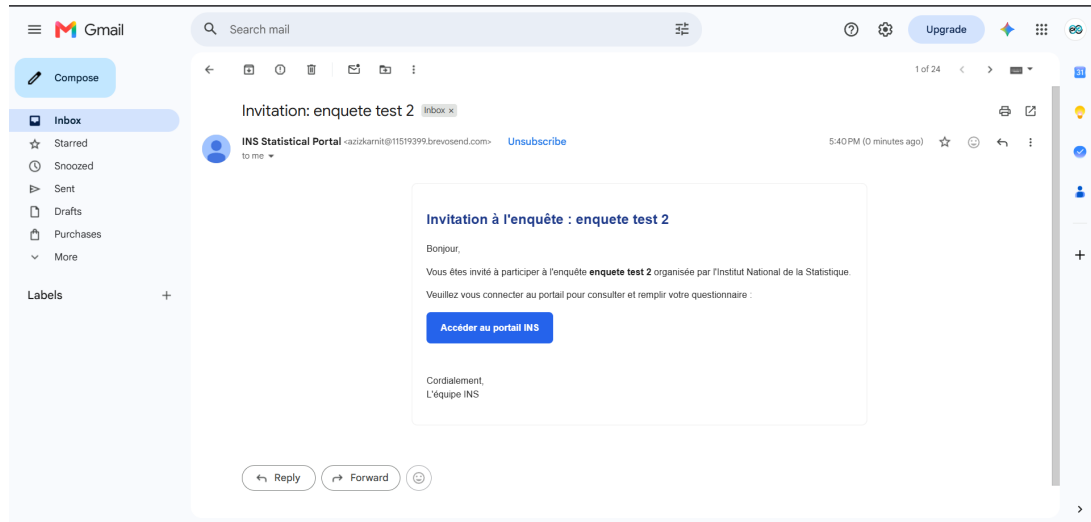


Figure 51: Survey Assignment Email Notification

4.5.2 Consult Assigned Results Interface

Figure 52 presents the survey reports and results interface allowing contacts to view past survey submissions and download published reports.

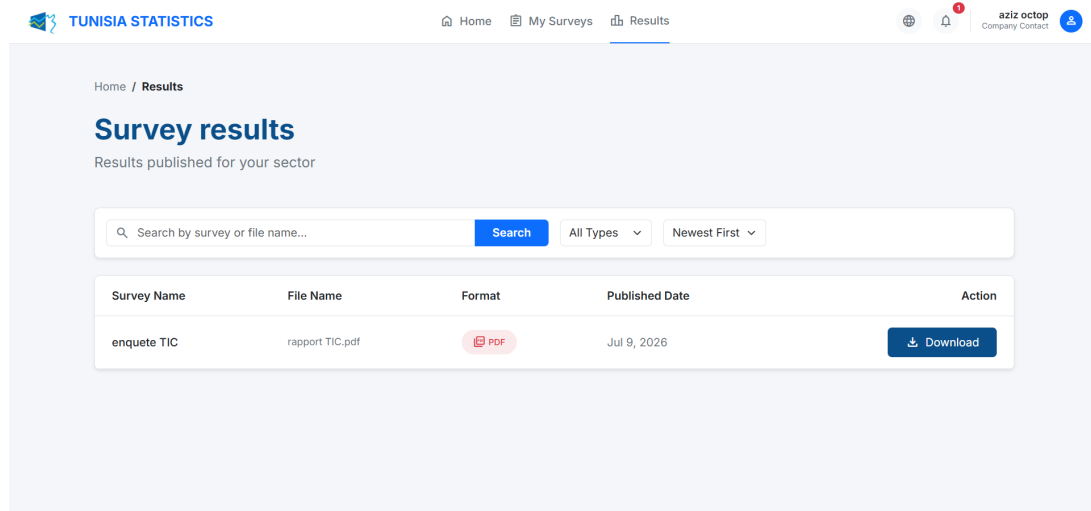


Figure 52: Consult Assigned Results & Reports Interface

4.5.3 Manage System Settings Interface

Figure 53 shows the administrative system settings page for configuring global security parameters (2FA expiration, lockout duration, max attempts), default language, and email service providers.

TUNISIA STATISTICS

Home > System Settings

System Settings

Configure global platform parameters and security protocols.

Security

Code expiration (minutes)

5

Duration in minutes for 2FA codes.

Maximum sign-in attempts

3

Number of failed attempts before account lock.

Account lock duration (hours)

24

Time before a locked account can try again.

Password-only access duration (hours)

1

Time window for password-only login before re-authentication.

Language

Default language

French

Initial language for new users.

Email

Email provider

Brevo

SMTP/API provider for platform communications.

My Profile | Sign out

Figure 53: System Settings Configuration Interface

4.5.4 Manage Locked Accounts & Security Monitoring

Figure 54 displays the security monitoring table listing user accounts suspended due to repeated failed authentication attempts, allowing administrators to restore access.

TUNISIA STATISTICS

Home > Locked Accounts

Locked Accounts

Review and unlock user accounts that have been temporarily suspended.

Search

Search by name or email... Search

Showing 1 locked accounts

Clear all filters

User	Email Address	Locked Since	Unlocks At	Actions
Joseph White ID: #120	azizy9460@gmail.com	Jul 21, 05:50 PM	Jul 22, 05:50 PM	

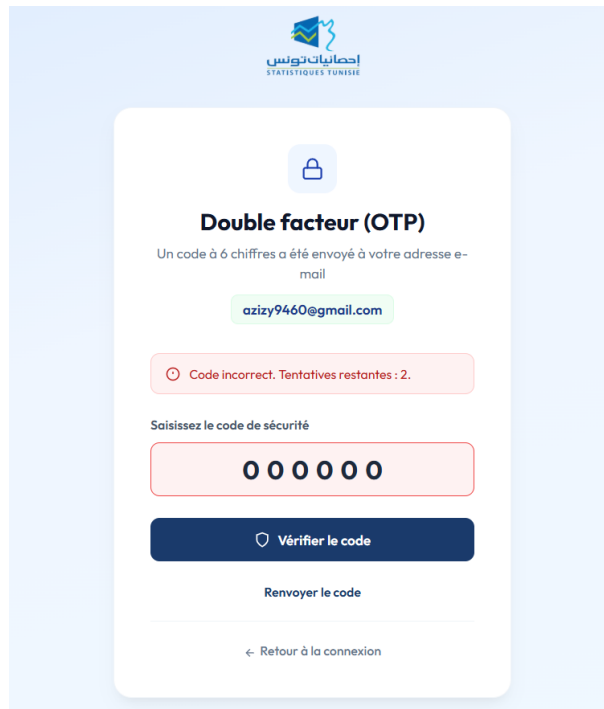
Showing 1 to 1 of 1 results

< Previous Page 1 of 1 Next >

My Profile | Sign out

Figure 54: Locked Accounts Management Interface

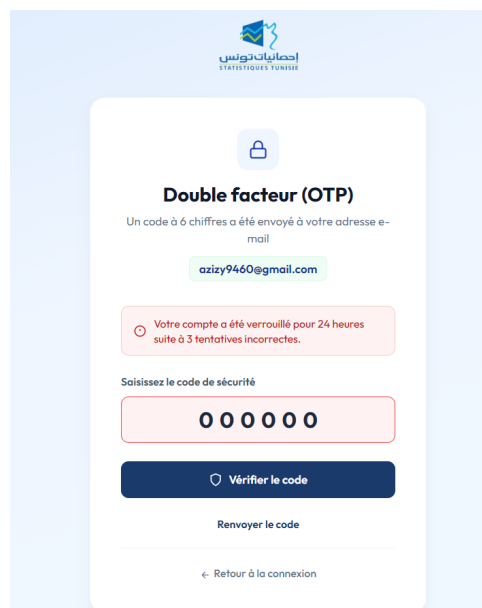
Figure 55 shows the inline error feedback during two-factor authentication when an incorrect OTP is supplied.



The screenshot displays a login interface for 'STATISTIQUES TUNISIE' (Tunisian Statistics). The title is 'Double facteur (OTP)'. Below the title, it states 'Un code à 6 chiffres a été envoyé à votre adresse e-mail' (A 6-digit code has been sent to your email address) and shows the email 'azizy9460@gmail.com'. A red error message box indicates 'Code incorrect. Tentatives restantes : 2.' (Incorrect code. Remaining attempts: 2). Below this, there is a prompt 'Saisissez le code de sécurité' (Enter the security code) and a 6-digit input field containing '0 0 0 0 0 0'. A dark blue button labeled 'Vérifier le code' (Verify the code) is present, along with a link 'Renvoyer le code' (Resend the code). At the bottom, there is a link 'Retour à la connexion' (Return to login).

Figure 55: Incorrect OTP Validation Error

Figure 56 illustrates the security notification banner informing the user when their account is locked due to security policy enforcement.

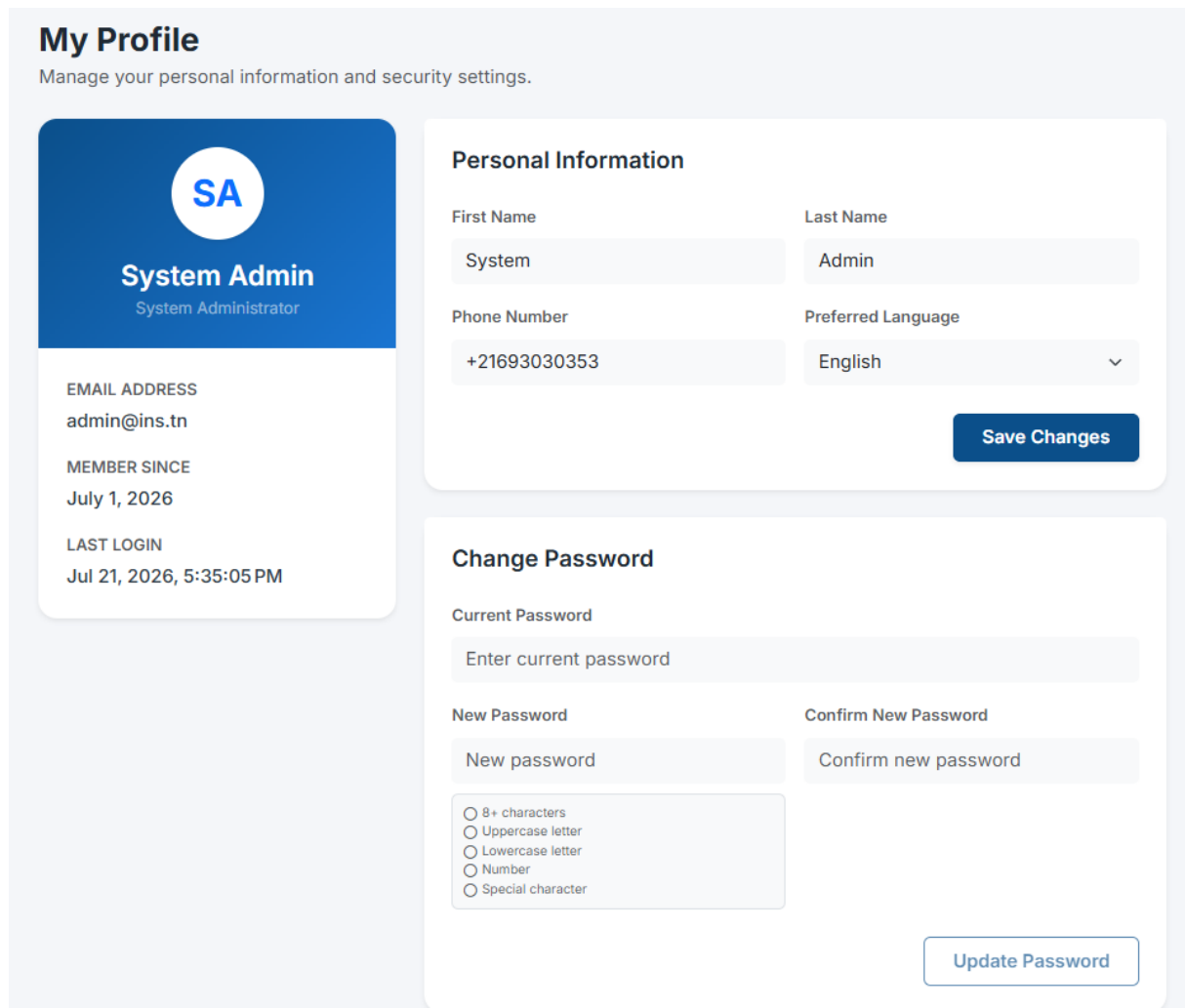


The screenshot displays the same login interface as Figure 55, but with a different error message. The red error message box now states 'Votre compte a été verrouillé pour 24 heures suite à 3 tentatives incorrectes.' (Your account has been locked for 24 hours due to 3 incorrect attempts). The rest of the interface, including the email 'azizy9460@gmail.com', the 6-digit input field with '0 0 0 0 0 0', the 'Vérifier le code' button, the 'Renvoyer le code' link, and the 'Retour à la connexion' link, remains the same.

Figure 56: Account Lockout Security Warning

4.5.5 Manage Profile Interface

Figure 57 shows the user profile management interface for administrators, enabling personal information updates and password changes.



The interface is titled "My Profile" with the subtitle "Manage your personal information and security settings." It is divided into two main sections: "Personal Information" and "Change Password".

My Profile
Manage your personal information and security settings.

Personal Information

First Name: System
Last Name: Admin
Phone Number: +21693030353
Preferred Language: English (dropdown menu)
Save Changes

Change Password

Current Password: Enter current password
New Password: New password
Confirm New Password: Confirm new password
☐ 8+ characters
☐ Uppercase letter
☐ Lowercase letter
☐ Number
☐ Special character
Update Password

System Admin
System Administrator

EMAIL ADDRESS
admin@ins.tn

MEMBER SINCE
July 1, 2026

LAST LOGIN
Jul 21, 2026, 5:35:05 PM

Figure 57: Admin Profile Management Interface

Figure 58 illustrates the profile interface for company contacts.

Home / My profile

My profile

Manage your personal information and account settings.

[Save changes](#)

Personal information

First name

Last name

Position

Your official title within the organization.

aziz

octop

IT

Company

Company name

Identifier

Sector

Beta Agri Co

ENT-0002

Agriculture

Account

Email address

Preferred language

Contact support to change your login email.

octoshoptn@gmail.com

English

Security

Ensure your account stays secure by regularly updating your password.

[Change password](#)

Figure 58: Contact Profile Management Interface

4.6 UUID Token Architecture & Security Analysis

A key technical innovation implemented in Sprint 1 is the passwordless UUID token access system for external company respondents. Forcing external enterprise representatives to maintain traditional credentials creates account management friction and reduces statistical response rates.

To solve this, the platform generates a Universally Unique Identifier (UUID v4) for each (contact_id, passage_id) assignment pair. The token is mapped in the questionnaire_tokens database table with a strict UniqueConstraint("contact_id", "passage_id"), ensuring one active token per respondent per survey passage.

When a company contact clicks their survey invitation link, the request is directed to the external questionnaire application with the UUID embedded as a parameter (?token=uuid). The external application executes a server-to-server REST call to the PFA backend endpoint POST /api/v1/ext/resolve, protected by a shared API key (X-API-Key). The endpoint validates the token, enforces passage closing date restrictions, initializes collection

tracking, and returns questionnaire data without exposing company PII.

Listing 4.1 shows the database model, and Listing 4.2 shows the token resolution endpoint.

```
1 class QuestionnaireToken(Base):
2     __tablename__ = "questionnaire_tokens"
3     __table_args__ = (
4         UniqueConstraint(
5             "contact_id", "passage_id",
6             name="uq_contact_passage_token"
7         ),
8     )
9     id: Mapped[int] = mapped_column(
10         Integer, primary_key=True, index=True
11     )
12     token: Mapped[str] = mapped_column(
13         String(50), unique=True, index=True
14     )
15     contact_id: Mapped[int] = mapped_column(
16         Integer, ForeignKey("contacts.id")
17     )
18     company_id: Mapped[int] = mapped_column(
19         Integer, ForeignKey("companies.id")
20     )
21     passage_id: Mapped[int] = mapped_column(
22         Integer, ForeignKey("survey_passages.id")
23     )
```

Listing 4.1: Questionnaire Token Database Model
(app/models/questionnaire_token.py)

```
1 @router.post("/resolve", response_model=ResolveTokenResponse)
2 def resolve_uuid_token(
3     payload: ResolveTokenRequest,
4     db: Session = Depends(get_db),
5     api_key: str = Depends(verify_api_key)
6 ):
7     token_obj = db.query(QuestionnaireToken).filter(
8         QuestionnaireToken.token == payload.uuid
9     ).first()
10    if not token_obj:
11        raise HTTPException(
12            status_code=404,
13            detail="UUID token not found or invalid"
14        )
15    passage = token_obj.passage
16    if passage.closing_date < datetime.datetime.now():
17        raise HTTPException(
```

```

18         status_code=400,
19         detail="The survey passage has closed."
20     )
21     return ResolveTokenResponse(...)

```

Listing 4.2: External Token Resolution Endpoint
(app/api/v1/external_integration.py)

4.7 Sprint 1 Conclusion

This chapter detailed the development of Sprint 1, which successfully delivered the company portal capabilities allowing external contacts to consult surveys, download templates, and upload their submissions. Additionally, secondary administrative tools such as system settings configuration, locked account management, and the passwordless UUID token integration were completed. The next chapter details Sprint 2, which focuses on the notification engine, systemic auditing, and overall system testing.

Chapter 5

Sprint 2 – Notification Engine & Systemic Auditing

5.1 Introduction

This section details the development of Sprint 2, which focuses on the notification engine, comprehensive systemic auditing, and platform validation. It presents the sprint backlog, the detailed refinement of each user story, class diagram updates, dynamic behavior models, user interface implementations, and system testing.

5.2 Sprint 2 Backlog

The Sprint 2 backlog encompasses the functional requirements for user notification management and security audit logging.

Table 24: Sprint 2 Backlog

ID	User Story	Priority
US11	As a user, I want to Consult notifications to view and manage system alerts and activity updates.	High
US12	As a System Administrator or Survey Manager, I want to Consult audit logs to monitor system security and track user actions.	High

5.3 Sprint 2 Refinement

In this section, we refine the functional details and textual specifications for the use cases of Sprint 2.

5.3.1 Refinement of the Use Case “Consult notifications”

The diagram below illustrates the refinement of the “Consult notifications” use case and its interactions with system actors:

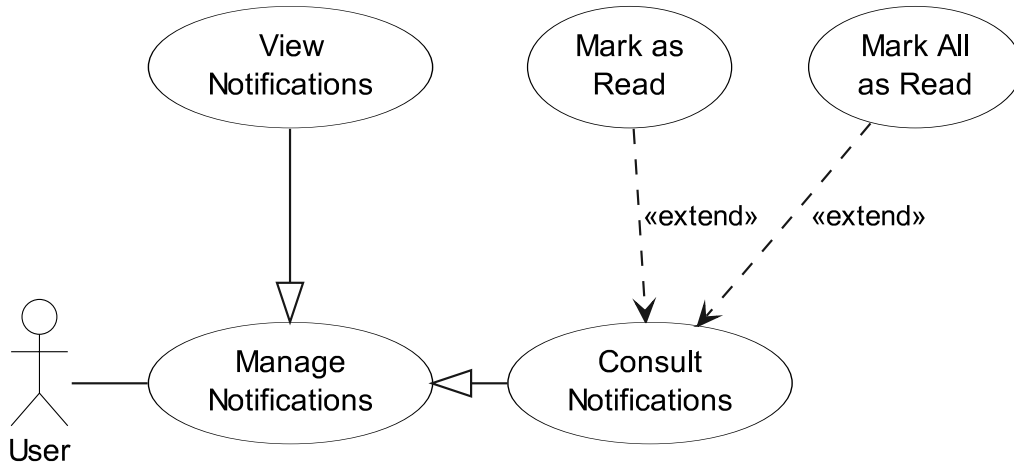


Figure 59: Use Case Diagram – Consult notifications

Table 25 presents the textual description of the “Consult notifications” use case:

Table 25: Textual description – Consult notifications

Use Case	Consult notifications
Actor	Any authenticated user
Pre-condition	The user is logged in to the platform.
Post-condition	The user views their notifications and can mark items as read.
Main Scenario	1. The user clicks on the notification bell icon in the top header. 2. The system fetches and displays a dropdown list of recent unread notifications. 3. The user selects “View All Notifications” to open the full notification center. 4. The system lists all notifications categorized by date and severity. 5. The user clicks “Mark as Read” on an item or “Mark All as Read”. 6. The system updates the read status in the database and updates the header badge counter.
Alternative Scenarios	• No notifications: The system displays a “No new notifications” placeholder.
Exception	Real-time connection drop; the system retries fetching notifications on the next polling interval.

5.3.2 Refinement of the Use Case “Consult audit logs”

The diagram below illustrates the refinement of the “Consult audit logs” use case and its interactions with system actors:

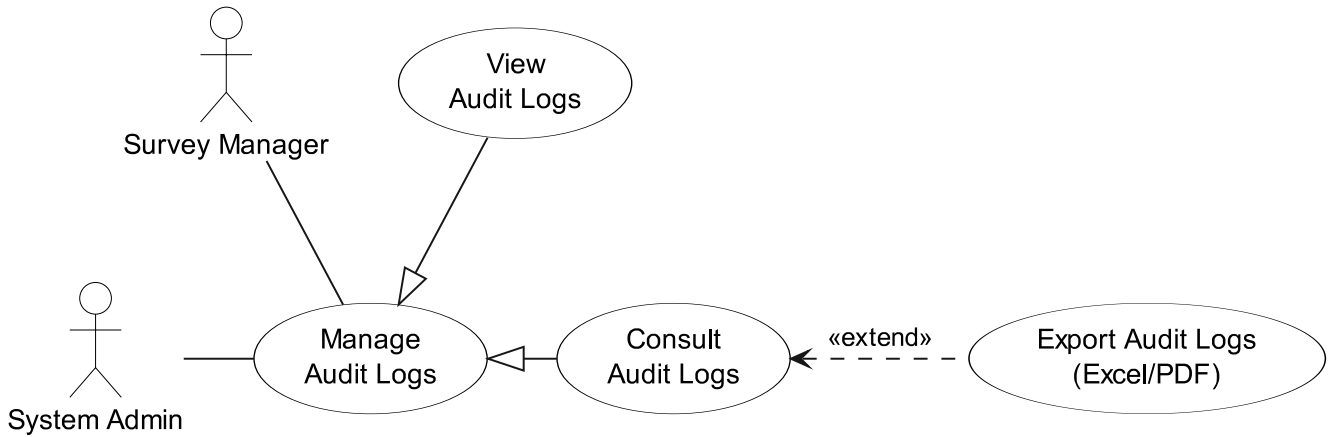


Figure 60: Use Case Diagram – Consult audit logs

Table 26 presents the textual description of the “Consult audit logs” use case:

Table 26: Textual description – Consult audit logs

Use Case	Consult audit logs
Actor	System Administrator, Survey Manager
Pre-condition	The user is logged in with administrative or managerial permissions.
Post-condition	Audit records are queried, filtered, and optionally exported.
Main Scenario	1. The administrator navigates to the “Audit Logs” section. 2. The system loads the audit trail table displaying timestamp, actor, action type, target entity, and IP address. 3. The administrator applies filters (e.g., date range, user, action type). 4. The system updates the table view with matching audit events. 5. (Optional) The administrator clicks “Export Audit Log”. 6. The system triggers a background task to generate an Excel/PDF export report.
Alternative Scenarios	• No matching logs: The system displays a “No audit events found” message.
Exception	Query timeout; the system displays an error and prompts for narrower date filtering.

5.4 Sprint 2 Conception

In this section, we present the structural and dynamic models for Sprint 2.

5.4.1 Class Diagram – Sprint 2 Core Domain

The class diagram below models the complete system architecture including the notification and audit entities:

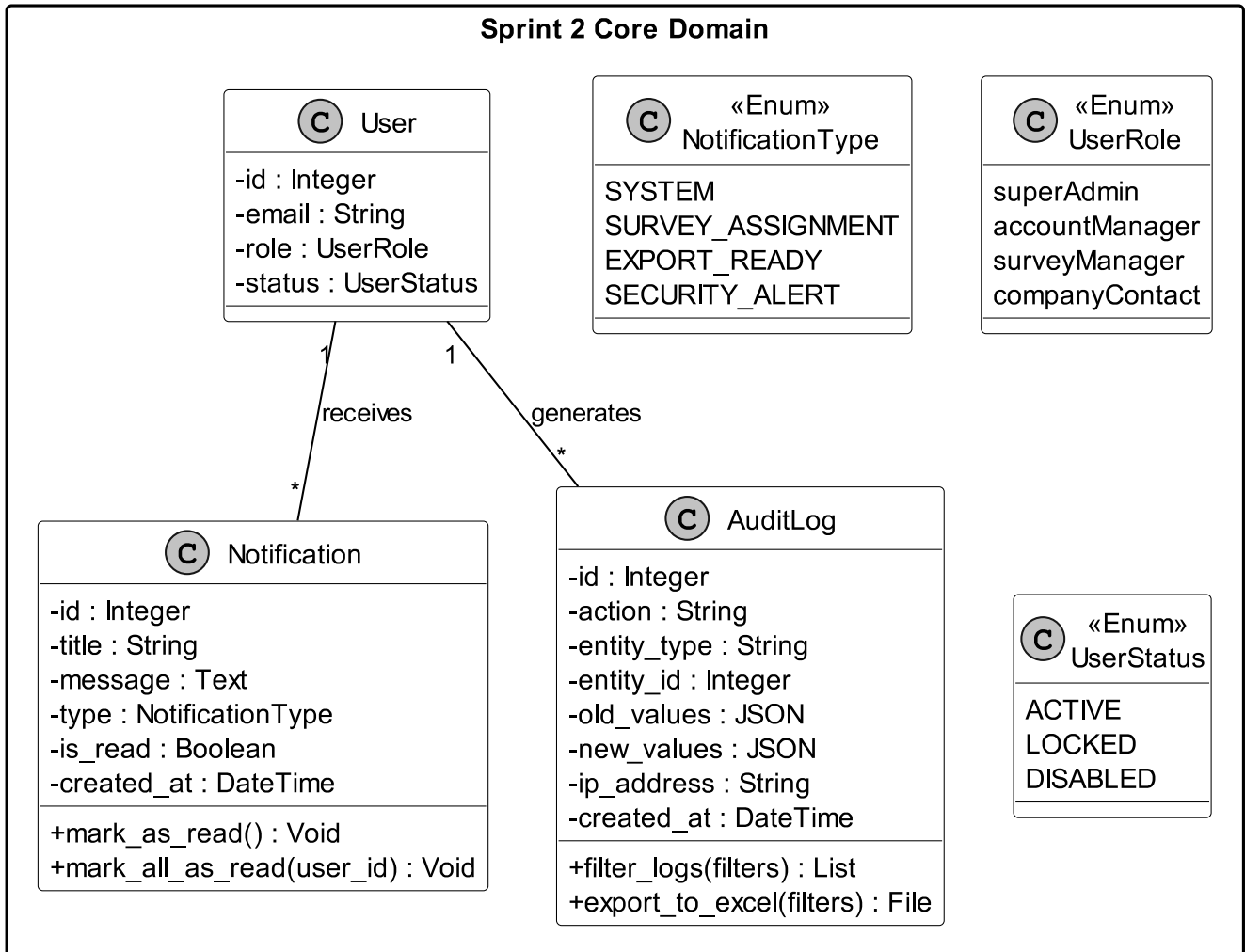


Figure 61: Sprint 2 Class Diagram

The Sprint 2 class diagram (Figure 61) completes the platform data architecture by adding systemic monitoring and notification entities. The `AppNotification` class stores persistent user alerts linked to `User` via a foreign key, recording notification title, message body, read status, type, and contextual action URL. Systemic security auditing is supported by `AuditLog`, which captures system events with actor user ID, action type (LOGIN, CREATE, UPDATE, DELETE, EXPORT), entity target, timestamp, and client

IP address. Furthermore, `ImportSession` and `CompanyImportStaging` model the staging workflow for bulk company file ingestion.

5.4.2 Activity Diagram – Use Case “Consult notifications”

The activity diagram below illustrates the multi-tier workflow and decision logic across system layers for real-time notification delivery and user interactions:

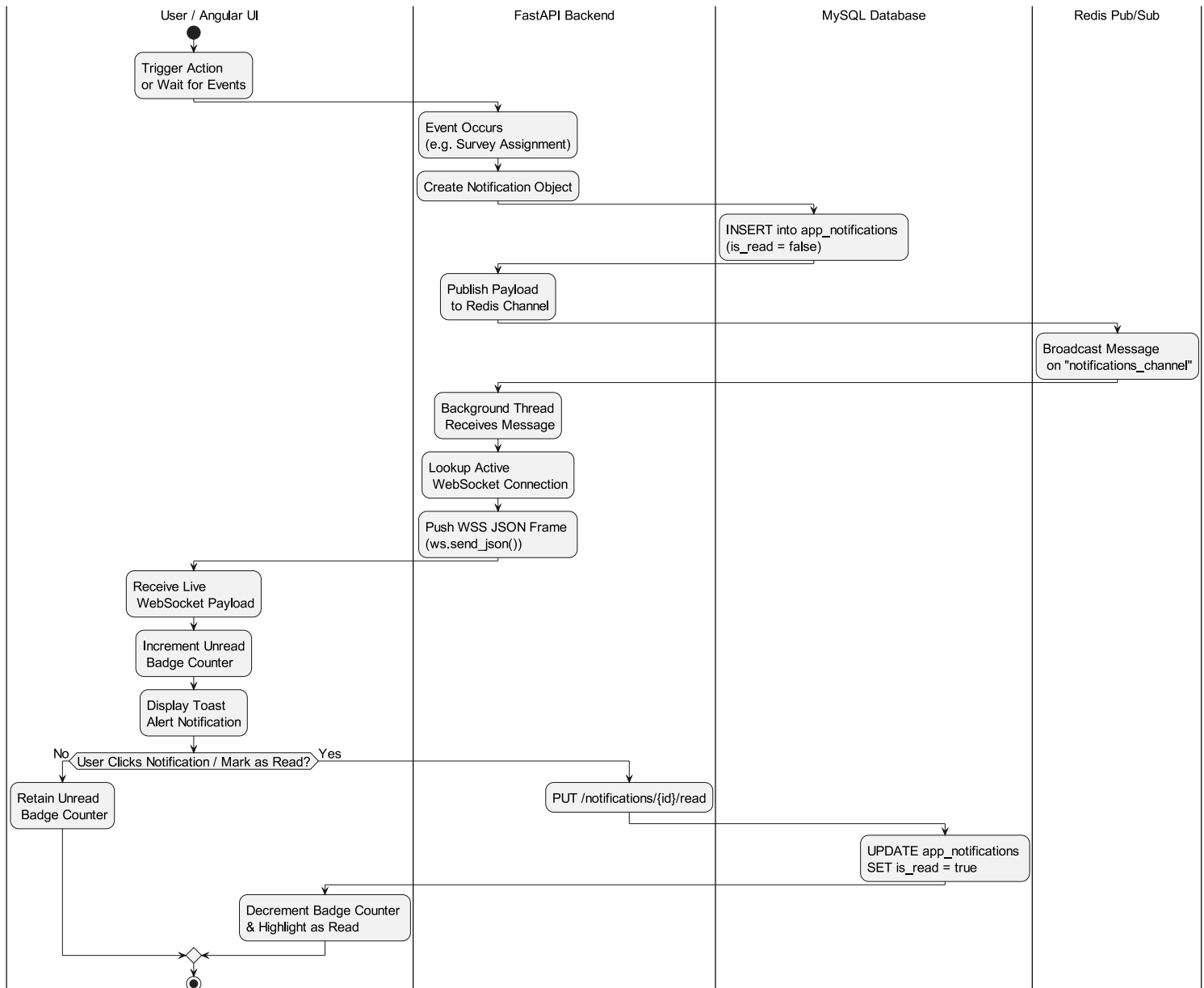


Figure 62: Activity Diagram – Consult notifications

5.4.3 State Diagram – Notification Life Cycle

The state diagram below models the transitions of a notification from creation and real-time delivery to user consultation and archival:

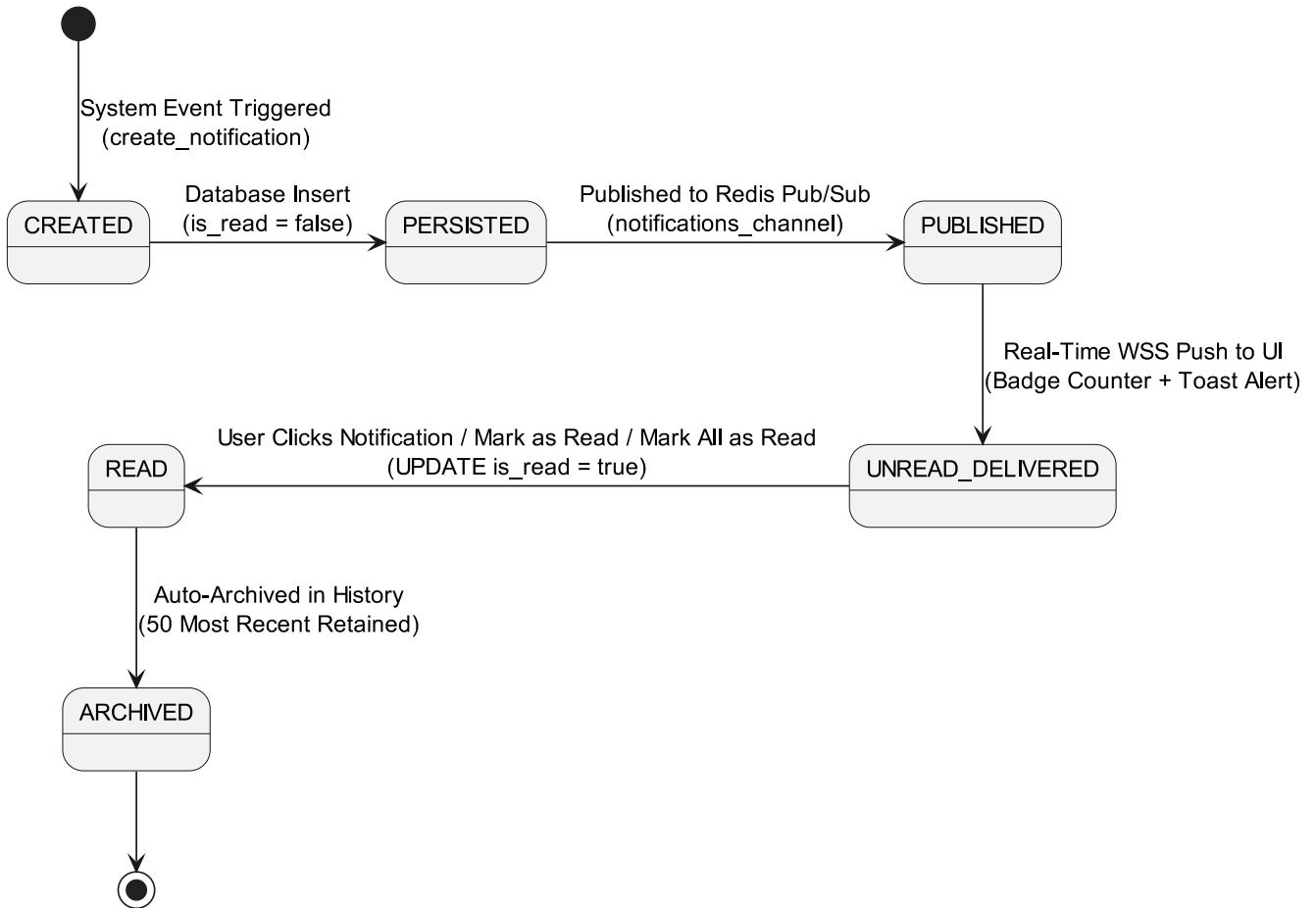


Figure 63: State Diagram – Notification Life Cycle

5.4.4 Sequence Diagram – Use Case “Consult audit logs”

The sequence diagram below details the step-by-step interactions between the System Administrator, Frontend UI, API Controller, Celery Worker, and Database during audit log consultation and asynchronous Excel export:

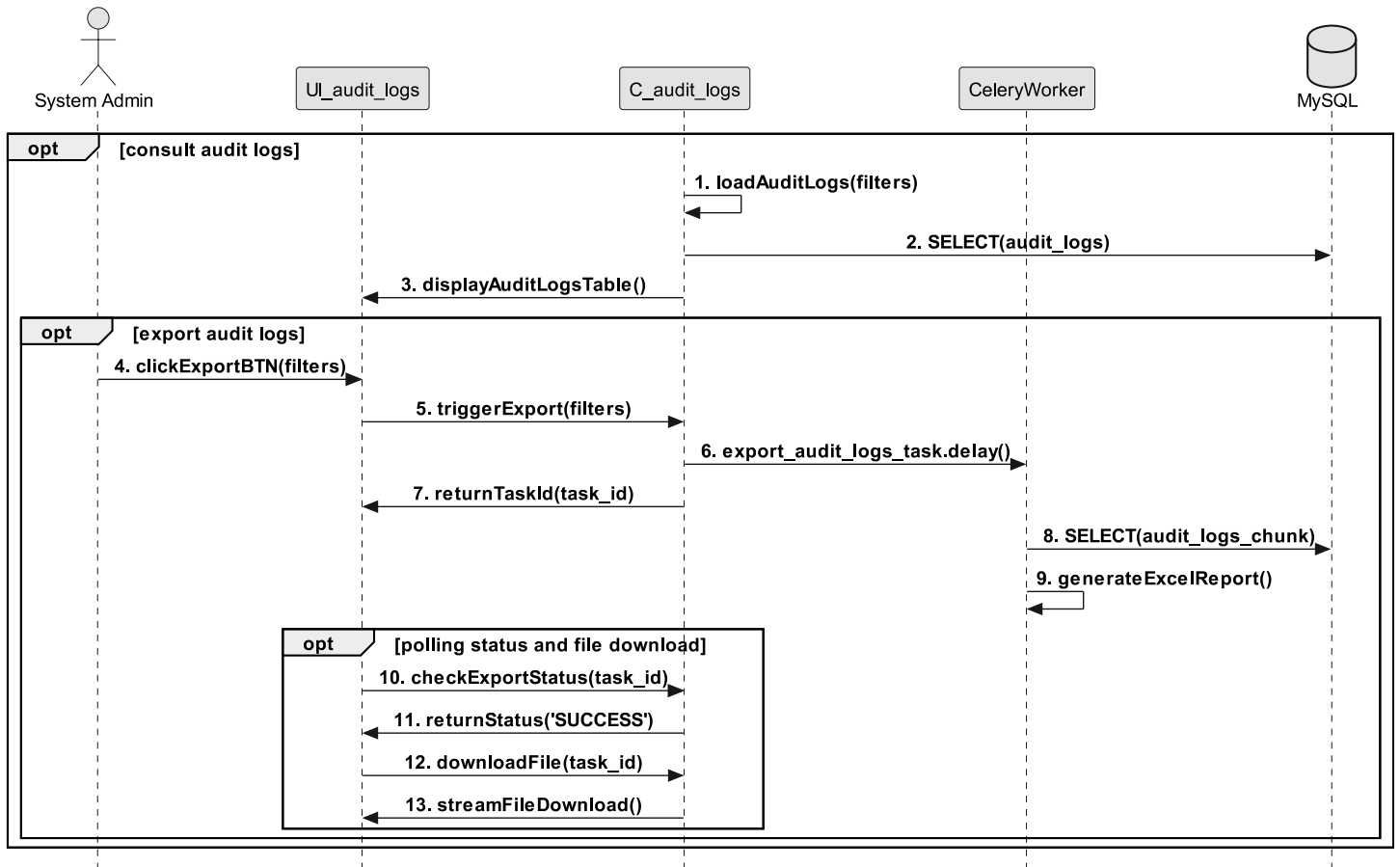


Figure 64: Sequence Diagram – Consult audit logs

5.5 Sprint 2 Implementation

This section presents the user interface implementations developed during Sprint 2.

5.5.1 Consult Notifications Interface

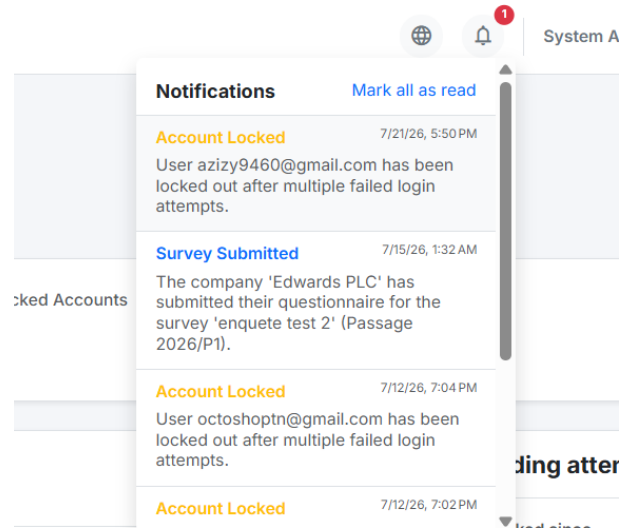


Figure 65: Consult Notifications Interface

Real-time notification delivery is powered by a dedicated WebSocket background thread connected to a Redis Pub/Sub channel (`notifications_channel`). When an event occurs (such as survey submission or batch import completion), the service publishes a JSON payload to Redis. The WebSocket connection manager receives the payload and dispatches it asynchronously to active client connections without polling overhead. Listing 5.1 presents the Redis listener implementation in the ConnectionManager.

```
1 def _listen_to_redis(self):
2     try:
3         redis = get_redis_client()
4         pubsub = redis.pubsub()
5         pubsub.subscribe("notifications_channel")
6         for message in pubsub.listen():
7             if message["type"] == "message":
8                 data = json.loads(message["data"])
9                 uid = data.get("user_id")
10                if uid and uid in self.active_connections:
11                    asyncio.run_coroutine_threadsafe(
12                        self.send_personal_message(data, uid),
13                        self.loop
14                    )
15    except Exception as e:
```

```
logger.error(f"Redis PubSub listener error: {e}")
```

Listing 5.1: Redis Pub/Sub WebSocket Manager (app/core/websockets.py)

5.5.2 Consult Audit Logs Interface

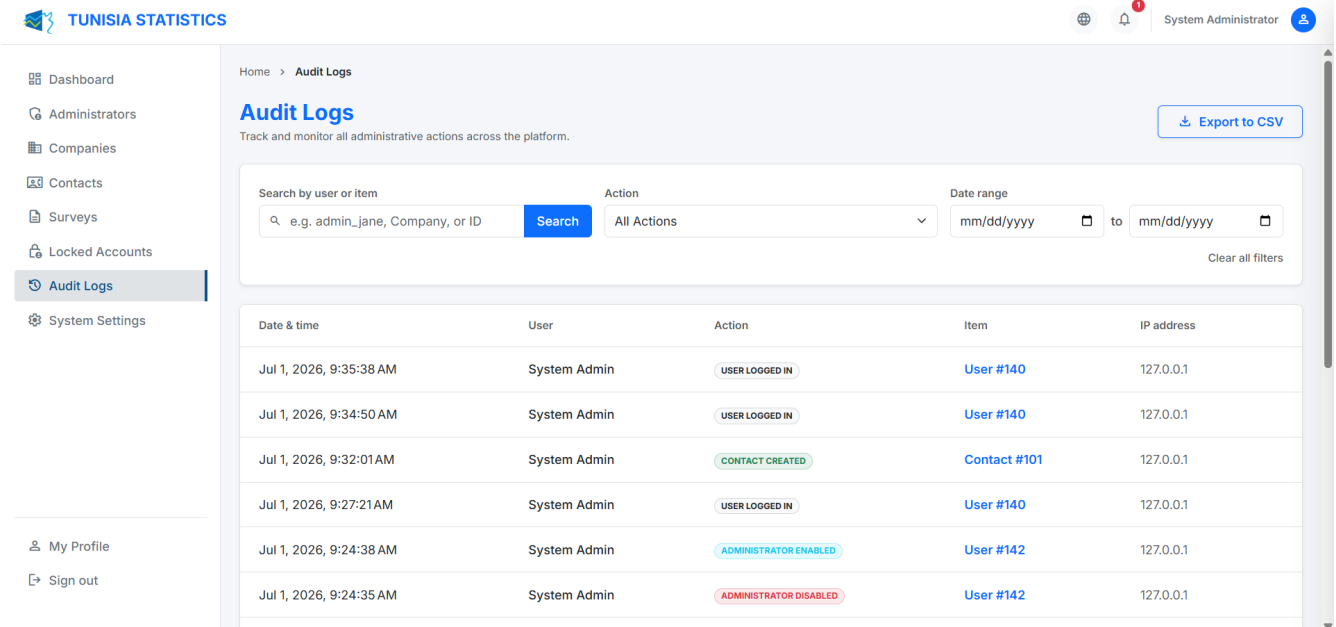


Figure 66: Consult Audit Logs Interface

5.6 Testing and System Validation

System validation was conducted to ensure functional correctness, security enforcement, and performance stability across all platform components.

5.6.1 Test Strategy

The verification strategy comprised two complementary methodologies:

1. **Manual REST API Verification:** Constructing and executing test suites using Postman [12] to validate response codes, payload structures, authentication guards, and error handling for all REST endpoints.
2. **Synthetic Stress Testing:** Developing an automated dataset generator producing high-volume Excel files ($\approx 100\text{MB}$, 500,000+ rows) to stress test the asynchronous Celery parsing pipeline, memory limits, and staging table pagination under adversarial error conditions.

5.6.2 Test Cases and Results

Table 27 details representative test cases executed across functional modules during system verification:

Table 27: System Test Cases and Execution Results

ID	Component	Test Description	Expected Result	Status
TC-01	Authentication	Login with valid credentials	HTTP 200 + Redis OTP sent	PASS
TC-02	Authentication	Submit invalid OTP code	HTTP 400 “Invalid OTP”	PASS
TC-03	Security	3 consecutive failed logins	Account locked in Redis for 24h	PASS
TC-04	Security	Call protected endpoint without JWT	HTTP 401 Unauthorized	PASS
TC-05	Security	Use blacklisted token after logout	HTTP 401 “Token blacklisted”	PASS
TC-06	Admin Directory	Create admin with existing email	HTTP 400 “Email already exists”	PASS
TC-07	RBAC Guard	Account Manager accessing Admin API	HTTP 403 Forbidden	PASS
TC-08	Batch Import	Upload 100MB valid Excel sample	Async session created, rows staged	PASS
TC-09	Batch Import	Upload file with missing columns	Session marked failed with error details	PASS
TC-10	Batch Import	Duplicate tax ID in import file	Per-row validation error in staging	PASS
TC-11	Integration	Resolve valid UUID token	HTTP 200 + Question payload	PASS
TC-12	Integration	Resolve UUID for closed passage	HTTP 400 “Passage closed”	PASS
TC-13	Integration	Resolve invalid UUID token	HTTP 404 “Token not found”	PASS
TC-14	Notifications	Survey submission trigger	Real-time WebSocket delivery	PASS
TC-15	Audit Logs	Export 50,000 audit rows	Background task generates Excel file	PASS

5.6.3 Performance and Stress Test Verification

Stress testing focused on verifying backend resilience when parsing large company sample files. Synchronous upload processing in legacy systems caused main thread blocking and browser timeouts.

Under the new architecture, uploading a 100MB file (500,000 rows) returns an HTTP 202 response immediately with a task session ID. Celery worker threads parse the file in 2,000-row chunks using openpyxl read-only mode, keeping worker memory consumption below 50MB throughout execution. Staging rows are committed in bulk batches, and the Angular frontend polls progress with server-side pagination, maintaining fluid 60fps UI responsiveness.

5.7 Sprint 2 Conclusion

This chapter detailed the development of Sprint 2, which successfully established the real-time notification engine, systemic audit trail, and full system verification suite. All functional requirements across Sprints 0, 1, and 2 have been implemented, integrated, and validated.

General Conclusion

This graduation project, conducted in collaboration with the National Institute of Statistics (INS) Tunisia, designed and implemented a web platform for administrative tracking and statistical survey collection. The platform replaces legacy paper-based workflows with a decoupled architecture, role-segmented administrative controls, asynchronous background processing, and a passwordless external access model.

Development was conducted following the Agile Scrum methodology across three functional sprints:

- **Sprint 0 (Authentication & Organization Directory):** Implemented secure authentication with 2FA OTP verification, Redis-backed rate limiting and account lockout enforcement, and company/contact directory management.
- **Sprint 1 (Survey Management & Response Portal):** Developed survey passage configuration, sample mapping, company contact submission portals, and passwordless UUID token integration.
- **Sprint 2 (Real-Time Notifications, Auditing & Testing):** Built an event notification engine using Redis Pub/Sub and WebSockets, systemic audit logging with asynchronous Excel export via Celery, and performed end-to-end system testing.

From an engineering perspective, the platform decouples UI, business logic, persistence, and background tasks: an **Angular** single-page application frontend, a **FastAPI** backend, a **MySQL** database, and a **Redis + Celery** task worker infrastructure.

Architectural Challenges and Lessons Learned

During system development, key technical challenges were identified and addressed:

1. **Main API Freezing on Heavy File Operations:** Initial tests showed that synchronous parsing of 100MB Excel company samples blocked FastAPI's main event loop, causing API timeouts for concurrent users. This was solved by offloading parsing to background Celery tasks, writing validated rows to a `company_import_staging` table, and serving the frontend via server-side pagination.

2. **Single-Threaded Worker Bottlenecks:** Running Celery with a single thread caused small background tasks to queue behind multi-minute file imports. Increasing concurrency via threaded worker pools (`-P threads -c 4`) allowed small tasks to execute concurrently alongside heavy imports.
3. **Token Revocation on Logout:** JWT tokens are stateless by default. Immediate access revocation was achieved by storing a unique `jti` claim in Redis on logout, checked by the authentication dependency guard on every request.

System Limitations

The current implementation has two main operational limitations:

1. **External Questionnaire Engine Dependency:** The questionnaire filling interface is rendered by an external application linked via UUID tokens. While this decouples survey form rendering, it requires maintaining server-to-server API key synchronization.
2. **Single-Node Redis Broker:** In the development environment, Redis operates as a single instance. Production deployment requires configuring a Redis Sentinel cluster for high availability and failover.

Perspectives and Future Work

Future enhancements planned for the platform include:

1. **Automated CI/CD Pipeline:** Integrating automated GitHub Actions pipelines for running unit test suites and building Docker production containers automatically on code push.
2. **Statistical Anomaly Detection:** Implementing automated data validation rules to flag outlier responses prior to statistical aggregation.
3. **Offline Mobile Collection App:** Developing a Progressive Web App (PWA) for field enumerators collecting offline survey responses in rural regions.

Bibliography

- [1] National Institute of Statistics (INS) Tunisia: <http://www.ins.tn/>, consulted on 20/07/2026.
- [2] Angular Framework: <https://angular.io/>, consulted on 21/07/2026.
- [3] FastAPI Framework: <https://fastapi.tiangolo.com/>, consulted on 21/07/2026.
- [4] Python Language: <https://www.python.org/>, consulted on 22/07/2026.
- [5] Celery Task Queue: <https://docs.celeryq.dev/>, consulted on 22/07/2026.
- [6] Redis Data Store: <https://redis.io/>, consulted on 23/07/2026.
- [7] Docker Container Platform: <https://www.docker.com/>, consulted on 23/07/2026.
- [8] K. Schwaber and J. Sutherland: The Scrum Guide, <https://scrumguides.org/>, consulted on 24/07/2026.
- [9] Visual Studio Code IDE: <https://code.visualstudio.com/>, consulted on 24/07/2026.
- [10] Git Version Control: <https://git-scm.com/>, consulted on 25/07/2026.
- [11] GitHub Repository Hosting: <https://github.com/>, consulted on 25/07/2026.
- [12] Postman API Platform: <https://www.postman.com/>, consulted on 26/07/2026.
- [13] PlantUML Modeling Tool: <https://plantuml.com/>, consulted on 26/07/2026.
- [14] Overleaf Online LaTeX Editor: <https://www.overleaf.com/>, consulted on 27/07/2026.
- [15] MySQL Database System: <https://www.mysql.com/>, consulted on 27/07/2026.
- [16] SQLite Database Engine: <https://www.sqlite.org/>, consulted on 28/07/2026.
- [17] SQLAlchemy ORM: <https://www.sqlalchemy.org/>, consulted on 28/07/2026.

- [18] NGX-Translate Internationalization Library: <https://ngx-translate.org/>, consulted on 29/07/2026.
- [19] Loi n° 99-32 du 13 avril 1999, relative au système national de la statistique, Journal Officiel de la République Tunisienne, https://www.ins.tn/sites/default/files/loi%20n%C2%B099-32%20du%2013avril%201999,%20relatif%20au%20syst%C3%A8me%20national%20de%20la%20statistique%20link%208_0.pdf, consulted on 29/07/2026.
- [20] Open Data Kit (ODK): <https://getodk.org/>, consulted on 30/07/2026.
- [21] KoboToolbox Data Collection Tool: <https://www.kobotoolbox.org/>, consulted on 31/07/2026.
- [22] LimeSurvey Online Survey Tool: <https://www.limesurvey.org/>, consulted on 01/08/2026.
- [23] I. Sommerville: *Software Engineering*, 10th ed., Pearson, 2015.
- [24] R. T. Fielding: *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral dissertation, University of California, Irvine, 2000.
- [25] M. Jones, J. Bradley, N. Sakimura: *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
- [26] E. Gamma, R. Helm, R. Johnson, J. Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [27] P. Leach, M. Mealling, R. Salz: *A Universally Unique Identifier (UUID) URN Namespace*, RFC 4122, IETF, 2005.